Introduction to Large Language Models

Handling Long Sequences, Computational Complexity of Transformers, Fast Attention Mechanisms

Mitesh M. Khapra

AI4Bharat, Department of Computer Science and Engineering, IIT Madras

Motivation

The models that we have discussed so far have a limited context length of 512 or 1024(GPT-2)

What if we want to summarize a book that contains, say 64000 words?

What if we want to generate a story that contains at least 5000 words?

What if we want to find an answer to a question from an instruction manual that contains a thousand pages?

What about other sequences like audio and images (viewing pixels as sequences)?

The sequence length of an audio signal of 10s duration is about 80000 (assuming sampling rate of 8000 Hz)

The sequence length of a colour image of size 100 by 100 is 30000.

So, enabling the models to attend to longer sequences is crucial.

Motivation

The context lengths of some of the recent models are listed below

Model	Context length
LLAMA-2	4K
Mistral-7B	8K
GPT-4, Gemini 1.0	32K
Mosaic ML MPT	65K
Anthropic Claude	100K
Gemini 1.5 pro, GPT-4 Turbo	128K
Claude 2.1	200K
Gemini 1.5 pro (limited)	2 million (for public) 10 million (experimental)

Why is it difficult to scale the context length?

One of the primary factors is the computational complexity of the self-attention mechanism.

Let us first try to understand the computational complexity and memory requirements to train Transformer based models.

Module 1: Time and Space Complexity of Attention Mechanism

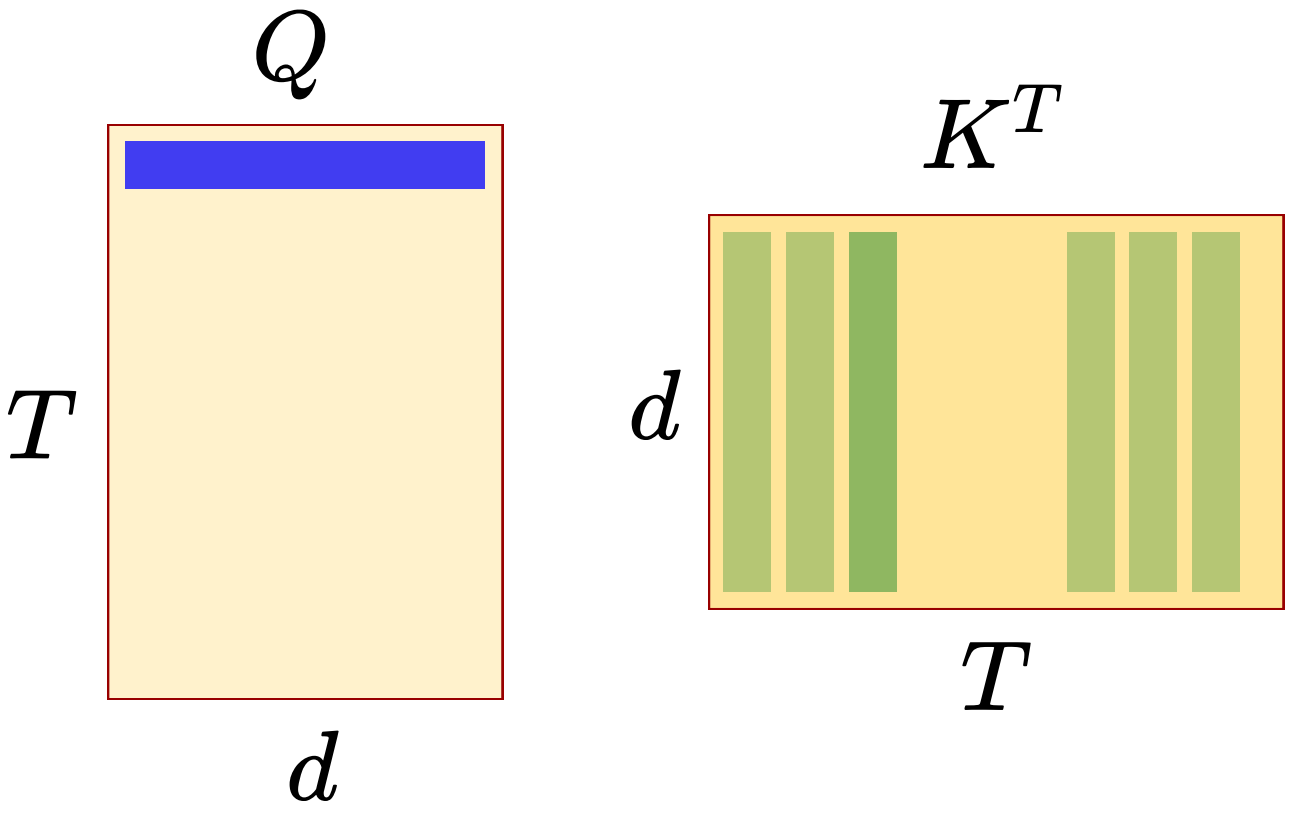
Mitesh M. Khapra

AI4Bharat, Department of Computer Science and Engineering, IIT Madras

Attention Mechanism

Context window size: T

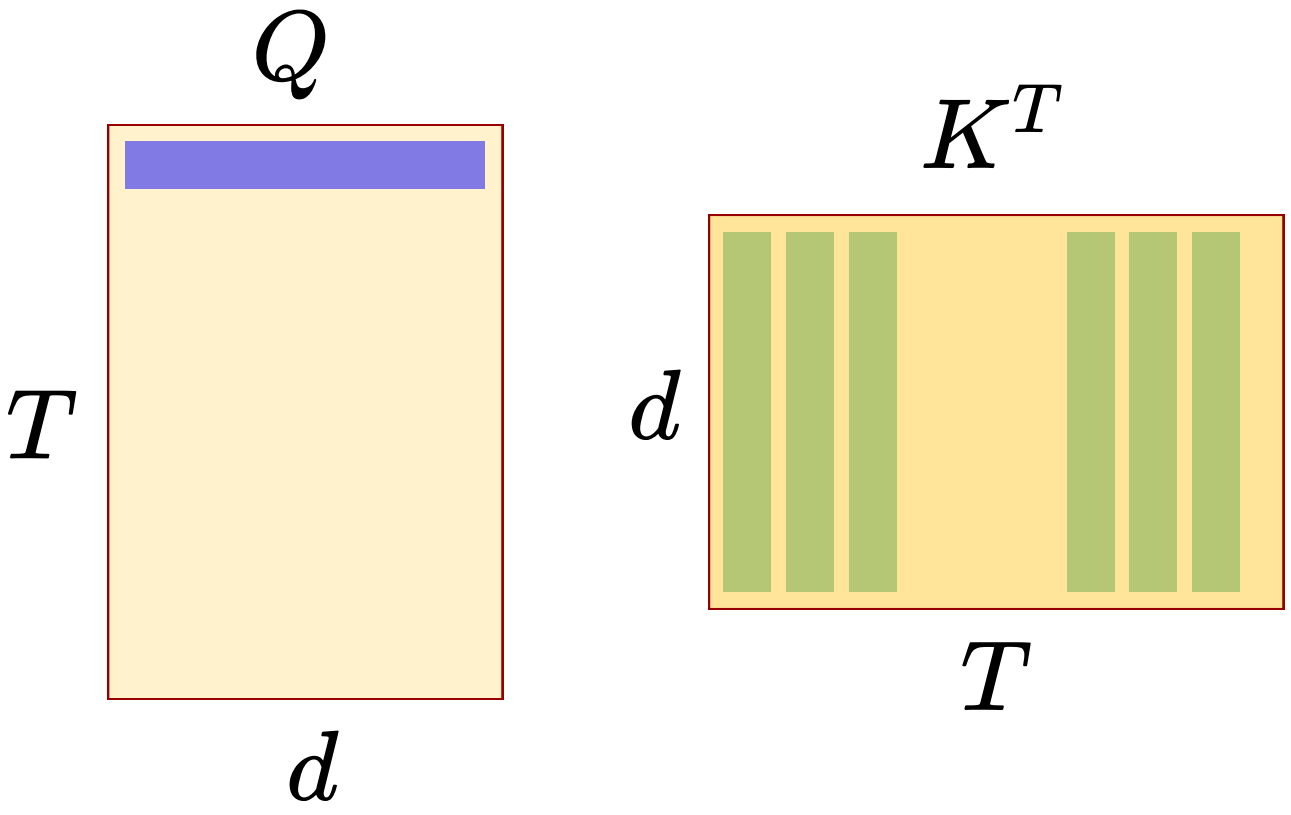
The size of the (projected) embeddings: $d = d_q = d_k$

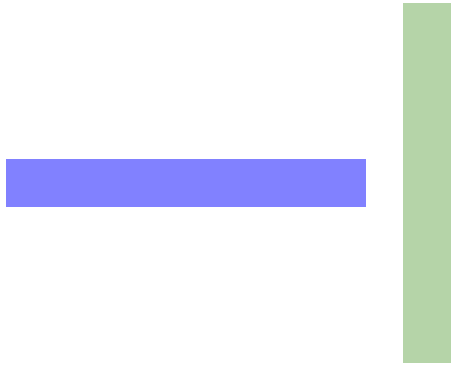


Attention Mechanism

Context window size: T

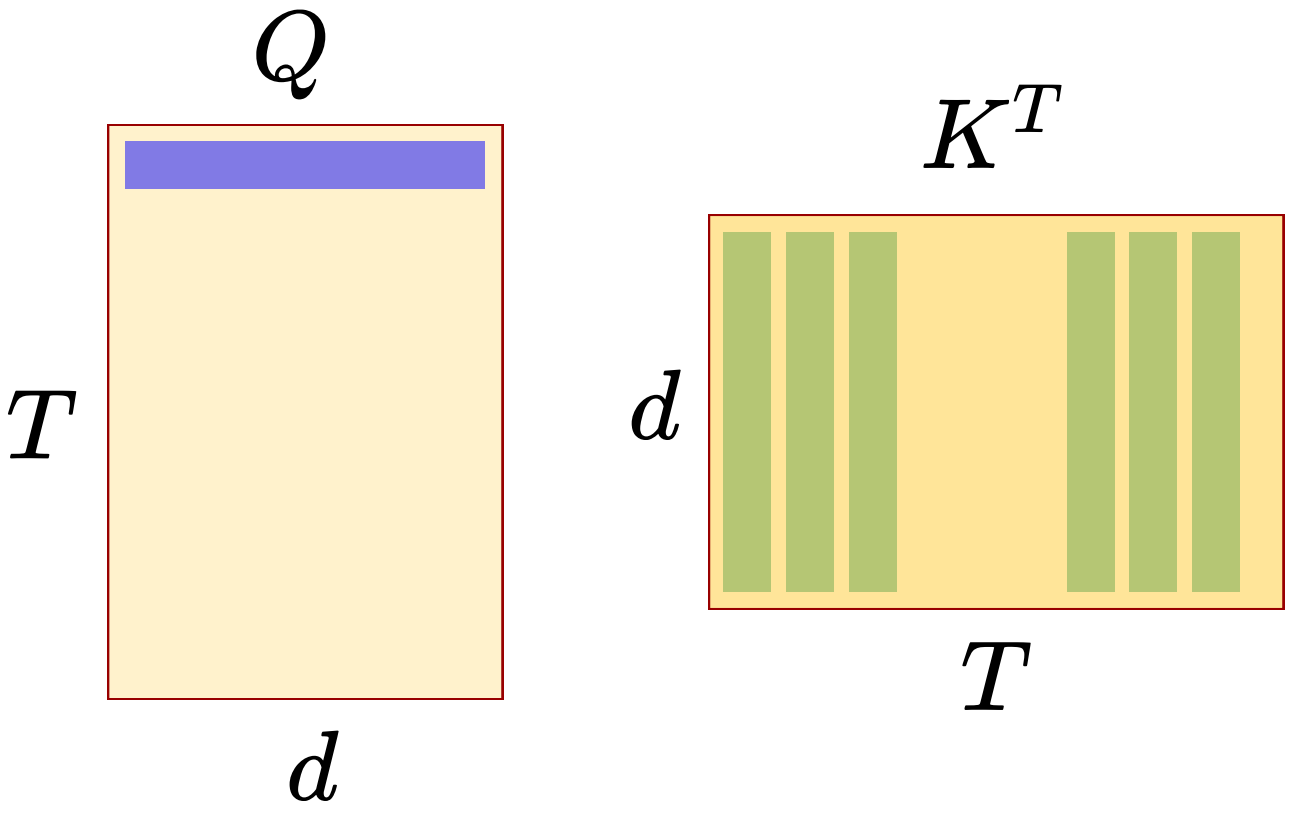
The size of the (projected) embeddings: $d = d_q = d_k$





One dot product between $d_q \cdot d_k^T$ requires d multiplications and $d - 1$ additions

Attention Mechanism



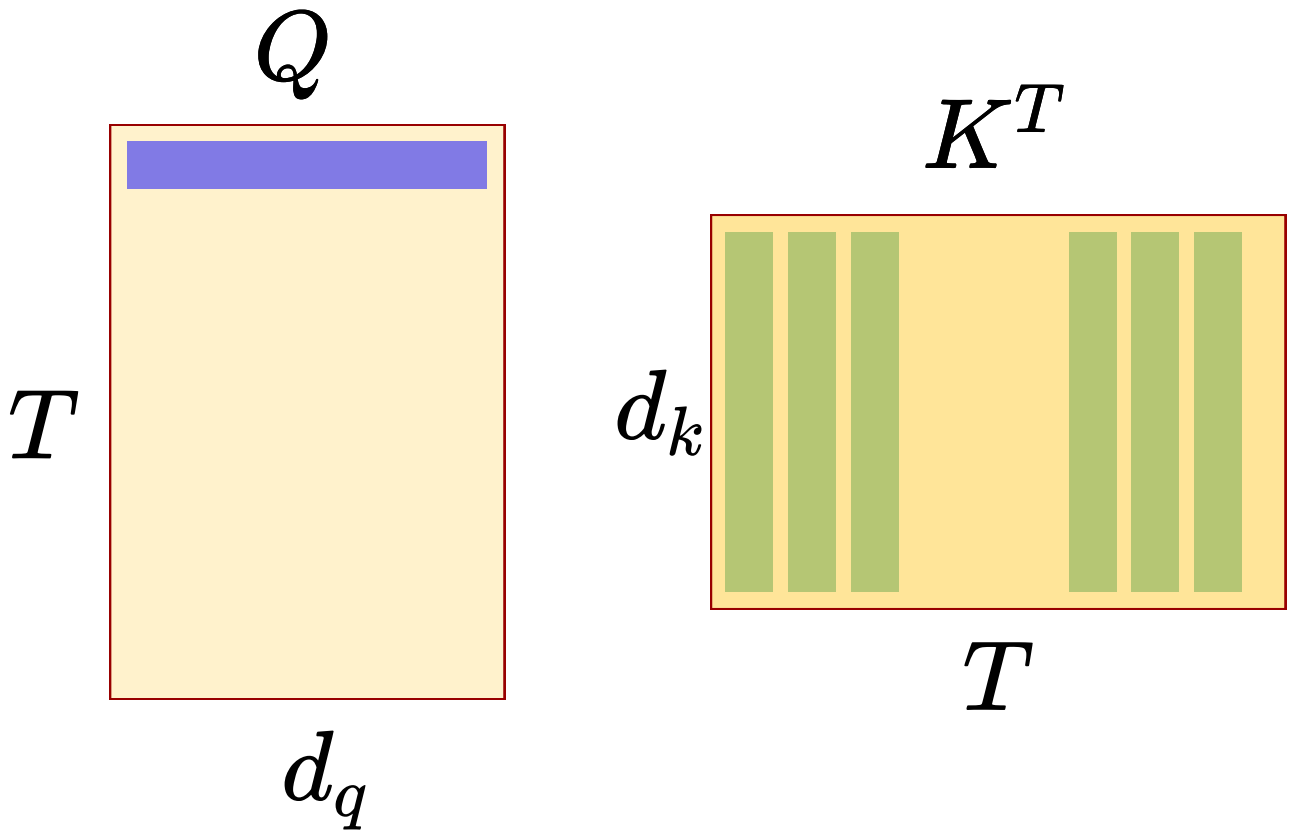
Context window size: T

The size of the (projected) embeddings: $d = d_q = d_k$

Number of operations for computing one row of unnormalized attention weights $Z \in \mathbb{R}^{T \times T}$ is..

$T \cdot d$ multiplications and $T \cdot (d-1)$ additions

Attention Mechanism



Context window size: T

The size of the (projected) embeddings: $d = d_q = d_k$

Number of operations for computing one row of unnormalized attention weights $Z \in \mathbb{R}^{T \times T}$ is:

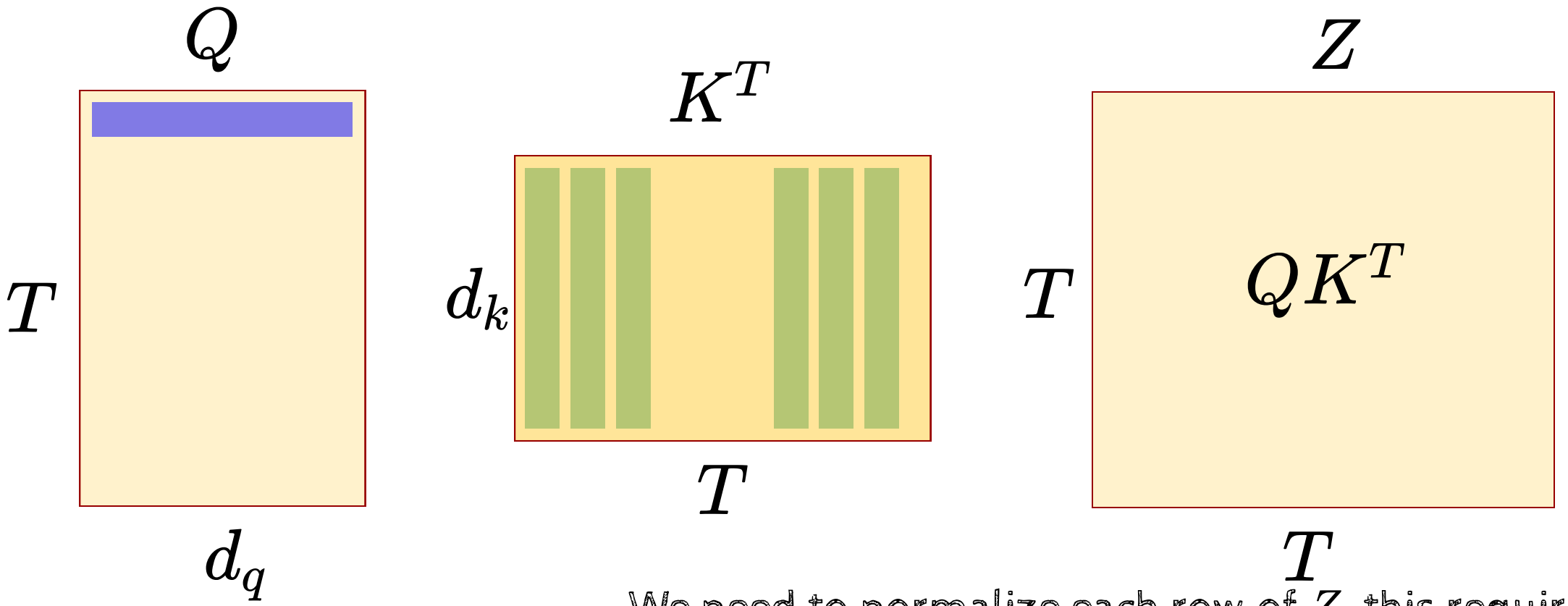
$T \cdot d$ multiplications and $T \cdot (d-1)$ additions

Therefore, to compute QK^T we require $\mathcal{O}(T^2 d)$ operations

The number of tokens T is typically greater than embedding dimension d.

What about computing attention score by normalizing the unnormalized matrix Z ?

Attention Mechanism



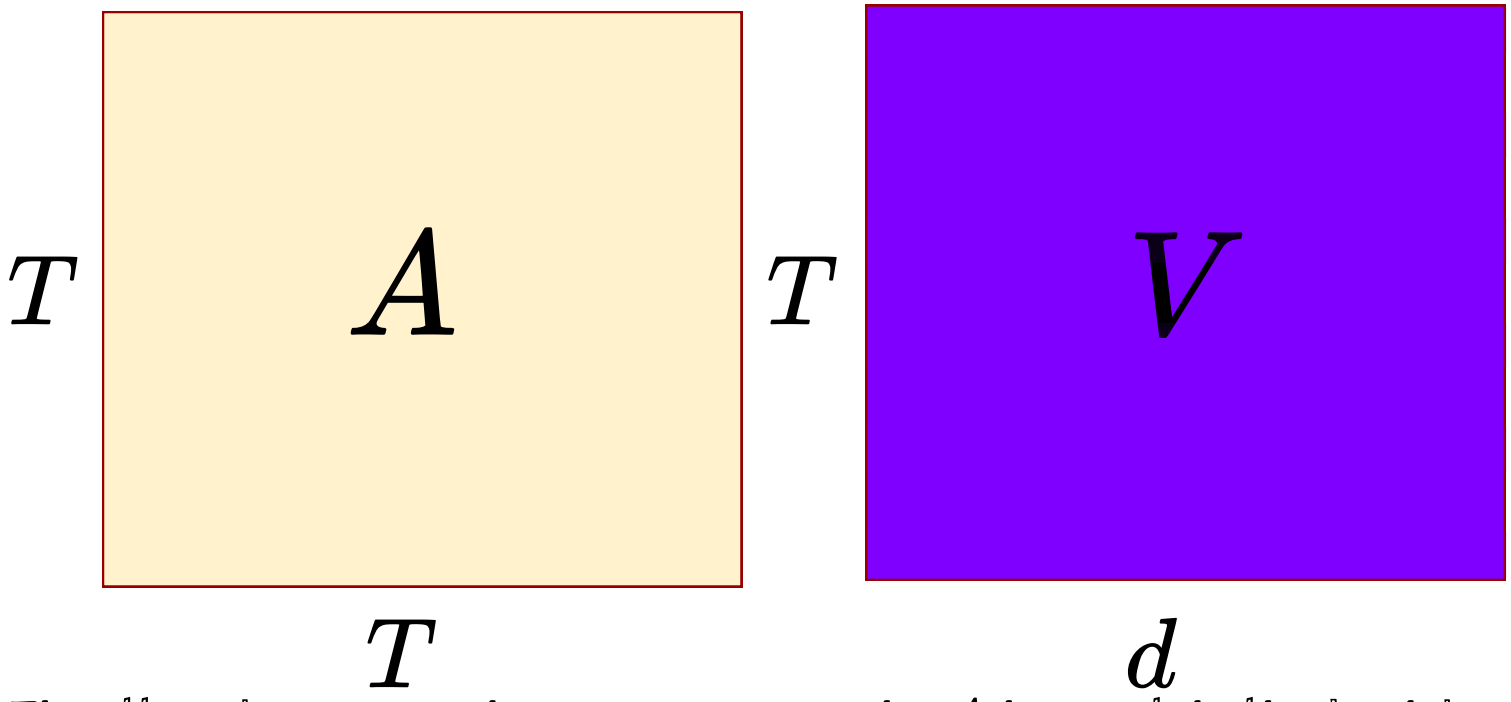
$$A = softmax(\frac{QK^T}{\sqrt{d}})$$

We need to normalize each row of Z, this requires $O(T)$ operations.

There are T rows in Z, therefore computational complexity for $softmax$ is $\mathcal{O}(T^2)$.

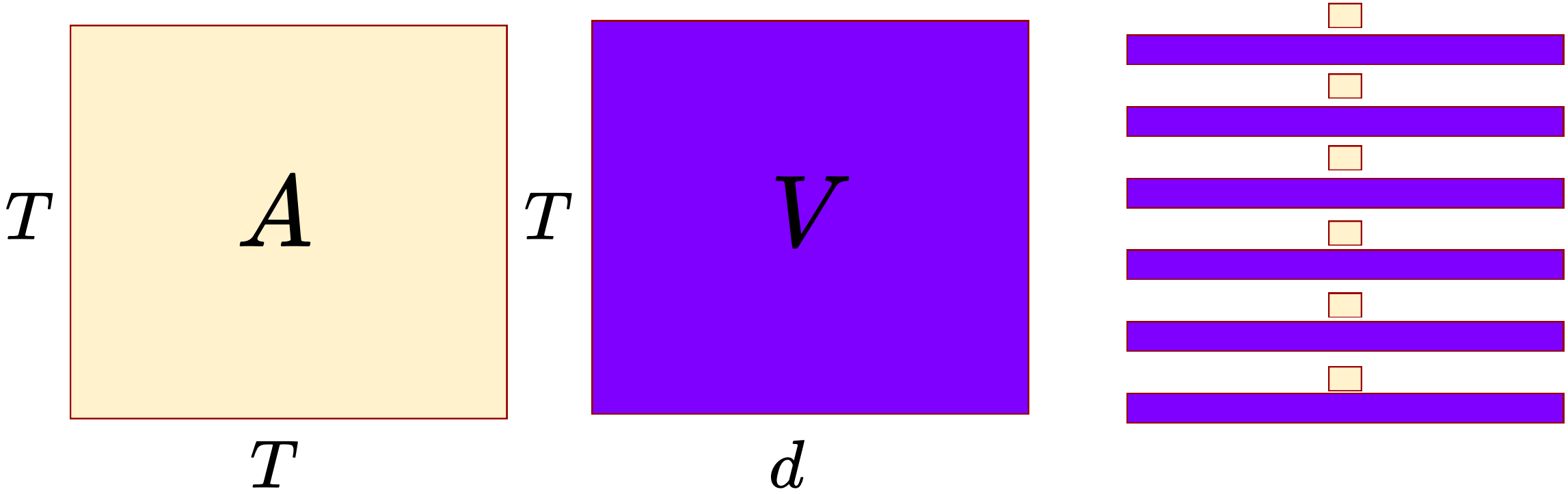
$\therefore$ The computational complexity of A is $\mathcal{O}(T^2 d)$.

Attention Mechanism



Finally, the attention score matrix A is multiplied with the value matrix V

Attention Mechanism



Finally, the attention score matrix A is multiplied with the value matrix V

This has a complexity of $O(T^2 d)$

Therefore, the entire self-attention block is quadratic $\mathcal{O}(T^2 d)$ in the length of the input sequence.

Module 2: Memory Requirements

Mitesh M. Khapra

AI4Bharat, Department of Computer Science and Engineering, IIT Madras

Notations

B: Batch Size

$|\mathcal{V}|$: Number of tokens in the vocabulary

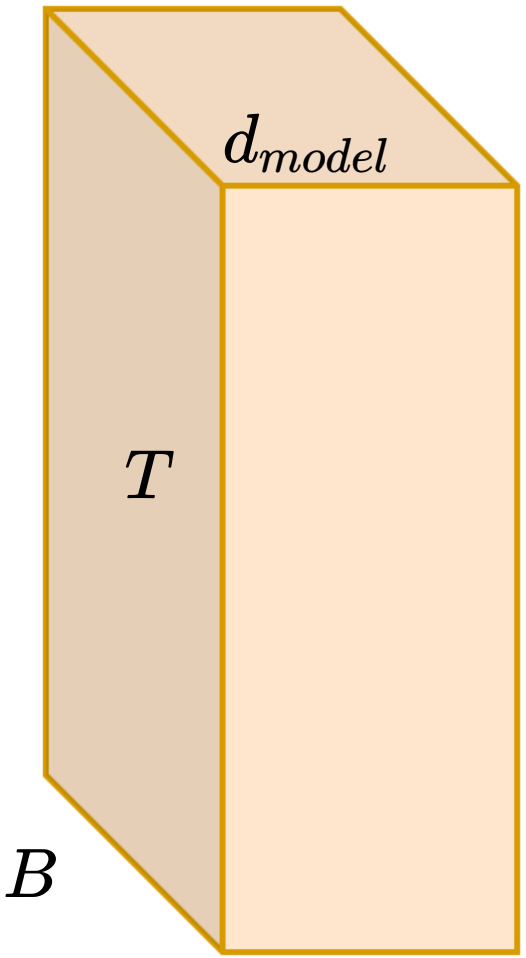
T : Context (sequence) Length

d_{ff} : Dimension of feed-forward layer

d_{model} : Dimension of input embedding, $d_{att} = 512 = \frac{d_{ff}}{4}$

n_h : Number of attention heads

N : Number of parameters in the model



Memory for Attention

For a single sample

$$
\begin{aligned}
mha &= (Q_{T\times d} + K^T_{d\times T} + V_{T\times d})n_h \\
&= 3n_h(T \times d) \\
&= 3T \times n_h d \\
&= 3T \times d_{model} \qquad \because n_h d = d_{model}
\end{aligned}
$$

$$
\begin{aligned}
Z &= QK^T \\
&= Bn_hT^2 \\
A &= softmax(QK^T) \\
&= Bn_hT^2
\end{aligned}
$$

For a batch of B samples

$$= 3BTd_{model}$$

Memory scales up linearly with respect to batch size B and quadratically with respect to the length of the context window T.

Key Take Aways

The complexity of Self-attention is quadratic in the length of the input sequence.

Memory scales up linearly with respect to batch size B and quadratically with respect to the length of the context window T.

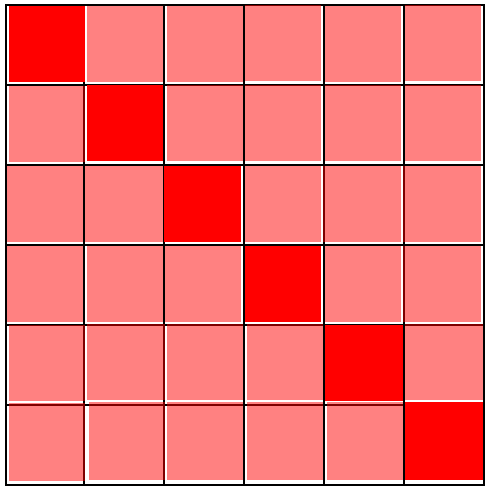
So, the next question is, How do we reduce the time complexity to enable models to handle long sequences (during both training and inference)?

Module 3: Fast Attention Mechanisms

Mitesh M. Khapra

AI4Bharat, Department of Computer Science and Engineering, IIT Madras

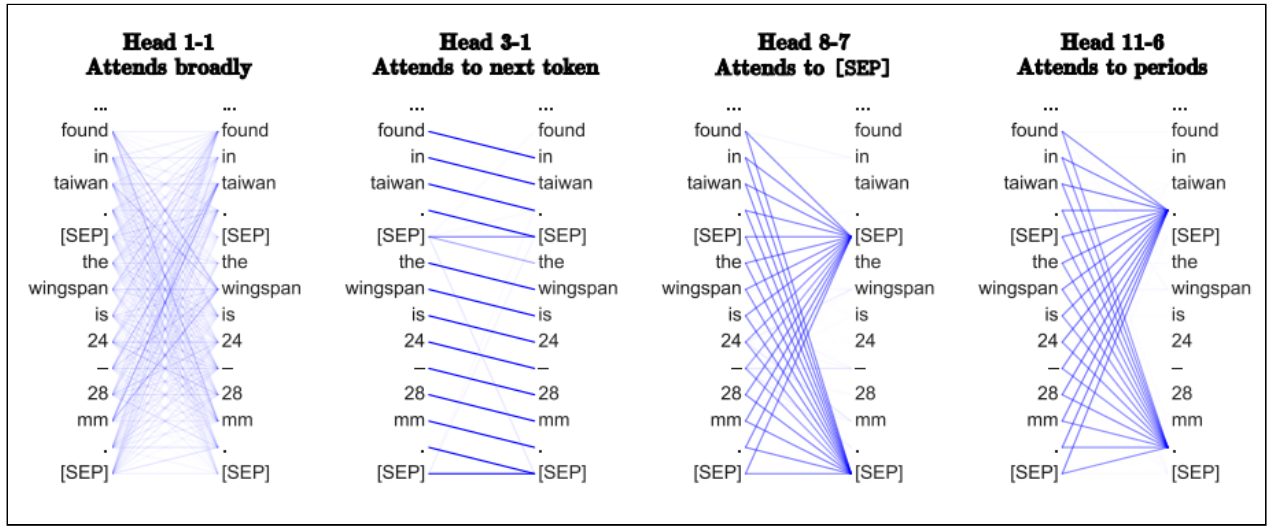
Objective



$O(T^2d)$

What we have is the full attention mechanism with $O(T^2d)$ complexity. What we want is sub-quadratic complexity $O(cTd)$. How do we achieve that?

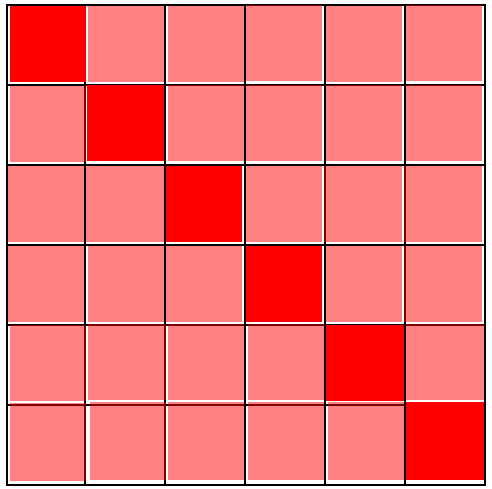
A study on BERT [Ref] in NLP tasks concluded that the neighbouring inner products are extremely important in self-attention and not all heads attend to all tokens.



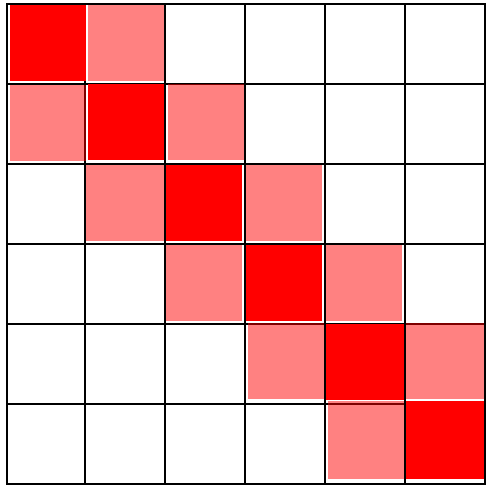
Well, we can localize the attention in multiple ways.

This gives rise to a plethora of approaches that approximate the full attention.

Approaches

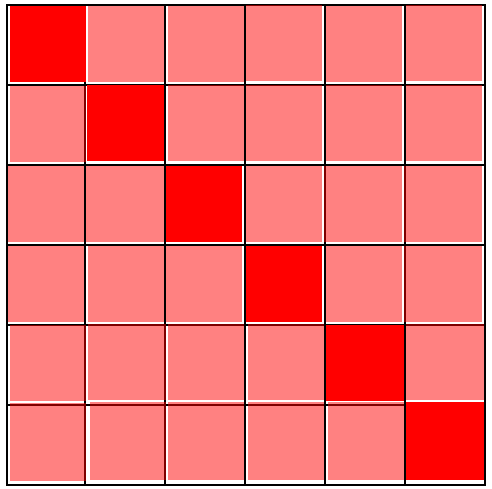


$O(T^2 d)$

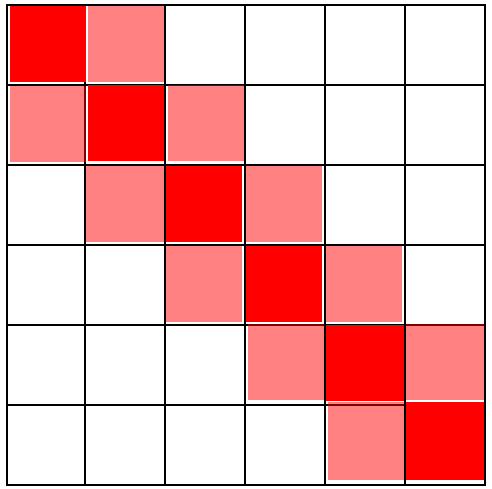


Strided Local Attention

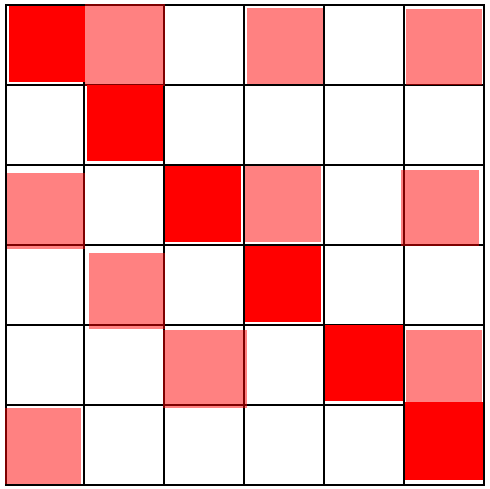
Approaches



$O(T^2 d)$

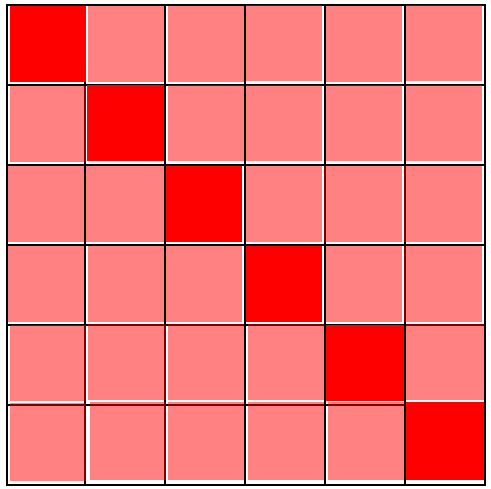


Strided Local Attention

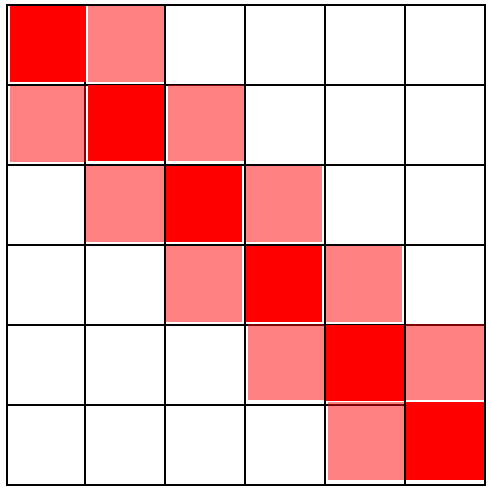


Random Local Attention

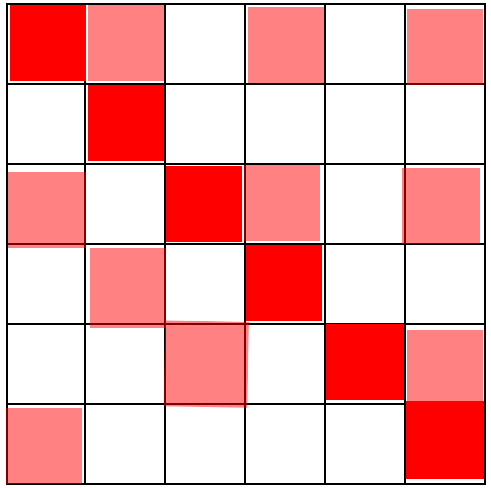
Approaches



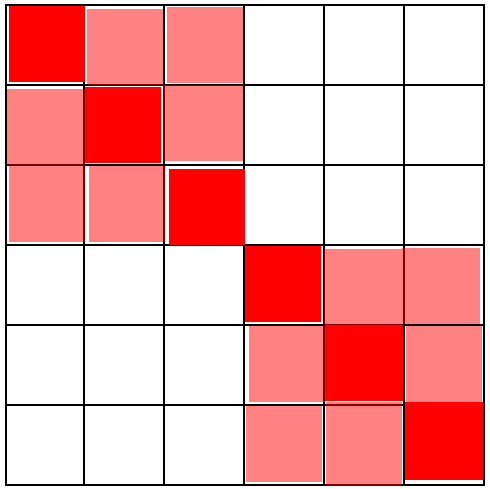
$O(T^2 d)$



Strided Local Attention

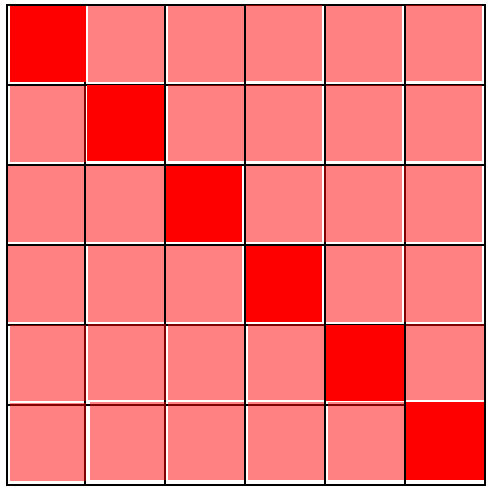


Random Local Attention

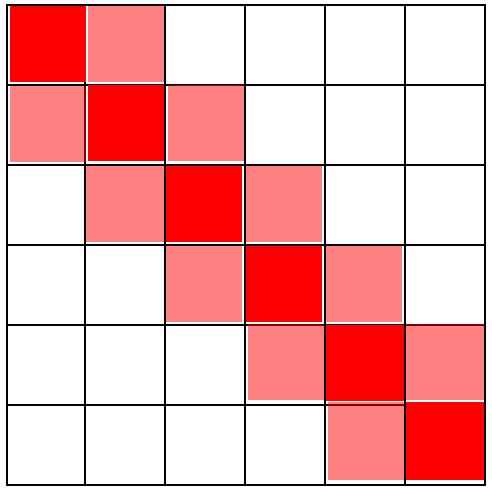


Sparse Block Attention

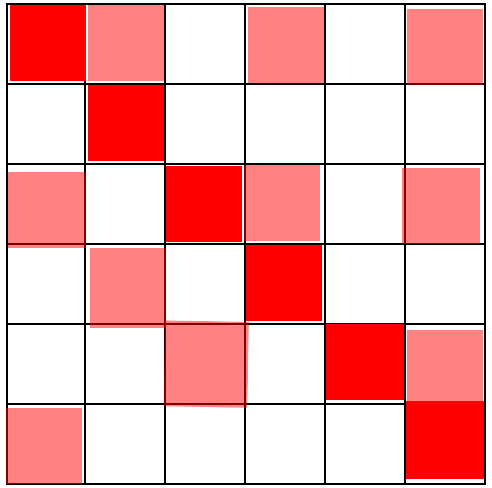
Approaches



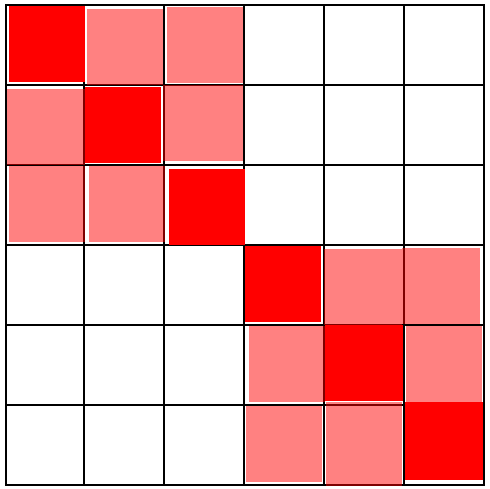
$O(T^2 d)$



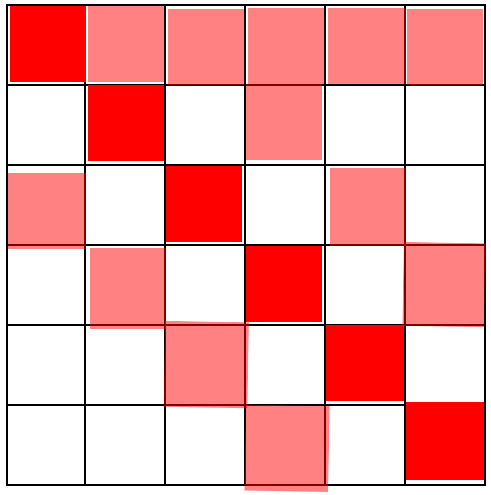
Strided Local Attention



Random Local Attention

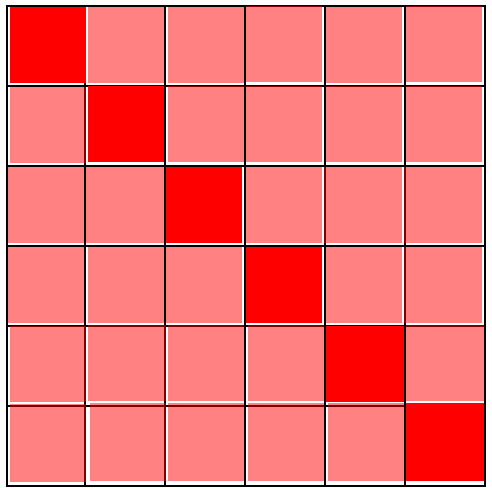


Sparse Block Attention

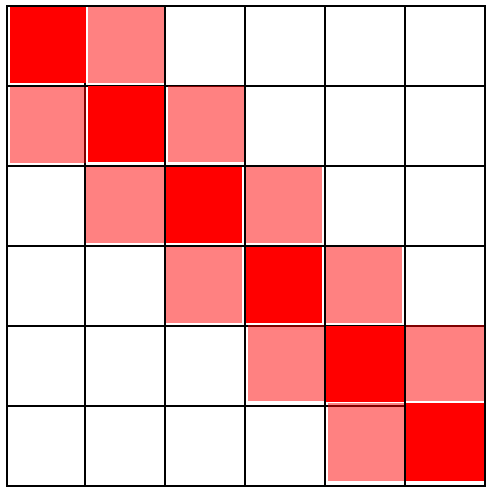


Global+local attention

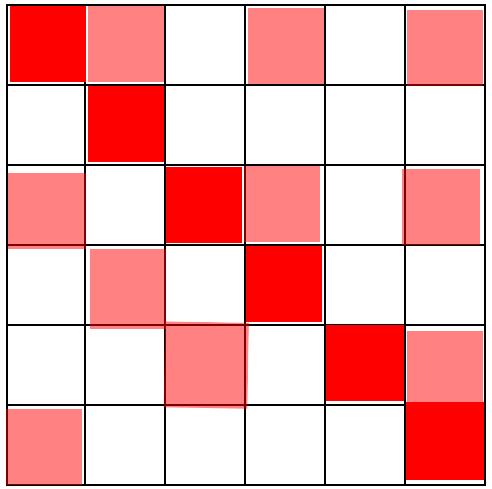
Approaches



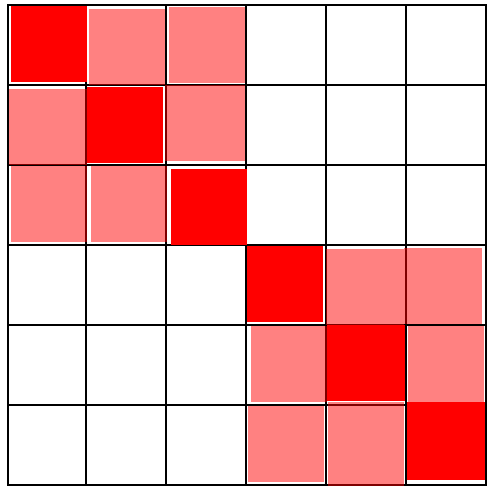
$O(T^2 d)$



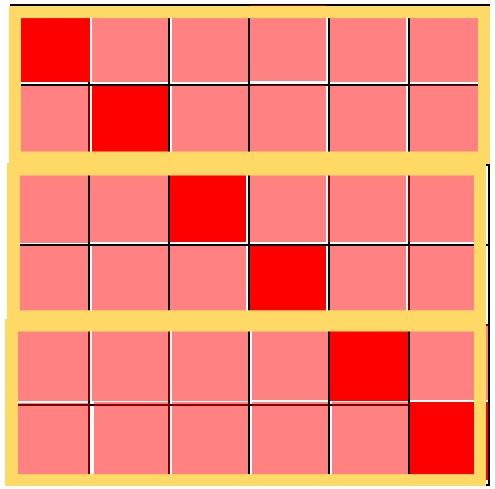
Strided Local Attention



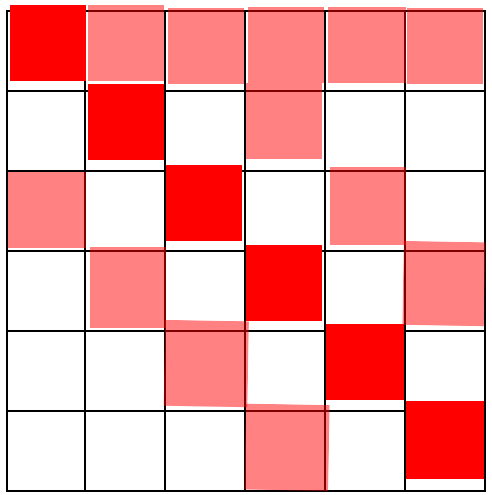
Random Local Attention



Sparse Block Attention



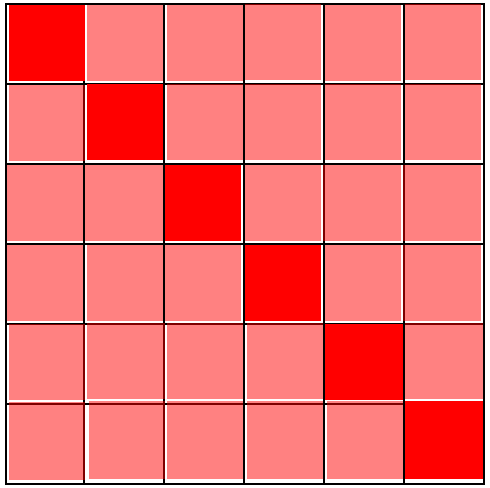
Flash Attention



Global+local attention

Full Attention

QK^T



(the score for the word itself ($q_i k_i^T$))

Computational complexity $O(T^2 d)$

Memory complexity $O(T^2)$

Our focus is on reducing the computational complexity.

Every word attends to every other word in the input sequence

Our planet is a lonely speck

Our planet is a lonely speck

⋮

Our planet is a lonely speck

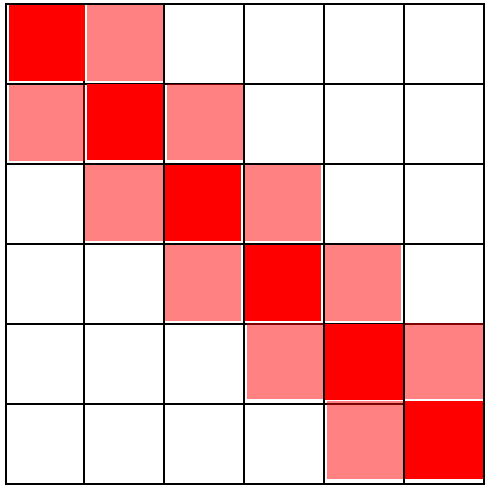
How do we reduce the computational complexity?

One way is to make it sparse by attending to a subset of words instead of all words

$O(c \times T \times d)$ where $c < T$

Strided Local Attention

QK^T



$c = 3$

Every word attends to a window of c words ($\lfloor \frac{c}{2} \rfloor$ to the left and $\lfloor \frac{c}{2} \rfloor$ words to the right)

Our planet is a lonely speck

Our planet is a lonely speck

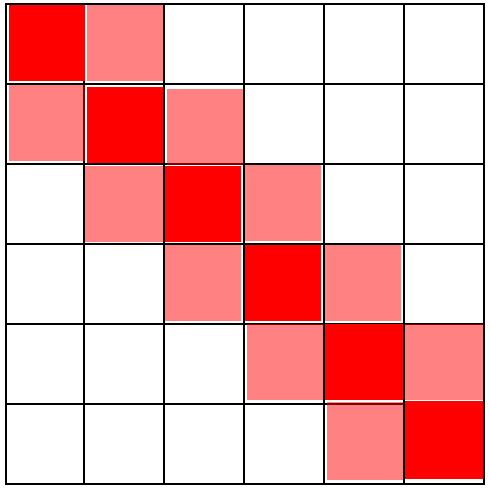
⋮

Our planet is a lonely speck

Linear Computational complexity $O(cTd)$

Strided Local Attention

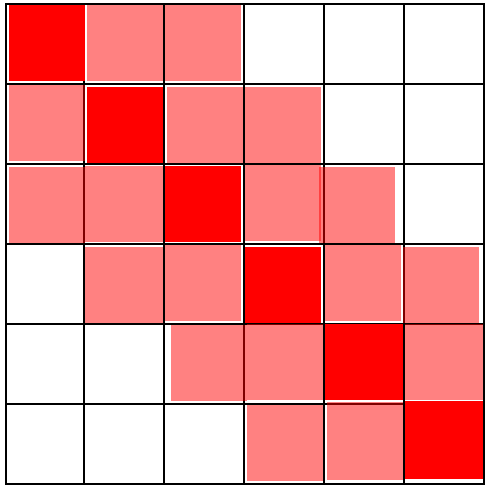
QK^T



$c = 3$

$O(3Td)$

QK^T



$c = 5$

$O(5Td)$

Every word attends to a window of c words ($\lfloor \frac{c}{2} \rfloor$ to the left and $\lfloor \frac{c}{2} \rfloor$ words to the right)

Our planet is a lonely speck

Our planet is a lonely speck

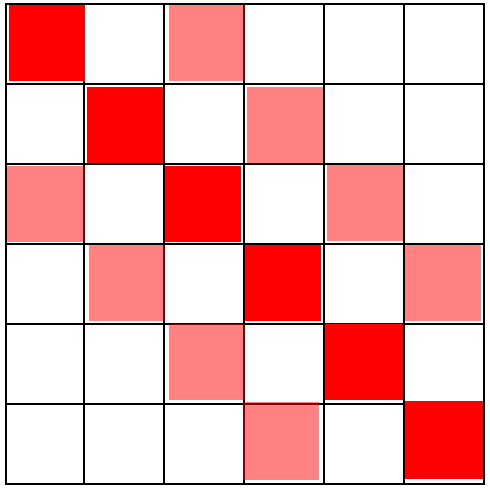
⋮

Our planet is a lonely speck

We could vary the size of the window for each layer in the model such that the top most layer uses the full (global) attention

Dilated Attention

QK^T



$c = 3$

$O(3Td)$

Every word attends to a window of c words ($\lfloor \frac{c}{2} \rfloor$ to the left and $\lfloor \frac{c}{2} \rfloor$ words to the right)

Our planet is a lonely speck

Our planet is a lonely speck

⋮

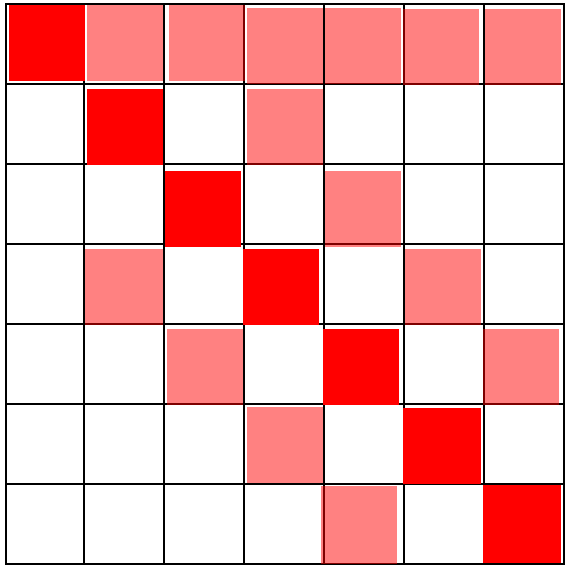
Our planet is a lonely speck

We could also use dilated attention (like dilated convolutions) by using different dilation rates in different attention heads.

Note that, there is no change in computational complexity.

Global + local Attention

$$QK^T$$



$$c = 3$$

$$O((3T + T)d)$$

A few words are global, every other word attends to a window of c words ($\lfloor \frac{c}{2} \rfloor$ to the left and $\lfloor \frac{c}{2} \rfloor$ words to the right).

[cls] Our planet is a lonely speck

Our planet is a lonely speck

Our planet is a lonely speck

⋮

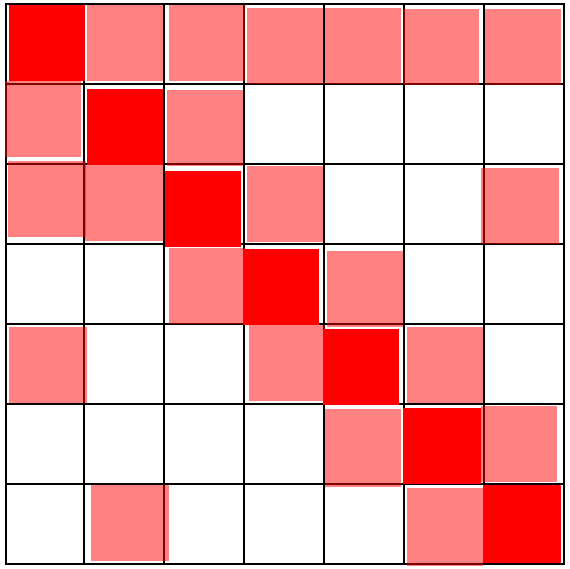
Our planet is a lonely speck

Tokens like [cls] requires full attention to get good performance in downstream tasks.

We can manually set tokens that require global attention based on the task.

Global + Random+ local Attention

$$QK^T$$



$$c = 3$$

$$O((3T + T + T)d)$$

The BIGBIRD model proposes adding random tokens.

A few words are global, every other word attends to a window of c words ($\lfloor \frac{c}{2} \rfloor$ to the left and $\lfloor \frac{c}{2} \rfloor$ words to the right).

[cls] Our planet is a lonely speck

Our planet is a lonely speck

Our planet is a lonely speck

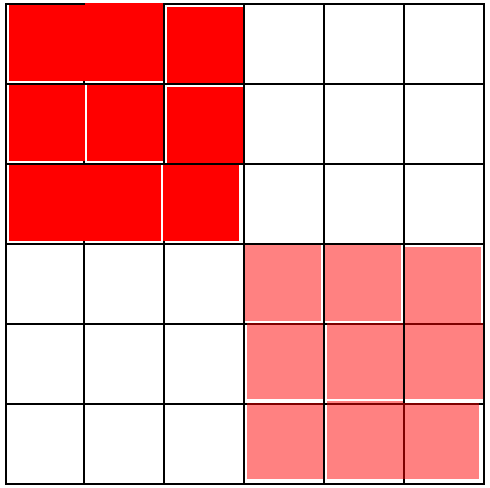
⋮

Our planet is a lonely speck

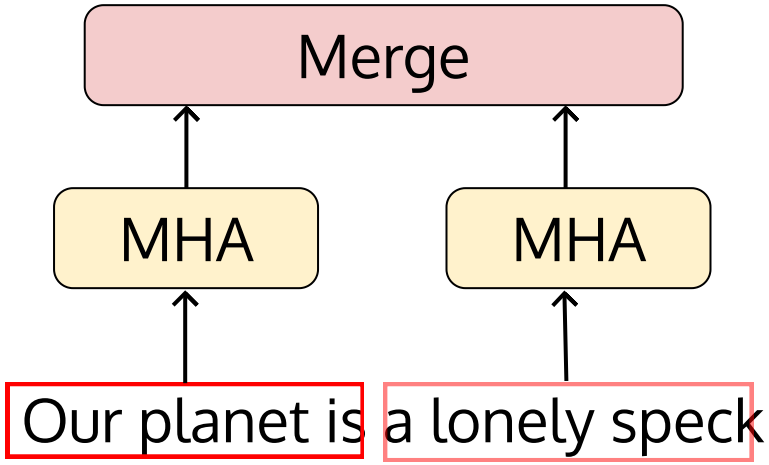
Tokens like [cls] requires full attention to get a good performance in downstream tasks.

Local Block Attention

QK^T



Computational complexity $O(\frac{T^2}{2})$



Another way is to divide the sequence into n blocks and do the computations independently as in [Ref]

Typically, $n = 2, 3, 4$. Therefore, for a context length of 512, the block sizes will be 256, 128, 64.

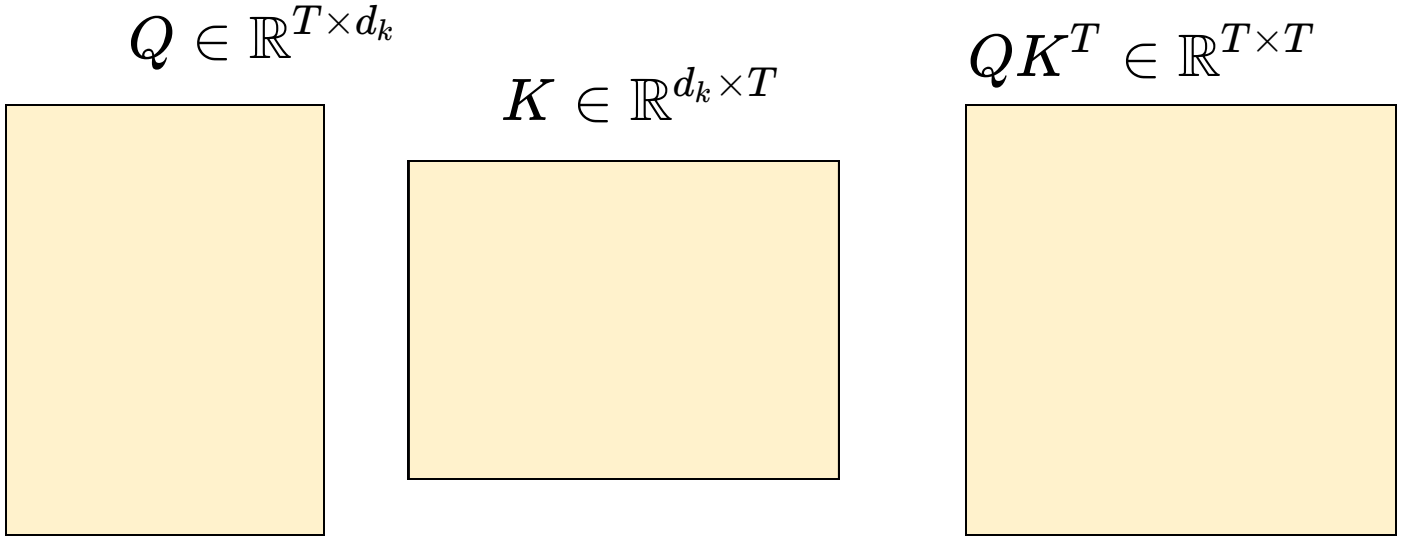
These are resonable context sizes for majority of NLP tasks.

We could also use different patterns in each attention head/layer.

Suppose we have $n = 4$ blocks. Then we can permute these blocks in 16 possible ways (how? will see soon)

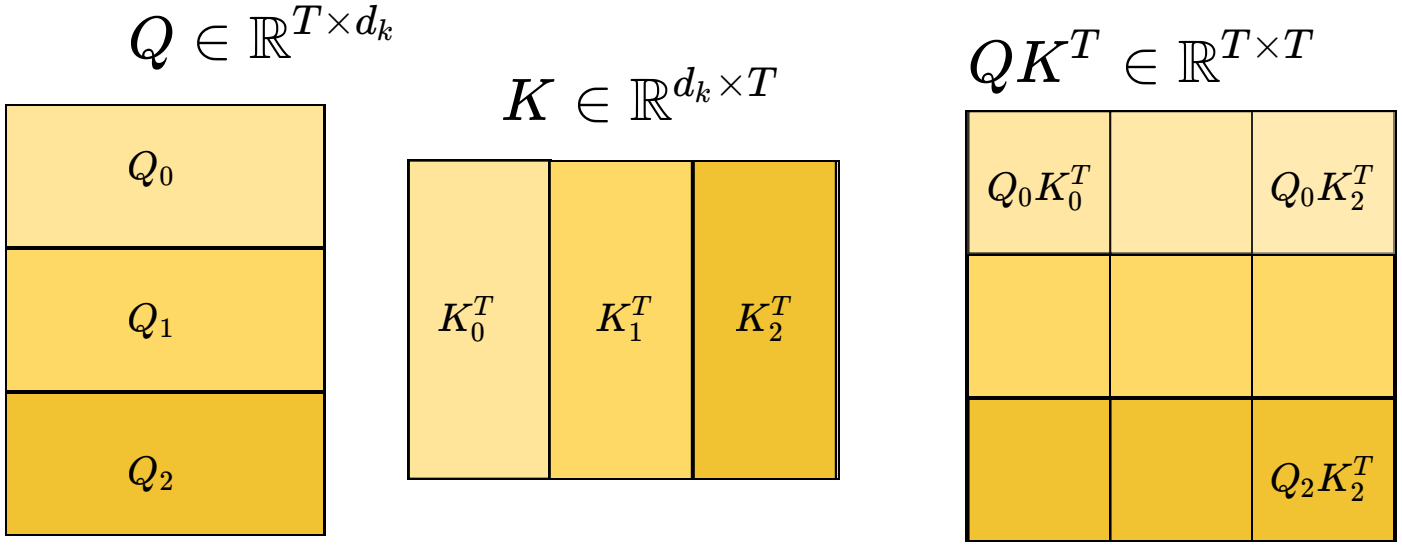
Therefore, we can permute these blocks in each attention head so that it could effectively capture long range dependencies.

Full Attention



Let's recall how we compute the attention matrix

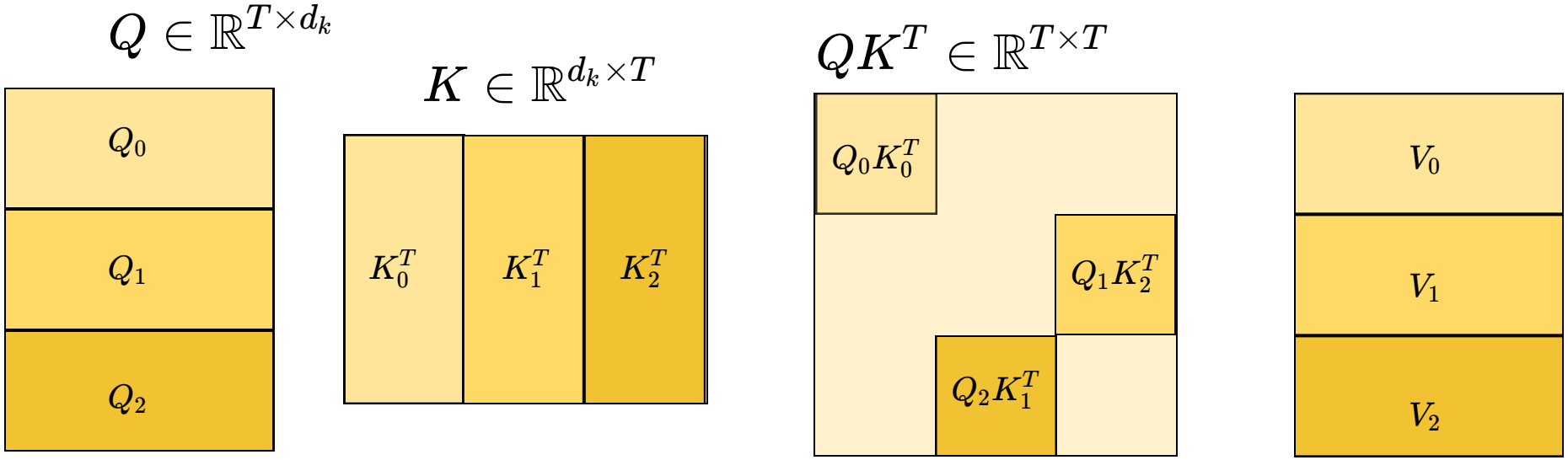
Full Attention



Let's recall how we compute the attention matrix.

We can divide Q and K matrices into n blocks each. This will result in n^2 blocks in QK^T as shown in the figure above.

Block Attention

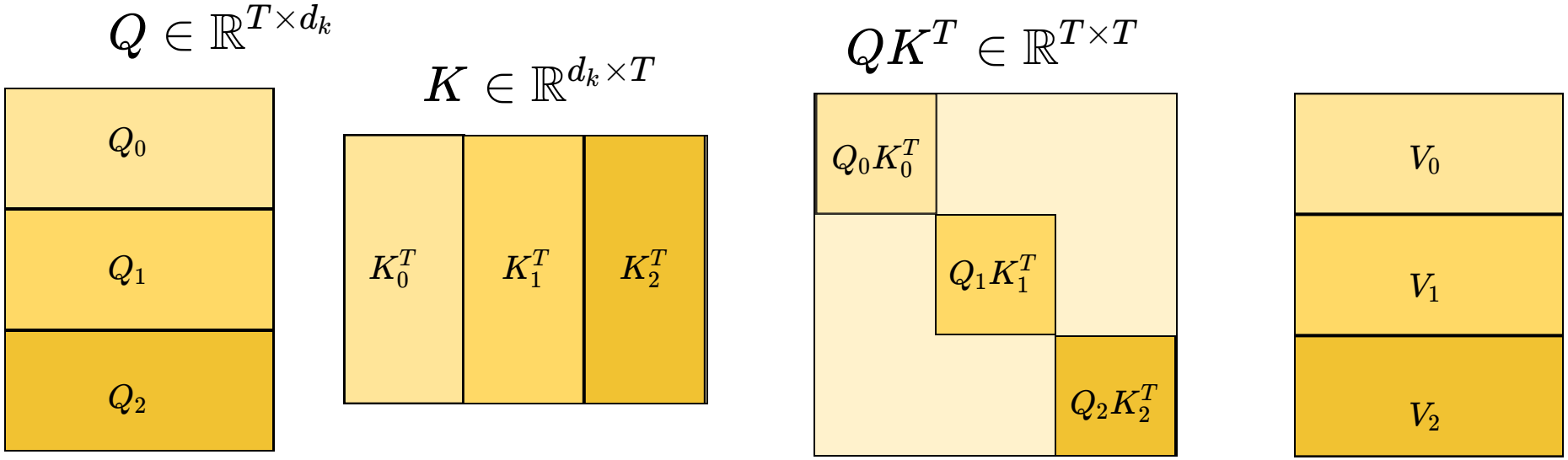


Let's recall how we compute the attention matrix.

We can divide Q and K matrices into n blocks each. This will result in n^2 blocks in QK^T as shown in the figure above.

Suppose we want to select only 3 blocks.

Block Attention



Let's recall how we compute the attention matrix

We can divide the each Q and K matrices into n blocks, this will result in n^2 blocks in QK^T as shown in the figure above

Suppose we want to select only 3 blocks.

There are many ways of selecting 3 blocks from the set of $B = 9$ blocks. Let's formalize this.

Block Attention

Consider the permutation,

$$\pi = \text{perm}(0, 1, 2, \cdots, n-1)$$

Let the k-th element of π be $\pi(k)$, We define the masking matrix M of size $T \times T$

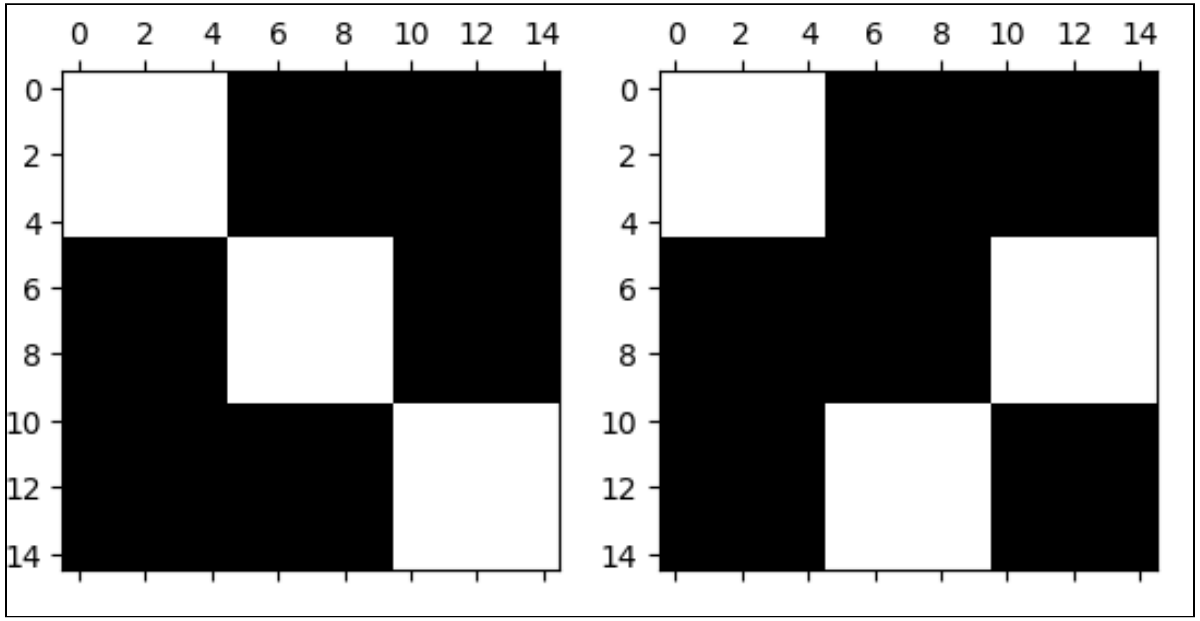
$$M_{ij} = \begin{cases} 1 & \text{if} \quad \pi(\lfloor \frac{in}{T} \rfloor) = \lfloor \frac{jn}{T} \rfloor \\ 0 & \text{otherwise} \end{cases}$$

Block Attention is given by,

$$Z = softmax(QK^T \odot M)$$

For example, suppose context length $T = 15$ and the number of blocks $n = 3$.

Then we would have 3! ways of permuting the blocks.



$Identity: \ \pi = (0, 1, 2)$ $\quad n = 3, T = 15$

$\pi = (0, 2, 1)$ $\quad n = 3, T = 15$

Write Q,K^T, and V into $n = 3$ block matrices (following the blocks in M)

$$\mathbf{Q} = (Q_0, Q_1, Q_2) \in \mathbb{R}^{5 \times d}$$

$$\mathbf{K} = (K_{\pi(0)}, K_{\pi(1)}, K_{\pi(2)}) \in \mathbb{R}^{5 \times d}$$

$$\mathbf{V} = (V_{\pi(0)}, V_{\pi(1)}, V_{\pi(2)}) \in \mathbb{R}^{5 \times d}$$

Computational Complexity

Using these we can compute Block Attention as follows

$$\text{Block-wise Attention}(Q, K, V, M) = \begin{bmatrix} softmax(Q_0 K_{\pi(0)}^T) V_{\pi(0)} \\ \vdots \\ softmax(Q_n K_{\pi(n-1)}^T) V_{\pi(n-1)} \end{bmatrix} \begin{matrix} \rightarrow \frac{T}{n} \times \frac{T}{n} \times d \\ \\ \rightarrow \frac{T}{n} \times \frac{T}{n} \times d \end{matrix}$$

$$= O\left(\frac{T}{n} \times \frac{T}{n} \times n \times d\right)$$

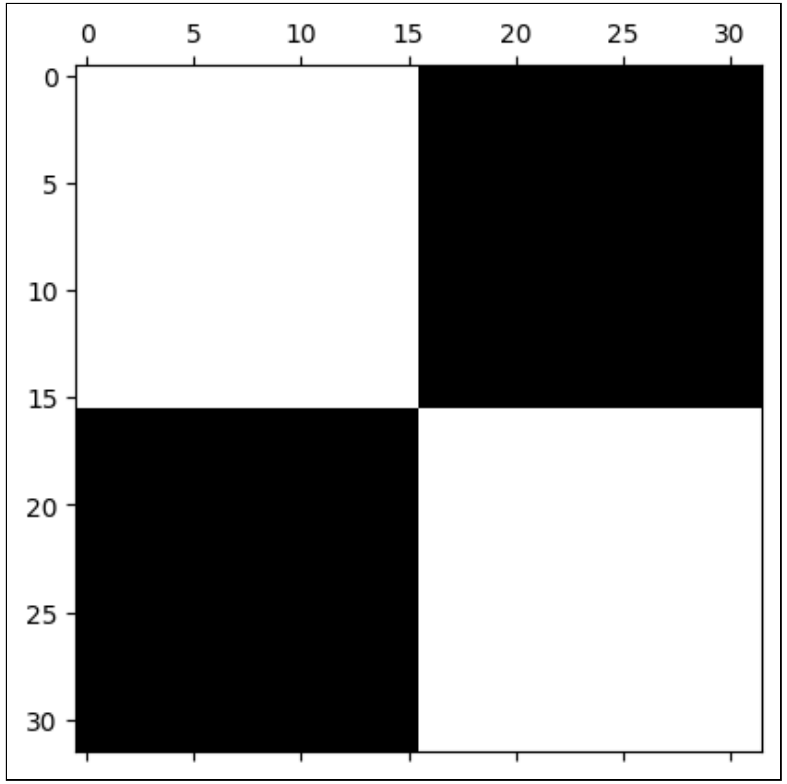
$$= O\left(\frac{T^2 d}{n}\right)$$

We can extend the same concept to Multi-head attention by allowing each head to use a different masking matrix (derived from the permutation function).

This would be called Blockwise Multi-Head Attention.

What happens if we increase value of n?

$T = 32, n = 2$

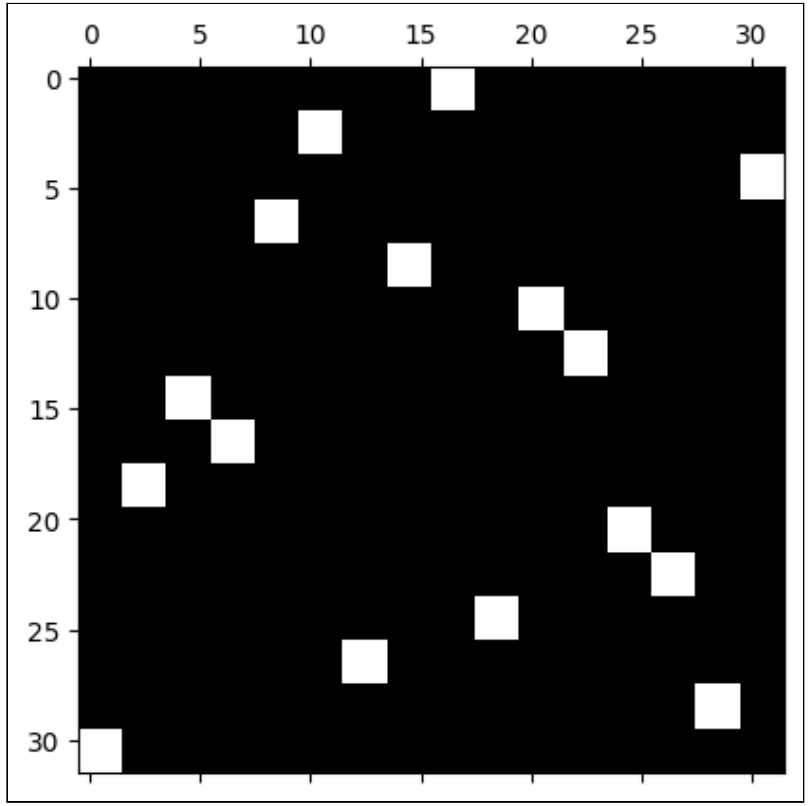


$Identity : (0, 1)$

50% non-zero values

(50% sparse)

$T = 32, n = 16$



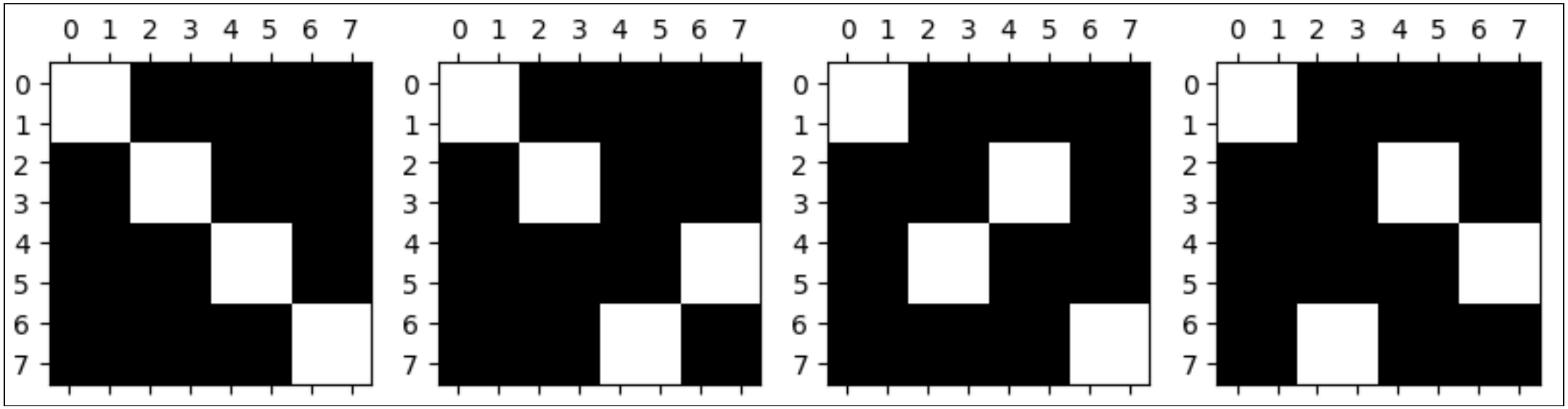
$(8, 5, 15, 4, 7, 10, 11, 2, 3, 1, 12, 13, 9, 6, 14, 0)$

6.25% non-zero values

(93% sparse)

Capturing Long Term dependency

Blockwise sparsity captures both local and long-distance dependencies in a memory-efficient way using different permutations



Empirically, it is observed that the identity permutation is more important than other permutations

Performance

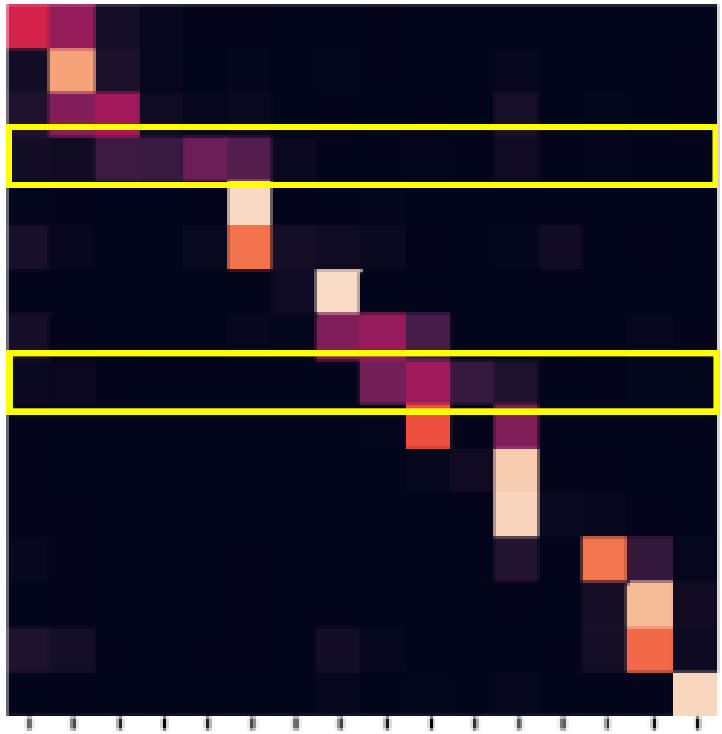
N	Model	Training Time (day)	Memory (per GPU, GB)	Heads Config.	Valid. ppl
512	RoBERTa-1seq	6.62	9.73	-	3.58
	BlockBERT n=2	5.83 (-12.0%)	7.91 (-18.7%)	10:2	3.56
	BlockBERT n=3	5.80 (-12.5%)	7.32 (-23.8%)	8:2:2	3.71
1024	RoBERTa-1seq	9.66	13.39	-	3.60
	BlockBERT n=2	7.51 (-22.3%)	9.73 (-27.3%)	9:3	3.57
	BlockBERT n=3	7.23 (-25.1%)	8.55 (-36.1%)	8:2:2	3.63

Using blockwise sparse attention reduces the training time significantly for longer sequences.

Note, however, that the perplexity score increases as the number of blocks n increases.

This is reasonable given that local attention is just an approximation to full attention. (This is the case for all local attention schemes.)

Locality Sensitive Hashing [Paper]



The attention matrix is mostly sparse !

Let us pick up two rows in the attention matrix show on the left

For the given query q_i, we know that the attention score is higher for some of the keys k_j

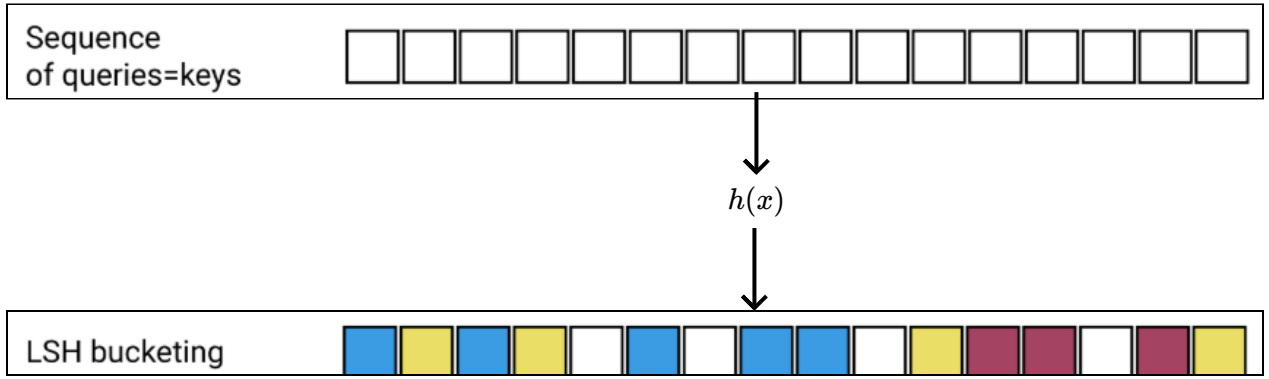
Can we find those k_j's without resorting to computing pair-wise dot product with all the keys?

It is similar to approximating attention using the K-Nearest Neighbours (KNN) search. For example, if K is of length 64K, for each q_i we could only consider a small subset of, say, the 32 or 64 closest keys

One of many possible ways of doing this is to use Locality Sensitive Hashing (LSH) (Yes, the same one we used for deduplication of contents!)

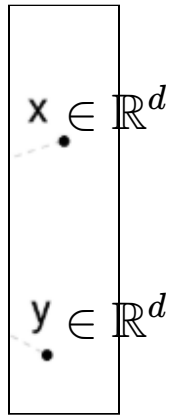
However, there is a catch!

For this to work, we need $Q = K$.This requires us to share the weight matrices $W_Q = W_K$.



We look for a hashing scheme that assigns each vector x (query) to a hash bucket such that nearby vectors falls in the same hash bucket and distant ones do not. This is called locality-sensitive hashing

For each query, it takes $log(T)$ time to find the approximate nearest neighbours [ref], therefore, its computational complexity is $O(T\log(T))$



Let's see how we construct hash buckets using a hash function with a help of an example (in 2D case)

Assume, $d_k = dq = 64$

Consider a query (and a key) $(x = q_1, y = k_2)$ that are away from each other and first projected onto a sphere

$$q_1, k_2 \in \mathbb{R}^{1\times 64}$$

Define the number of hash buckets, $(b = 4)$

Let the random rotation matrix R be of size $[d_k, b/2]$.

$$R \in \mathbb{R}^{64\times 2}$$

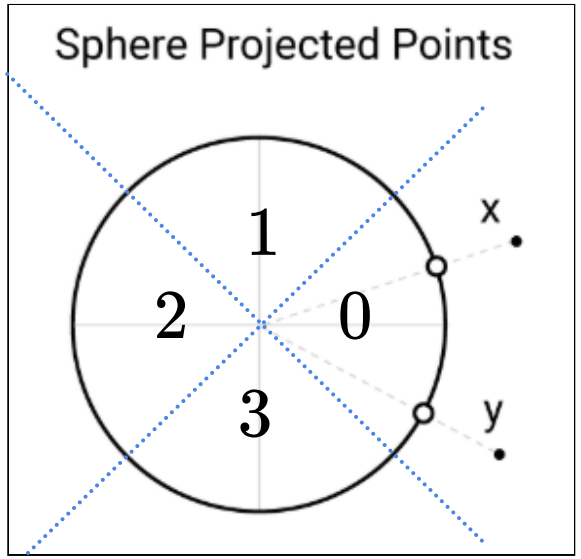
Let us define the hash function

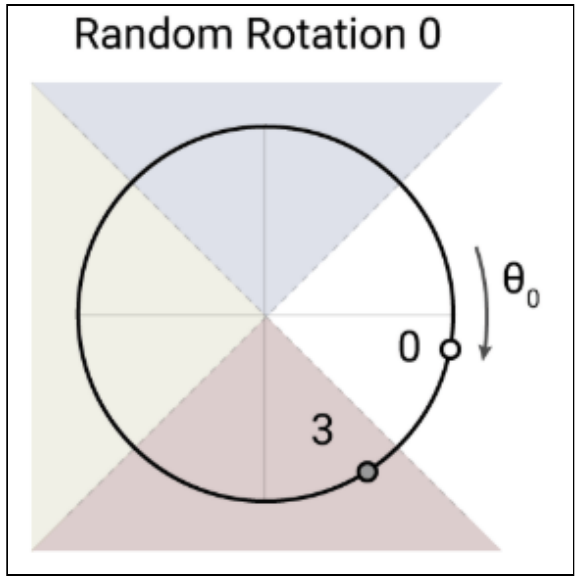
$$hash(x) = argmax(concat(xR, -xR))$$

for illustration,

$$hash(q_1) = argmax(concat(0.5, 0.25, -0.5, -0.25)) = 0$$

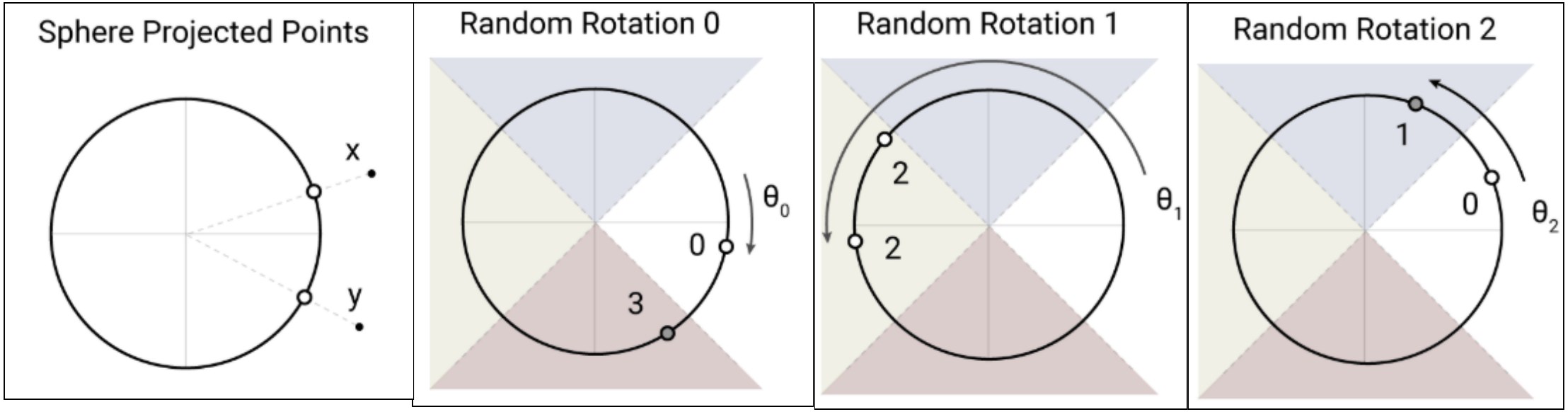
$$hash(k_2) = argmax(concat(-0.25, -0.5, 0.25, 0.5)) = 3$$





$$hash(q_1) = argmax(concat(0.5, 0.25, -0.5, -0.25)) = 0$$

$$hash(k_2) = argmax(concat(-0.25, -0.5, 0.25, 0.5)) = 3$$



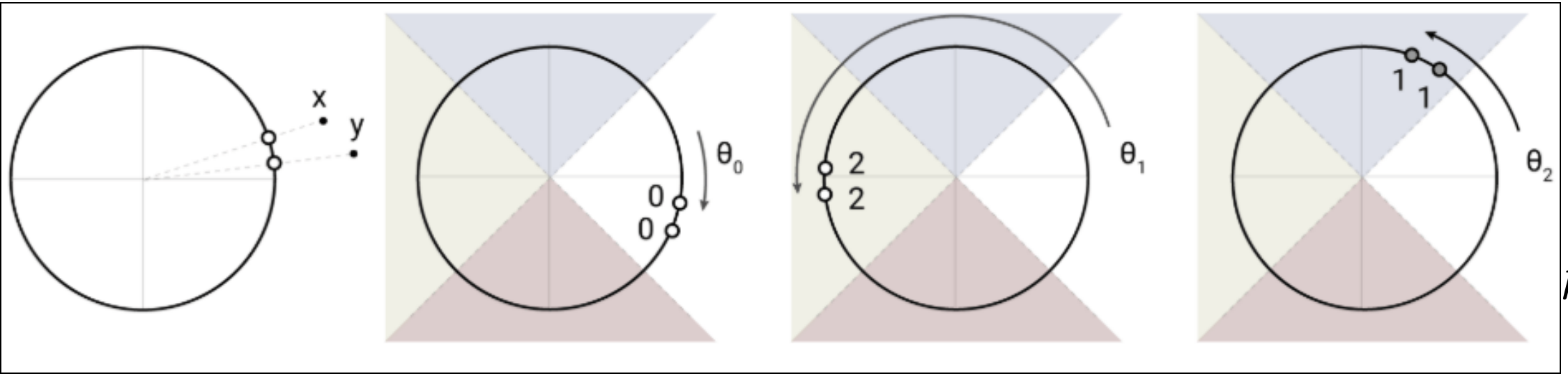
q_1 : 0 2 1

k_2 : 3 2 0

Recompute the hash by changing the rotation matrix a few times

The probability that the given query and the key share the same bucket is $\frac{1}{3}$. Therefore, they are not close to each other

Let's consider a query and a key that are close to each other



$q_1 : 0 \quad 2 \quad 1$

$k_2 : 0 \quad 2 \quad 1$

Recompute the hash by changing the rotation matrix few times

The probability that the queries share the same bucket is $\frac{3}{3}$. Therefore, they are close to each other

Therefore, all the points that are close to each other would share the same bucket.

Essentially compute the hash for all the queries (repeat n_R times if required)

This can be computed in parallel by multiplying all the queries with the random matrix R

Query	hash by R1	hash by R2
q_1	0	2
q_2	3	2
q_3	0	2
$\vdots$	$\vdots$	$\vdots$
q_{16}	0	2

Now, sort the queries by **bucket number**,

Essentially compute the hash for all the queries (repeat n_R times if required)

This can be computed in parallel by multiplying all the queries with the random matrix R

Query	hash by R1	hash by R2
q_1	0	2
q_3	0	2
q_{16}	0	2
$\vdots$	$\vdots$	$\vdots$
q_2	3	2

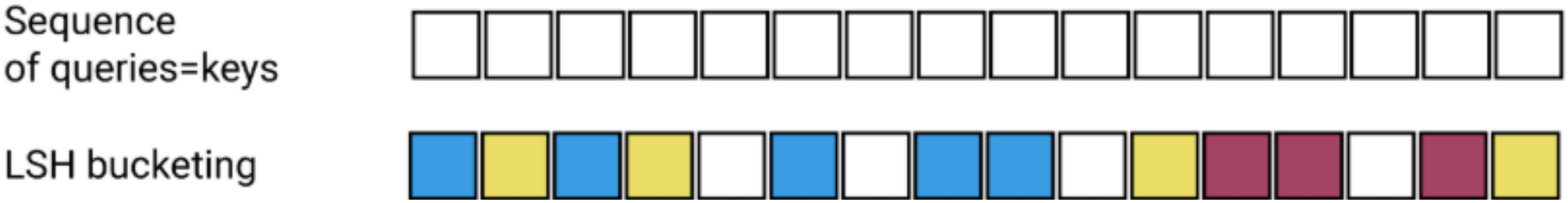
Now, sort the queries by **bucket number**,

and within each bucket, by sequence position

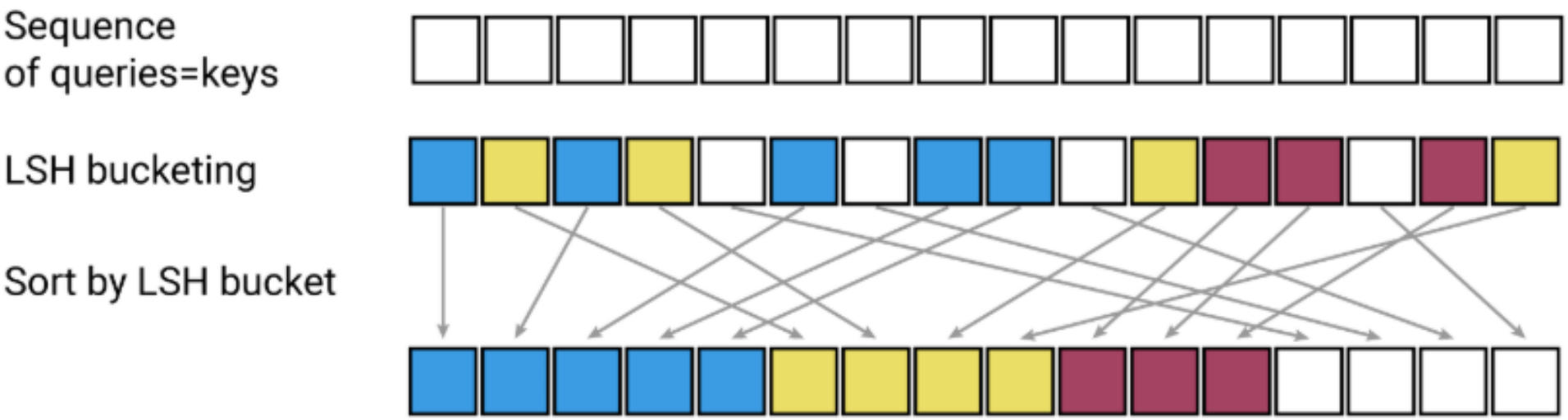
Compute the attention scores for queries within each bucket. For example, in ClM, q_3 attends to q_1, and q_{16} attends to q_3 and q_1 and so on

The entire procedure is captured in the figure below

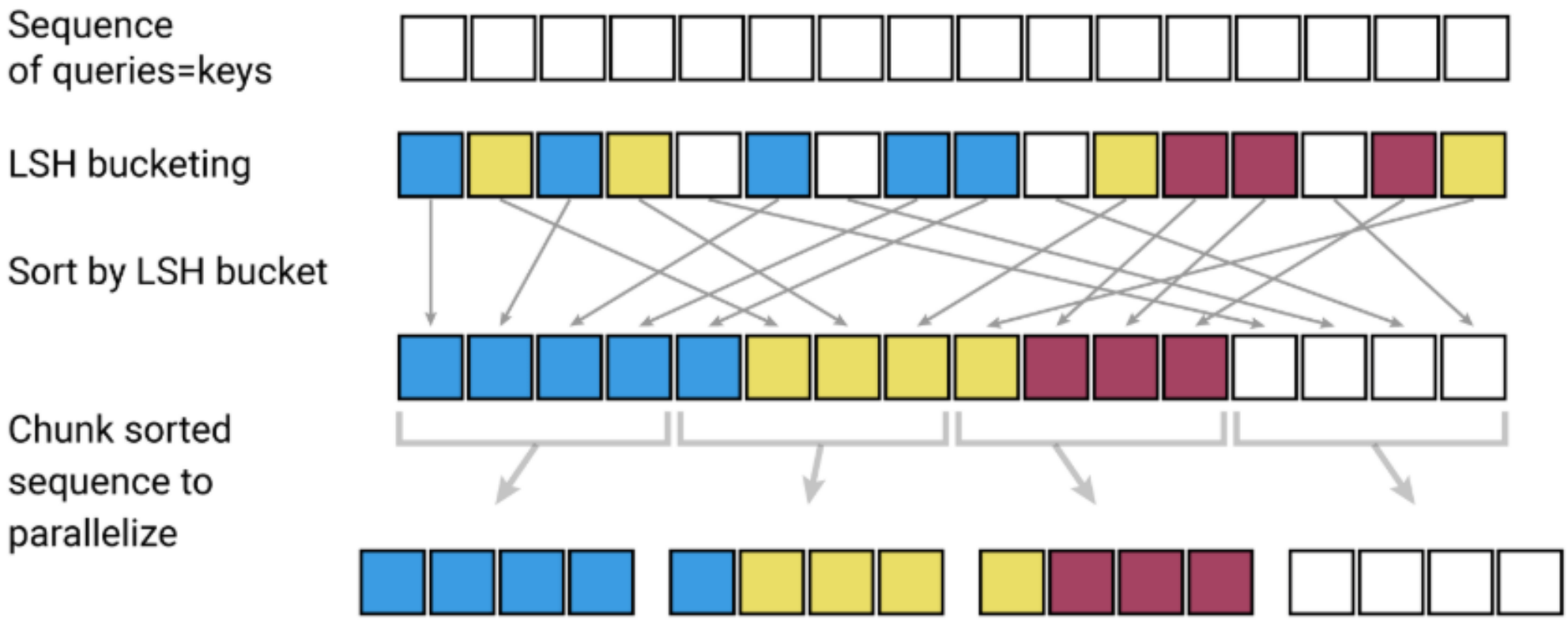
The entire procedure is captured in the figure below



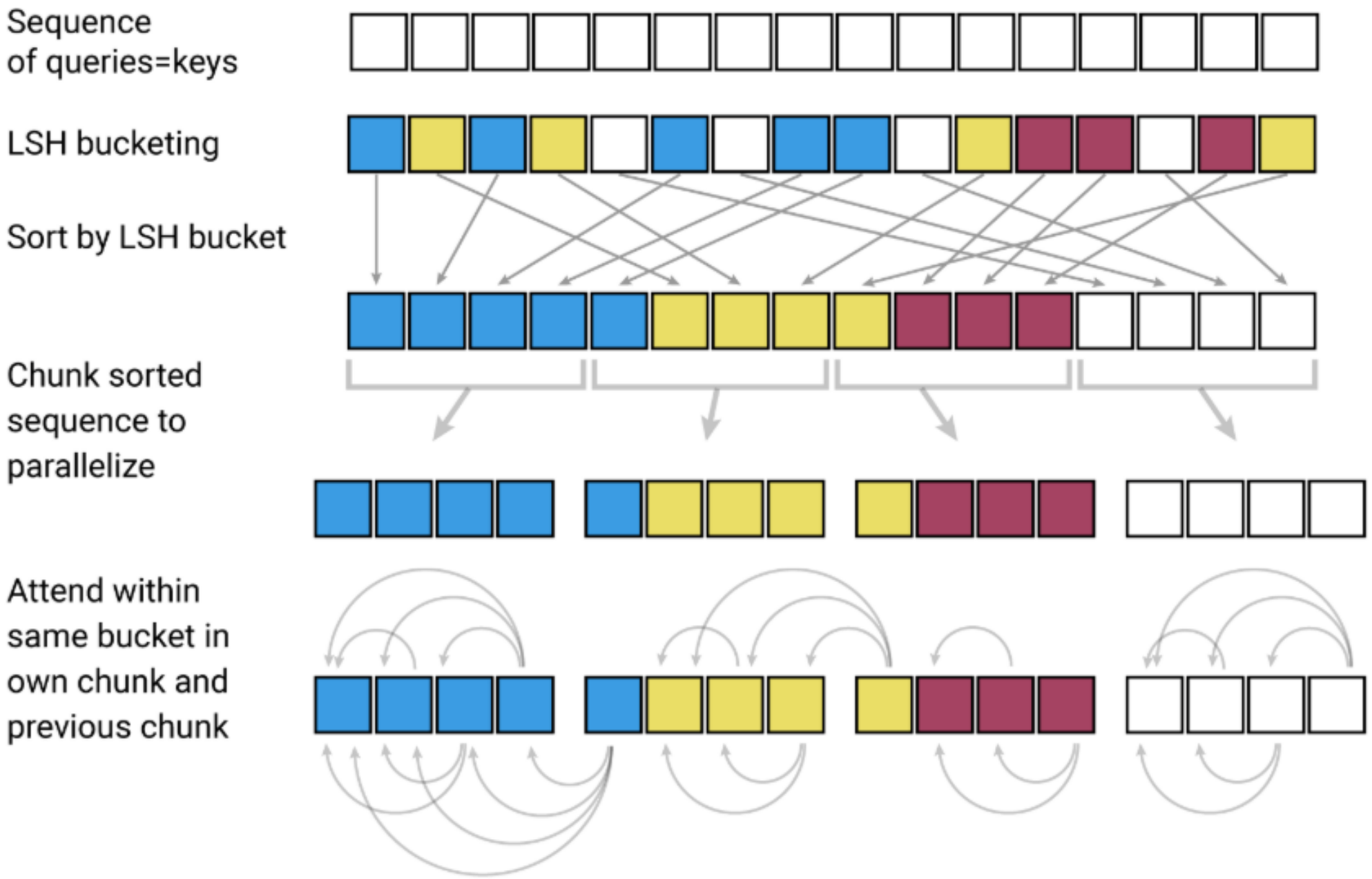
The entire procedure is captured in the figure below



The entire procedure is captured in the figure below



The entire procedure is captured in the figure below

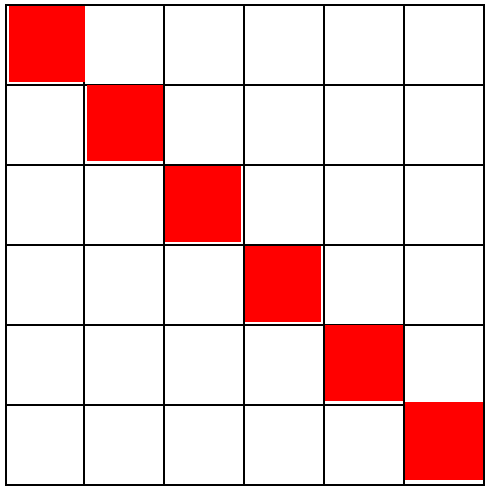


Self-attention is Low rank [paper]

The methods that we have seen so far localize or sparsify the attention matrix in different ways

The theoretical time complexity is sub-quadratic $O(cTd)$, with c being a constant (manually set, or function of T such as $log(T)$)

Can we reduce it to $O(Td)$? Yes, as follows :-)

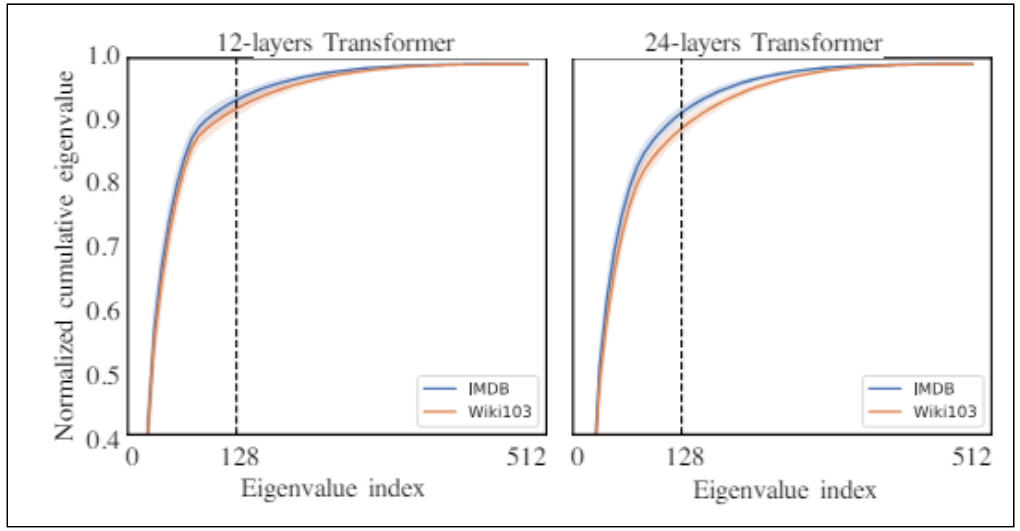


Well, this is not helpful to capture the context.

However, we can see from empirical observation that the self-attention matrix is essentially low-rank (especially for the top layers)!

$$A = softmax(\frac{QK^T}{\sqrt{d}})$$

$$A \in \mathbb{R}^{512 \times 512}$$



Consider a RoBERTa model trained on Wiki-103 (MLM task) and a classification task.

Apply SVD on the matrix A across different layers and different heads of the model, and plot the normalized cumulative singular value averaged over 10k sentences.

Suppose we plot the distribution of all 512 singular values . Then, for it to be a low-rank it should follow a ____ distribution.

They observed a long tail distribution. Hence they concluded the matrix A is a low-rank matrix.

Using this observation, they show that there exists a low-rank approximation of QK^T and we can let the network to find (learn) it! How?

$$Q \in \mathbb{R}^{T\times d} \qquad K \in \mathbb{R}^{T\times d} \qquad V \in \mathbb{R}^{T\times d}$$

Introduce two learnable linear projection matrices

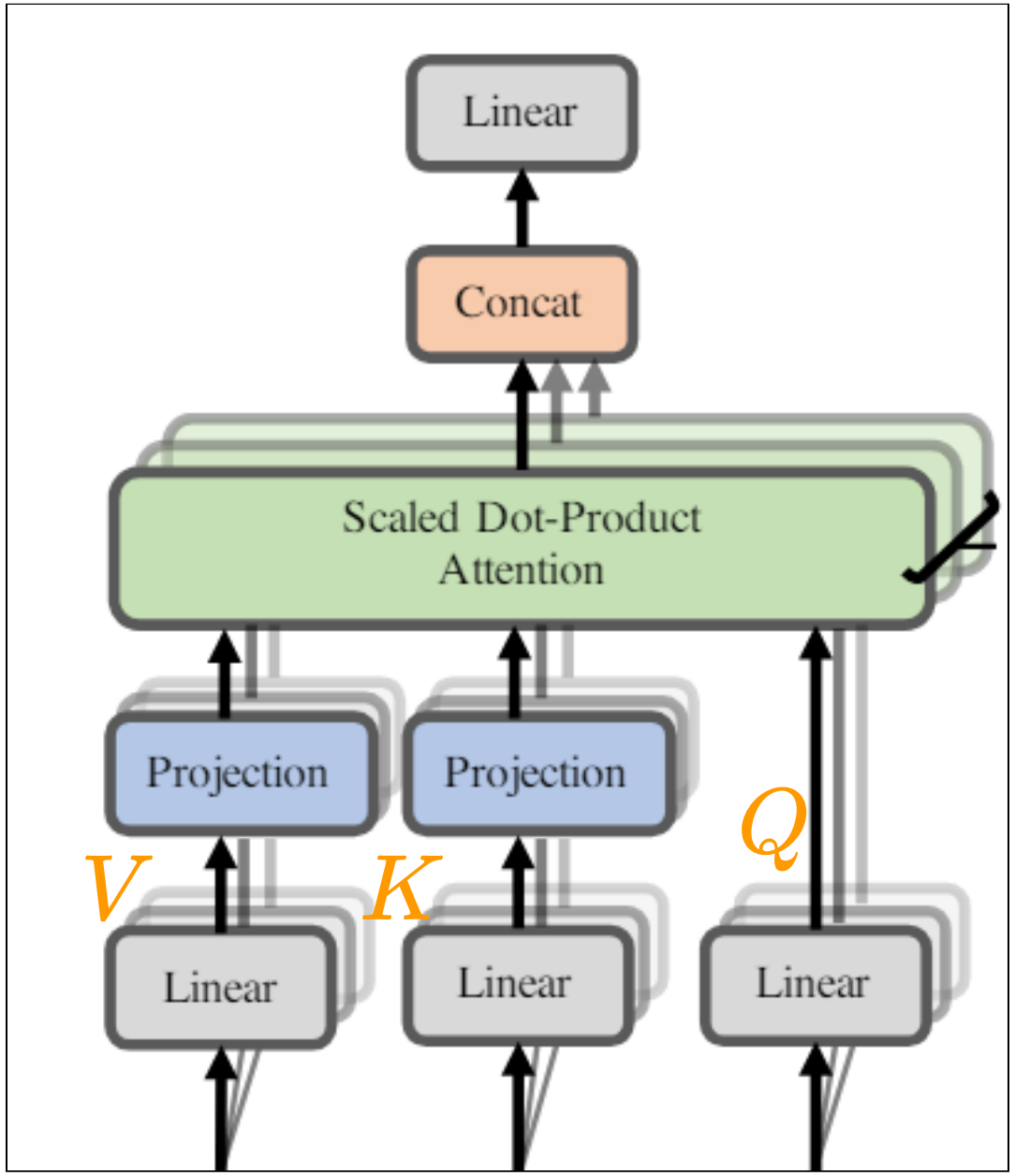
$$E, F \in \mathbb{R}^{k\times T}$$

$$A = softmax(\frac{Q(EK)^T}{\sqrt{d}})$$

$$A \in \mathbb{R}^{T\times k}$$

$$\boxed{O = AFV}$$

The computational complexity is $O(kTd)$, where k is the rank and can be set according to the error ϵ



Kernel Methods [paper]

All that we need is an attention matrix such that each row sums up to one

$$Atten = softmax(\frac{QK^T}{\sqrt{d}})$$

$$Atten = D^{-1}A$$

where,

$$A = exp(QK^T/\sqrt{d})$$

$$D = diag(A\mathbf{1})$$

$\mathbf{1}$ is a vector of all ones of length T

Fundamentally, the exponential function in A takes in the query and key vectors and outputs a positive number.

So we can generalize this

$$A(i,j) = SM(q_i, k_j) = \mathcal{K}(q_i, k_j))$$

$$\mathcal{K} : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}_+$$

$\mathcal{K}$ is called a Kernel function, and defined as follows

$$\mathcal{K}(q_i, k_j) = \mathbb{E}(\phi(q_i)^T \phi(k_j))$$

$$\phi(x) : \mathbb{R}^d \to \mathbb{R}_+^r, r > 0$$

Where $\phi(x)$ is some **randomized** feature mapping and $\mathbb{E}$ is an expectation operator.

Note that $r > 0$, and it is not necessary that $r < d$.

For example, $T = 64K, d = 4K$, then $r = 8k$ is also reasonable

How do we compute $\phi(x)$? $\phi(x) : \mathbb{R}^d \to \mathbb{R}^r_+$

The generalized form of a kernel transformation is given by,

$$\phi(x) = \frac{h(x)}{\sqrt{r}} \left(f_1(w_1^T x), \cdots, f_1(w_r^T x), \cdots, f_l(w_1^T x), \cdots, f_l(w_r^T x) \right)$$

where,

$$f_1, f_2, \cdots, f_l : \mathbb{R} \to \mathbb{R}$$

$$w_1, \cdots, w_r : \mathbb{R}^d \quad \text{where } w_i \text{ is sampled from } \mathcal{N}(\mathbf{0}_d, \mathbf{I}_d)$$

$w_i^T x$ is a random projection, that is, the vector x is projected in a random direction given by w_i vector.

We can construct a random feature vector of size r by setting $l = 1$, considering $f_1 = exp$ and $h(x) = exp(\frac{-||x||^2}{2})$ (Note, this particular setting is theoretically motivated among many possible choices.)

Therefore,

$$\phi(x) = \frac{exp(\frac{-||x||^2}{2})}{\sqrt{r}}\left(exp(w_1^T x), \cdots, exp(w_r^T x)\right)$$

$$w_i \sim \mathcal{N}(\mathbf{0}_d, \mathbf{I}_d)$$

We can collect all w_is in a matrix $\mathbf{W}$

$$\phi(X) = \frac{1}{\sqrt{r}} exp\left(\mathbf{W}^T X - \frac{||X||^2}{2}\right)$$

where $\mathbf{W} \in \mathbb{R}^{d \times r}$,$X \in \mathbb{R}^{d \times T}$ and $||X||^2$ is the $L2-$norm of column vectors in X.

The $\mathbf{W}$ matrix is orthogonalized using Gram-Schmidt orthogonalization procedure.

Orthogonalization of W helps reduce the variance of the softmax estimator for any dimensionality of d. This requires $r \leq d$.

For all the queries q_is in Q, we can compute Q' as follows

$$Q' = \frac{1}{\sqrt{r}} exp\left(\mathbf{Q}\mathbf{W}^{\mathbf{T}} - \frac{||Q||^2}{2}\right) \qquad Q' \in \mathbb{R}^{T \times r}$$

here, $\mathbf{W} \in \mathbb{R}^{r \times d}$,$Q \in \mathbb{R}^{T \times d}$ and $||Q||^2$ is the $L2-$norm of row vectors in Q.

Similary, we can compute $K' \in \mathbf{R}^{T \times r}$.

The approximated attention matrix for the given query and key is given by

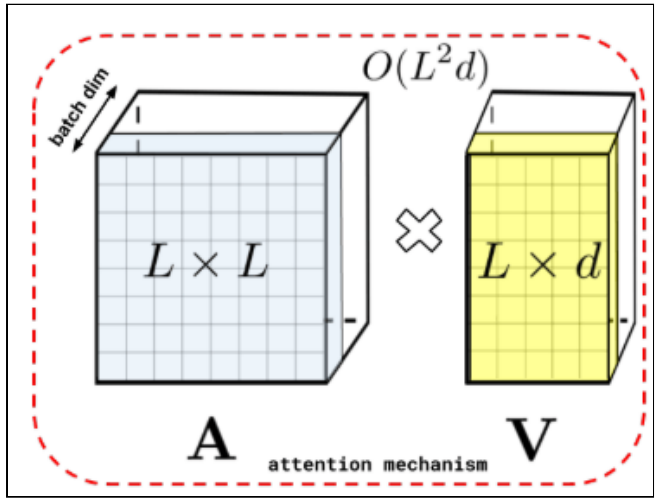
$$\hat{A}(i,j) = S\hat{M}(q_i, k_j) = \frac{1}{r} exp(q_i \mathbf{W}^T - \frac{||q_i||^2}{2}) exp(k_j \mathbf{W}^T - \frac{||k_j||^2}{2})^T$$

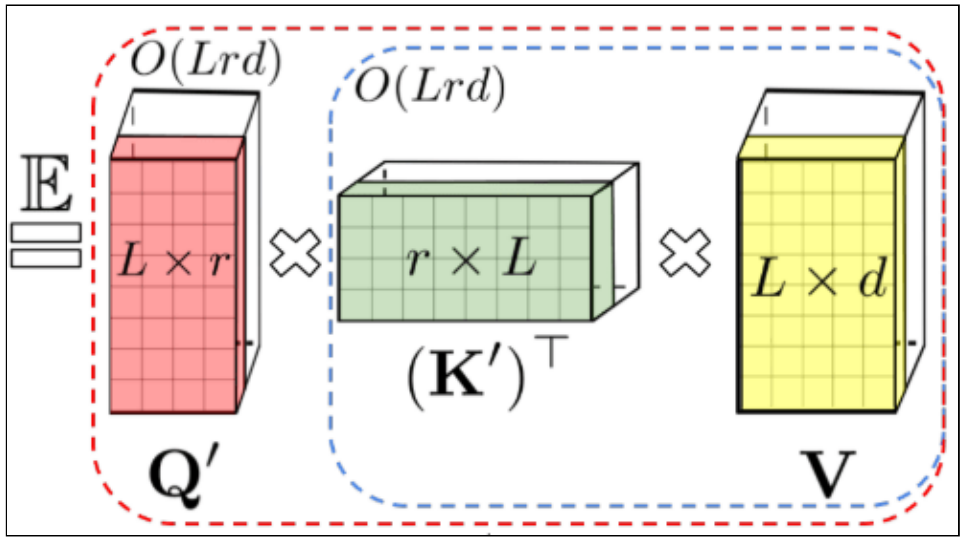
and the vectorized version is given by

$$\hat{A} = Q' K'^T$$

The time complexity is $O(rTd)$.

$L = T$ in the figure below





Reduced FLOPS != Reduced Wall clock speed

All these approaches that we have discussed so far approximated the full attention.

However, it does not translate to wall-clock speed-up!

That is, suppose we train a model with the compute requirement $C \propto T^2$ FLOPS for a day.

Reducing the compute requirement $C \propto \frac{T^2}{2}$ FLOPS will not reduce the training time to half a day

So, the problem is not in the algorithm but..

Module 4: Know your hardware to get wall-clock speed

Mitesh M. Khapra

AI4Bharat, Department of Computer Science and Engineering, IIT Madras

Flash Attention: Exact Attention with IO Awarness

All Transformer based language models are trained on GPUs.

However, it was observed that the implementation of MHA did not efficiently utilize GPUs.

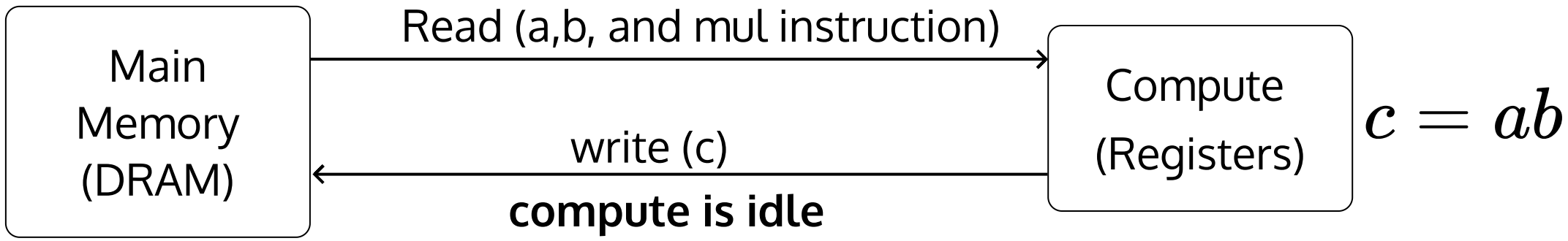
In fact, the training time is now memory bound not compute bound!

Let us try to understand this with a help of a simple example.

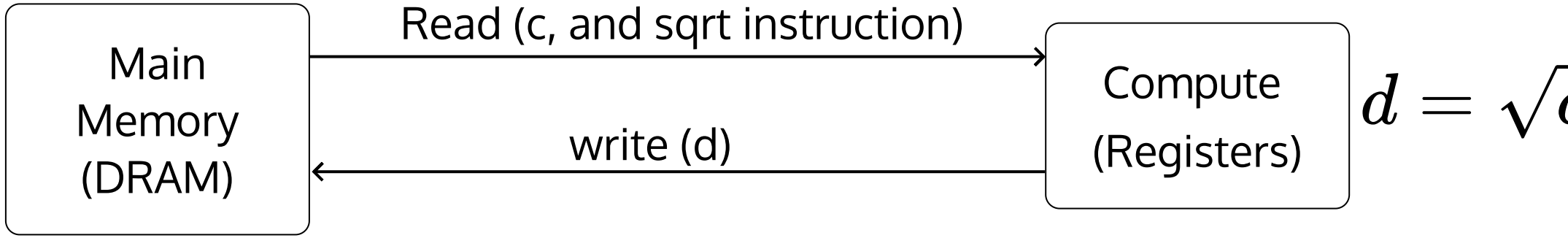
What are the sequence of steps involved in computing any mathematical operation? For example, consider a product of two numbers and computing square root of the result

$$c = ab$$

$$d = \sqrt{c}$$



$$c = ab$$



$$d = \sqrt{c}$$

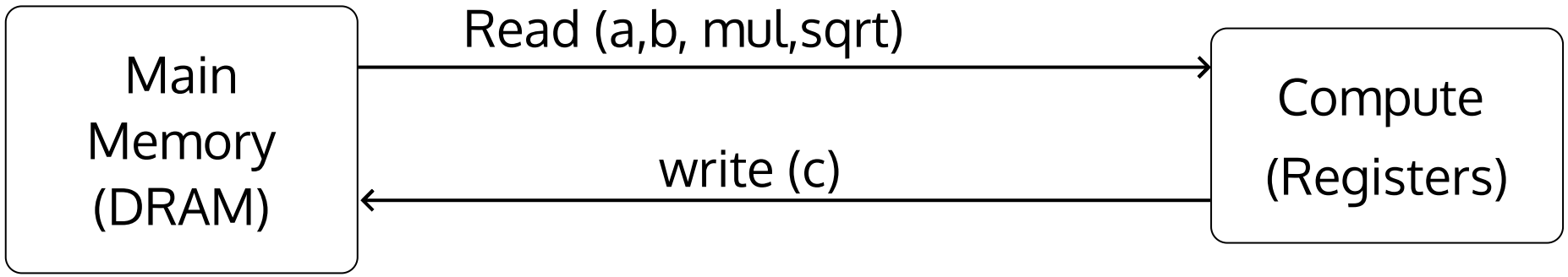
Typically, after executing an operation, the result is written back to main memory.

Therefore, there are two read operations, two write operations and two compute operations. The compute was not utilized during writing the result back to memory!

How do we skip this redundant reading from/ writing to memory as we are aware that the result c will be used in the subsequent operation?

Kernel fusion

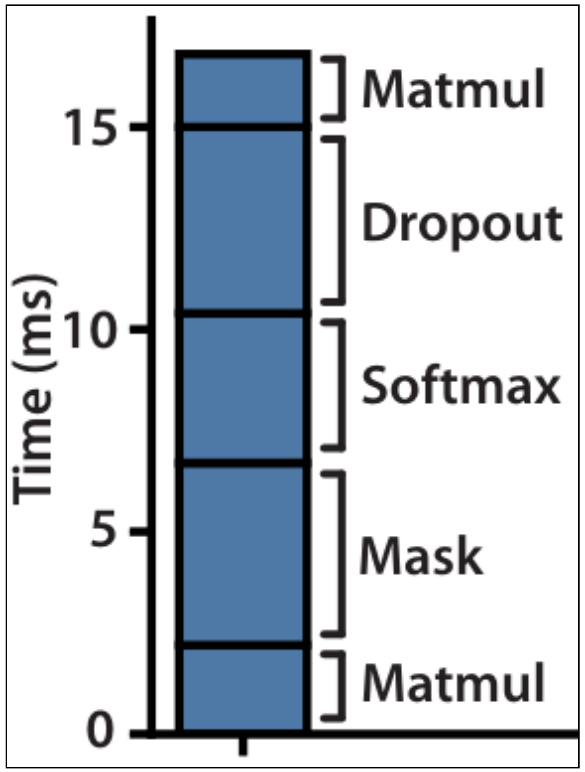
We can fuse the sequence of operation (called kernel fusion) as follows $c = \sqrt{ab}$



We can store the intermediate result of the multiply operation in the registers/cache memory and take square root on it, then write the final result back to main memory

Therefore, Kernel fusion helps optimally utilize the compute capacity by making it aware that the intermediate results need not be written back to main memory!

Let us see the amount of time spent by the naive implementation of MHA and MHA using Kernel fusion.



$$drop(softmax(QK^T + M))V$$

$$drop(softmax(QK^T + M))$$

$$softmax(QK^T + M)$$

$$QK^T + M$$

$$QK^T$$

We write the output back to memory at every step and read the previous output as an input for the next step!

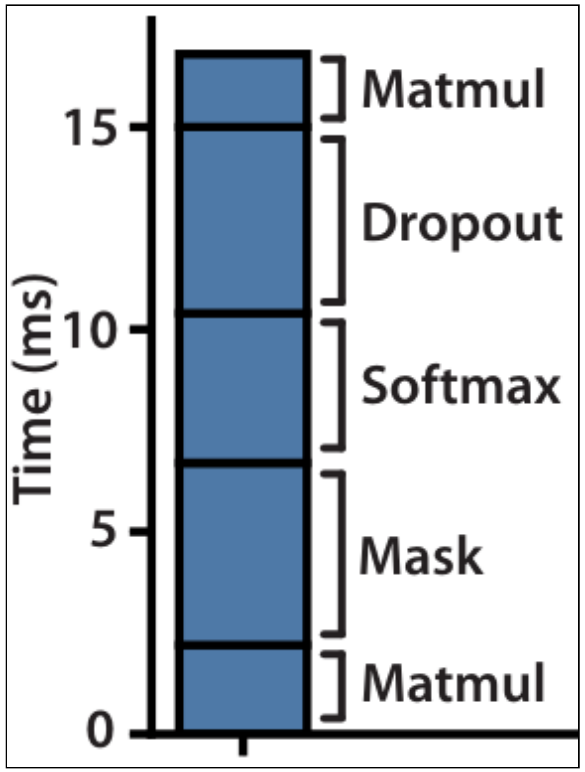
Naive implementation

Do you see anything surprising from the diagram?

Dropout, softmax, and masking take more time to execute than MatMul!

In general, all element-wise operations like activation, normalization are all memory bound

A general approach to alleviate this problem is to use kernel fusion!



$$drop(softmax(QK^T + M))V$$

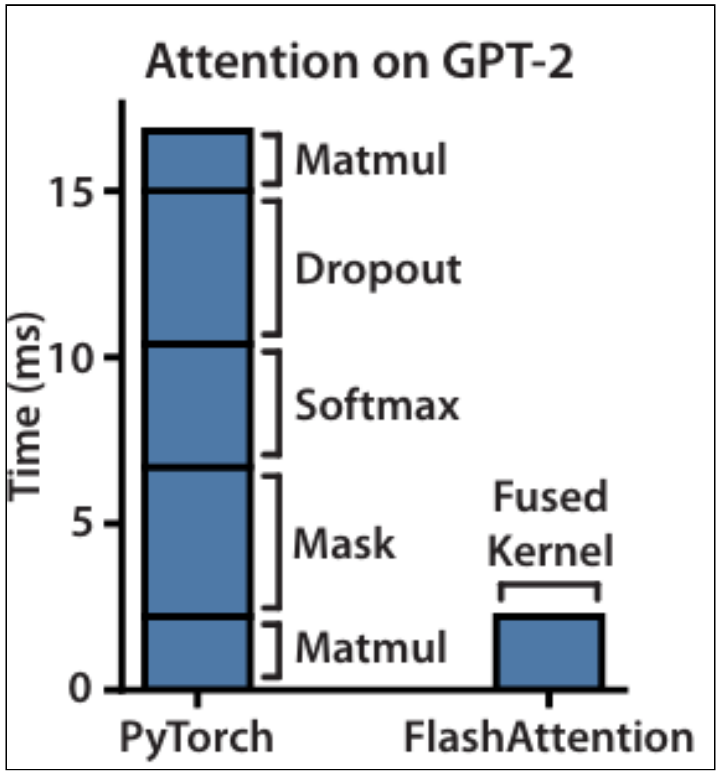
$$drop(softmax(QK^T + M))$$

$$softmax(QK^T + M)$$

$$QK^T + M$$

$$QK^T$$

Naive implementation



At every step, we write an output back to memory and read the previous output as an input for the next step!

There is a catch! We need to store QK^T for backward pass!

Getting into Details

$$S = QK^T \qquad P = Softmax(S) \qquad O = PV$$

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

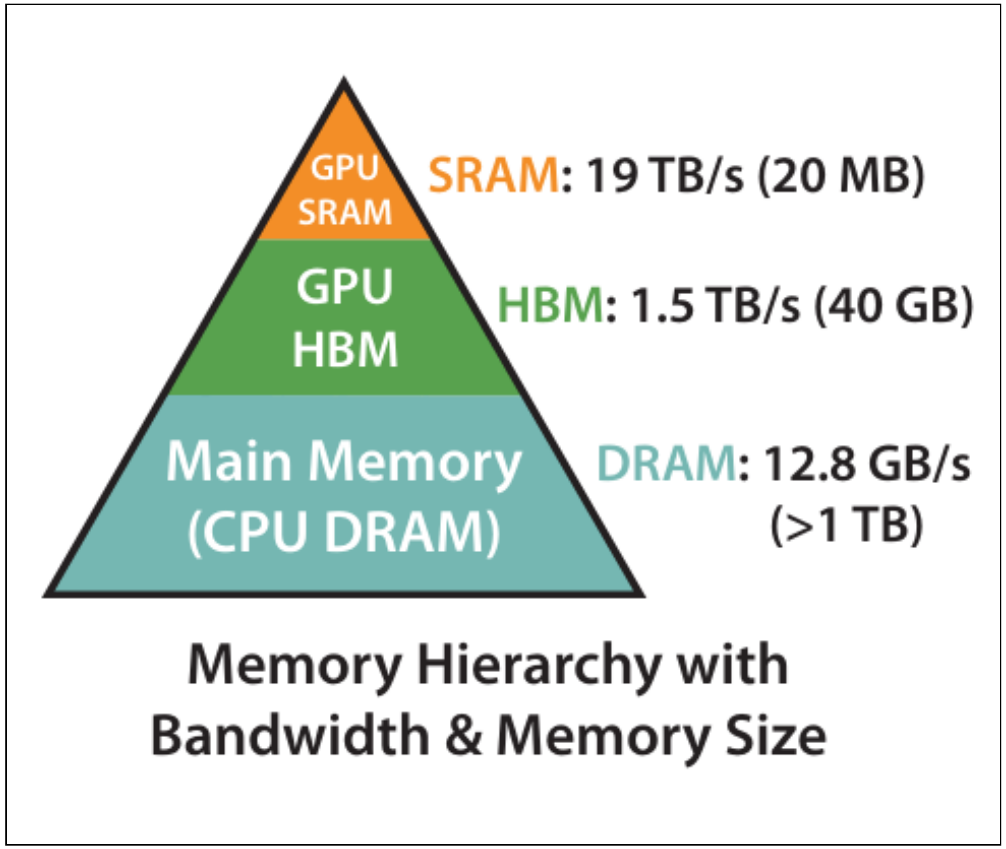
1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \mathrm{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
4: Return $\mathbf{O}$.

Standard attention is memory bound.

Objective: Given Q, K, V, compute O and write it to HBM (High Bandwidth Memory) (Note: using only one write operation).

We can not use entire Q, K, and V matrices. We need to divide each of these into blocks. Why?

Memory Hierarchy



High Bandwidth Memory (the one you get when you type *nvidia-smi* command) is the place where we load and store data, model parameters, and all intermediate results.

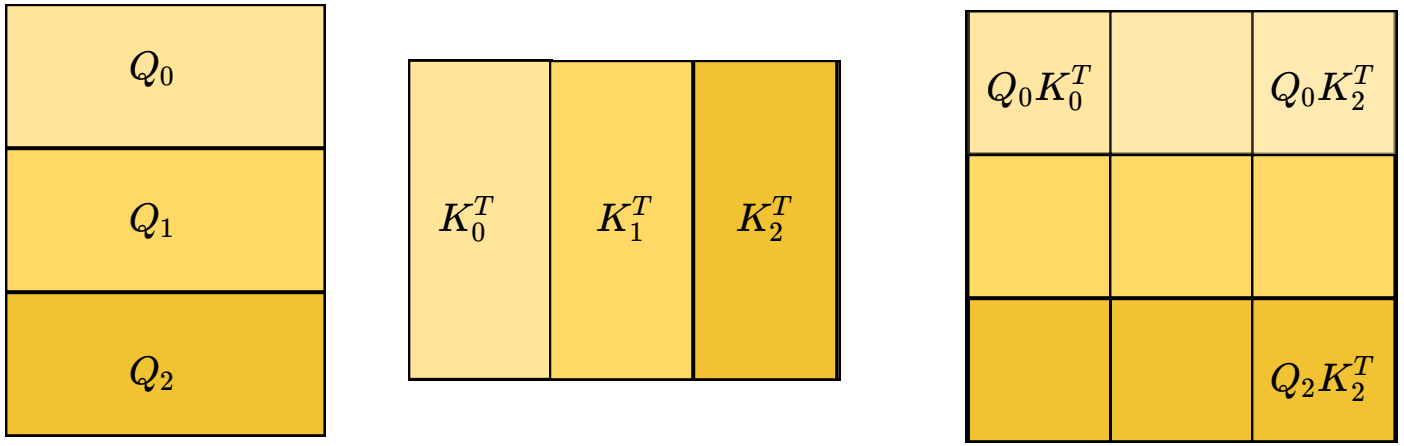
SRAM has small memory (20MB) (distributed across cores each of size about 100KB) with a high bandwidth (19TB/s)

Suppose we have a sequence of length 1024, to store Q, K using 4 bytes (32 bit) per element requires $2 \times 1024 \times 64 \times 4 = 524KB$.

So, it is not possible to fit in the entire Q and K matrices into SRAM.

IDEA

1. Tiling: Divide the matrices Q, K, V into small blocks (in HBM) such that it fits into the SRAM of size M (around 100 KB) and compute attention by blocks



$$S_{ij} = Q_i K_j^T$$

We apply the softmax function for each block of $Q_i K_j^T$.

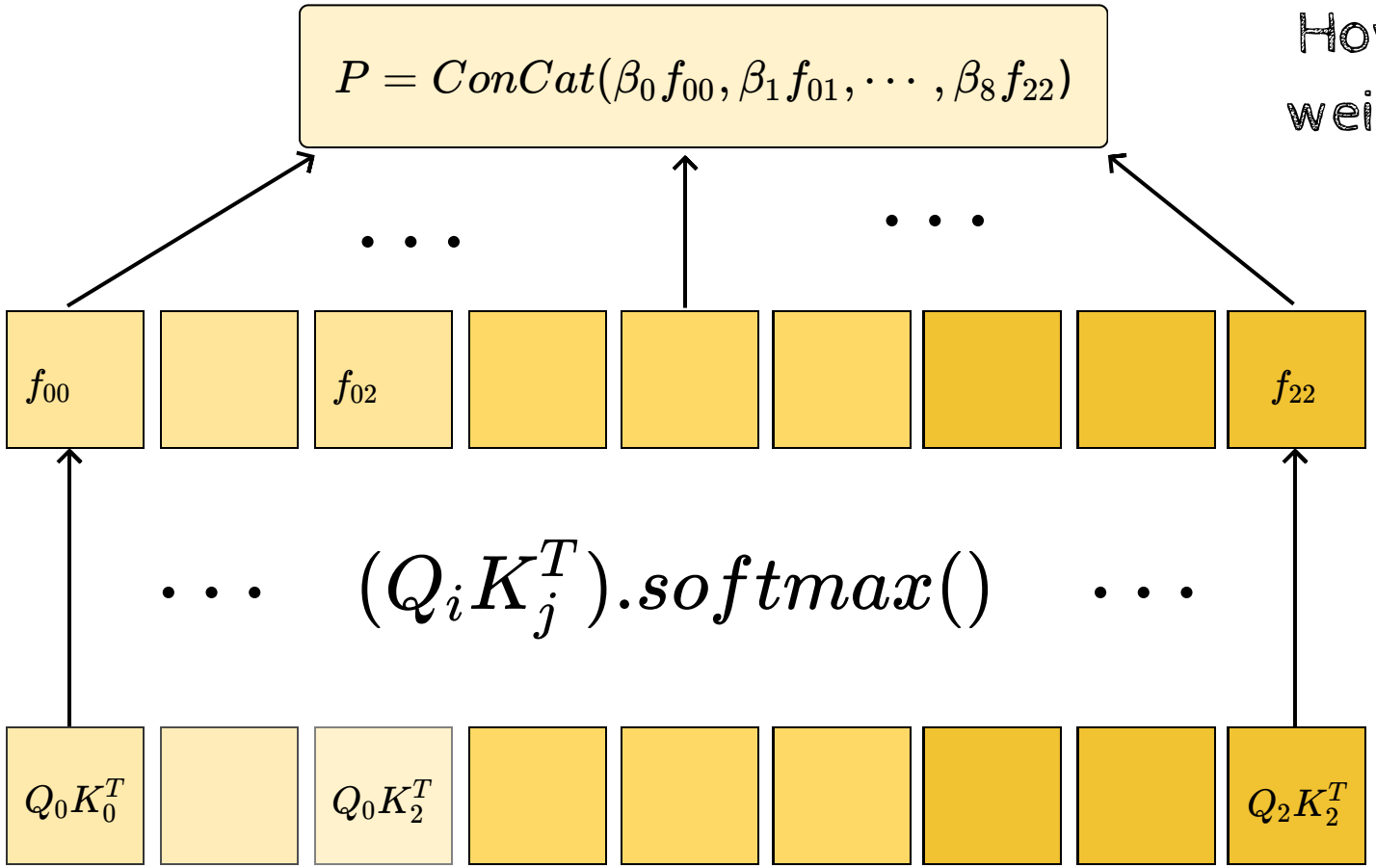
However, we have only S_{ij} but the softmax requires all the elements in S for normalization

$$P = Softmax(S)$$

How do we go from S_{ij}'s to P?

IDEA

1. Divide the matrices Q, K, and V into small blocks (in HBM) such that it fits into the SRAM of size M (around 100 KB)



How do we compute the weight β_i for each block ?

f_{ij} is an intermediate sequence

$S_{ij} = Q_i K_j^T$

IDEA

How do we compute β_i? Let us take a simple example to illustrate the idea

Before going further, we use a variation of softmax implementation called "Safe softmax" in which we subtract the maximum of the vector from all its elements because the exponential operator is more accurate for negative inputs [Ref]

$$\frac{e^{x_i}}{\sum_j e^{x_j}} = \frac{e^{x_i - m}}{\sum_j e^{x_j - m}}$$

where,

$$m = max(x)$$

Consider an example sequence x divided into two consecutive sub-sequences x^A and x^B.

$x = [x^A, x^B]$ where, x^A: -1.02 -1.09 0.24 0.87 -1.32, x^B: -0.6 -5.04 -2. 1.03 -0.14

Online softmax

How do we compute β_i? Let us take a simple example to illustrate the idea

Split

x^A: -1.02 -1.09 0.24 0.87 -1.32

x^B: -0.6 -5.04 -2. 1.03 -0.14

$m_1 = max(x^A)$ 0.87

$m_2 = max(x^B)$ 1.03

$f_1 = e^{x_i^A - m_1}$ 0.15 0.14 0.53 1. 0.11

$f_2 = e^{x_i^B - m_2}$ 0.2 0. 0.05 1. 0.31

$l_1 = \sum_j e^{x_j^A - m_1}$ 1.93

$l_2 = \sum_j e^{x_j^B - m_2}$ 1.56

Combine

$m = max(m_1, m_2))$ 1.03

$\beta_1 = e^{m_1 - m} = e^{0.87 - 1.03}$ 0.85

$\beta_2 = e^{m_2 - m}$ 1

$f = \beta_1 f_1 : \beta_2 f_2$

0.13 0.12 0.45 0.85 0.1 0.2 0. 0.05 1. 0.31

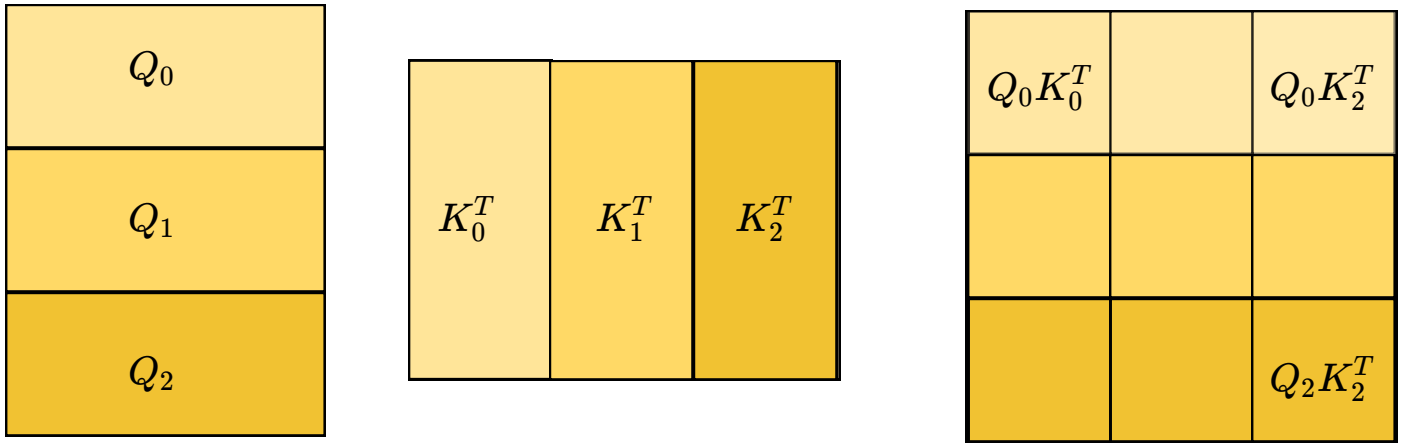
$l = \beta_1 l_1 + \beta_2 l_2$ 3.2

$P = f/l$

0.04 0.04 0.14 0.27 0.03 0.06 0. 0.01 **0.31** 0.1

IDEA

1. Divide the matrices Q, K, V into small blocks (in HBM) such that it fits into the SRAM of size M (around 100 KB)



$$S_{ij} = Q_i K_j^T$$

Subsequently, we apply the softmax function over each blocks of $Q_i K_j^T$.

However, recall that softmax requires all the elements in S

$$P = Softmax(S)$$

How do we go from S_{ij}'s to P?

We need to keep track of m_i's and l_i's to get P from f_i's

Algorithm

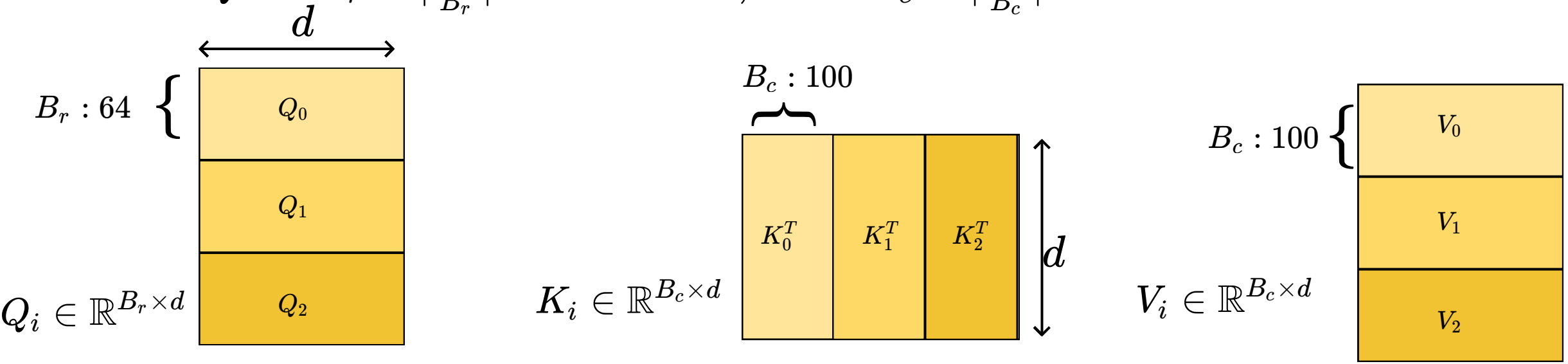
1. Get Q, K, V (in HBM) and size of SRAM M (around 100 KB)

2. Determine the block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = min((\frac{M}{4d}, d))$

   For, $d = 64$ and assigning 4 bytes per element, the $B_c = \frac{100 \times 1024\ B}{4 \times 64 \times 4\ B} = 100, B_r = 64$

3. Initialize $O \in \mathbb{R}^{N \times d}$, $l \in \mathbb{R}^N$,to zeros and $m \in \mathbb{R}^N$ to $-\infty$, (in HBM)

4. Divide Q into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks and K, V into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks



Algorithm

1. Get Q, K, V (in HBM) and size of SRAM M (around 100 KB)

2. Determine the block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = min((\frac{M}{4d}, d))$

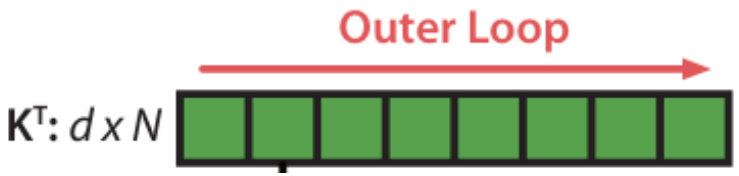
   For, $d = 64$ and assigning 4 bytes per element, the $B_c = \frac{100 \times 1024\ B}{4 \times 64 \times 4\ B} = 100, B_r = 64$

3. Initialize $O \in \mathbb{R}^{N \times d}$, $l \in \mathbb{R}^N$,to zeros and $m \in \mathbb{R}^N$ to $-\infty$, (in HBM)

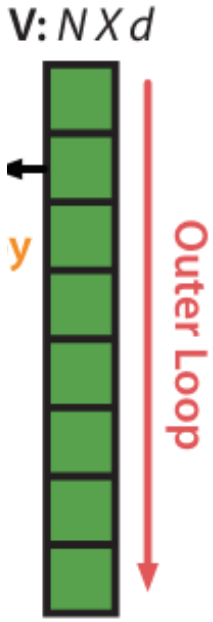
4. Divide Q into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks and K, V into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks

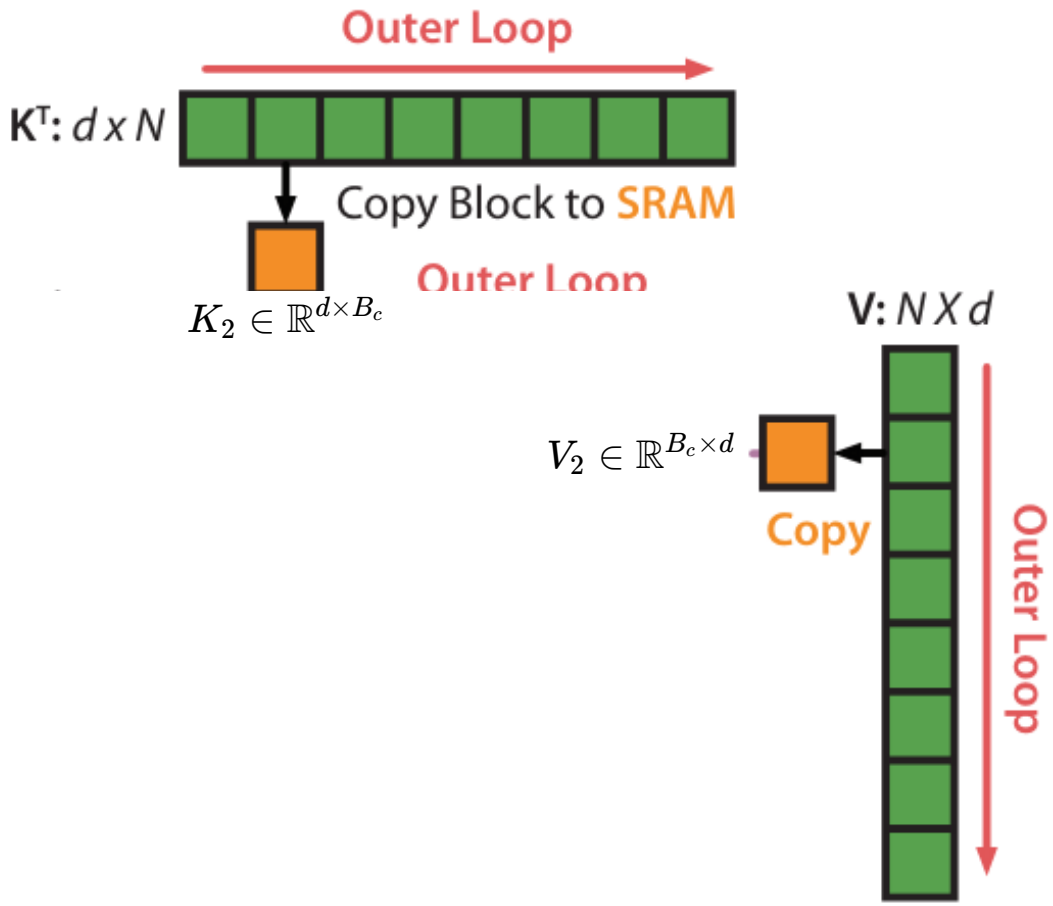
5. Divide O into T_r blocks, and l, m into T_r segments of size B_r

Let us visualize the subsequent steps



for $1 \leq j \leq T_c$:

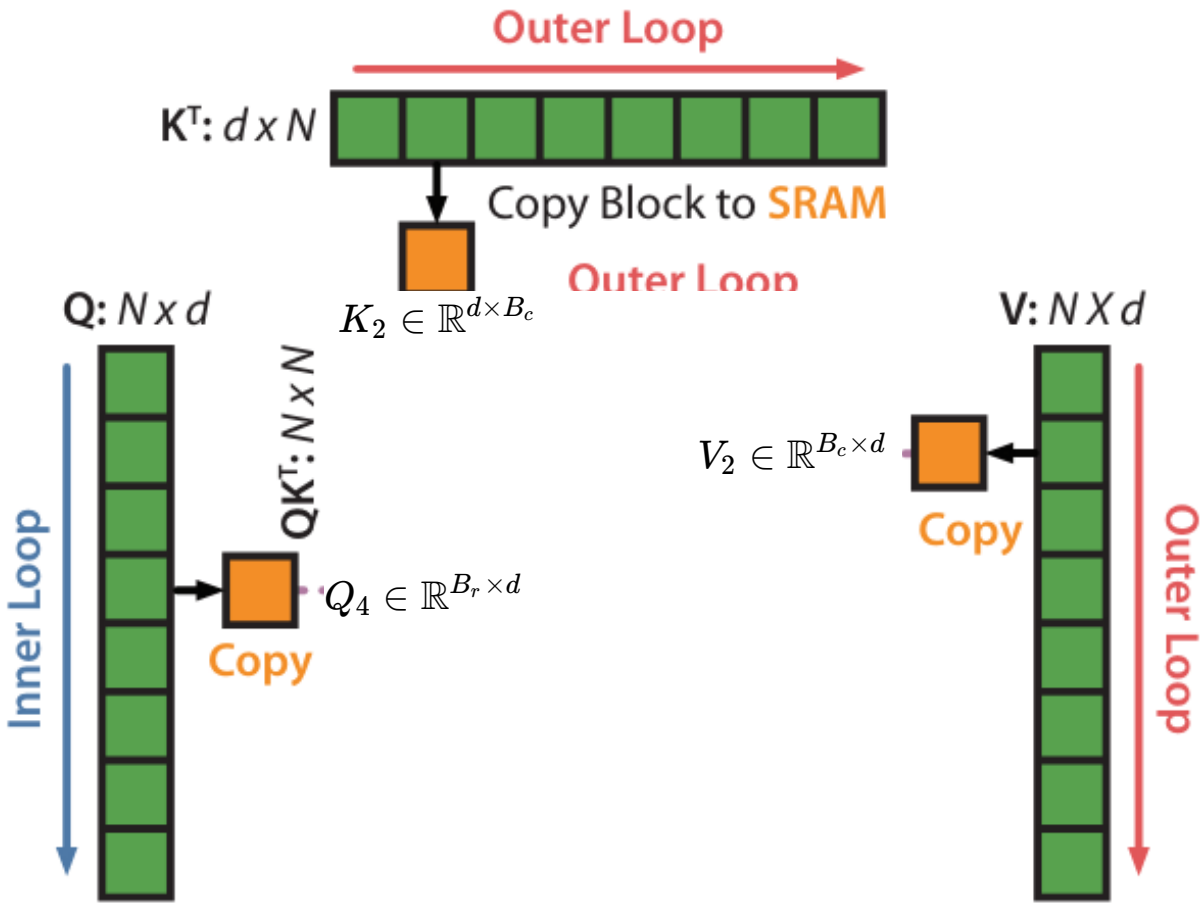




for $1 \leq j \leq T_c$:

Load, K_j, V_j from HBM to SRAM

for $1 \leq i \leq T_r$:



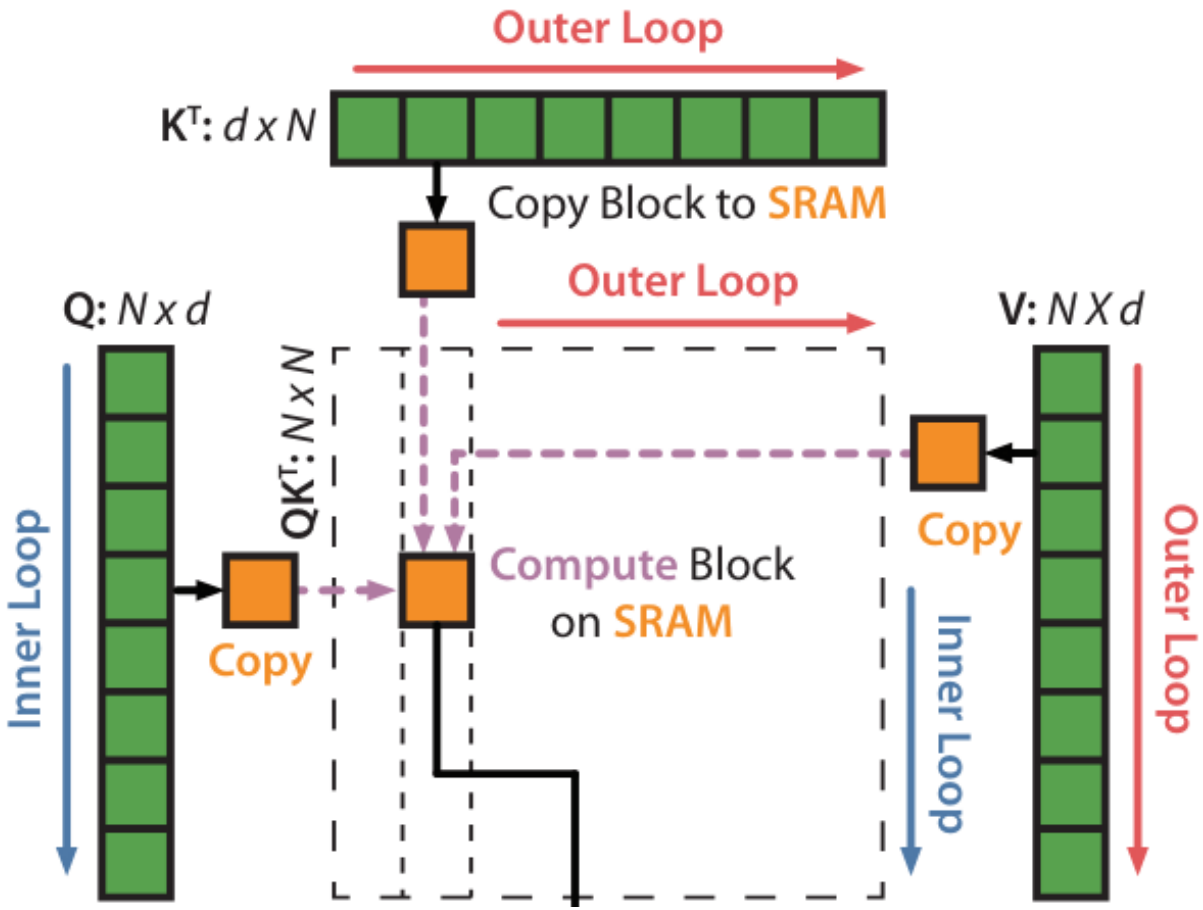
$$S = QK^T \qquad P = Softmax(S) \qquad O = PV$$

for $1 \leq j \leq T_c$:

Load, K_j, V_j from HBM to SRAM

for $1 \leq i \leq T_r$: #iterate over blocks of Q

Load, Q_i, O_i, l_i, m_i from HBM to SRAM



$$S = QK^T \qquad P = Softmax(S) \qquad O = PV$$

for $1 \leq j \leq T_c$:

Load, K_j, V_j from HBM to SRAM

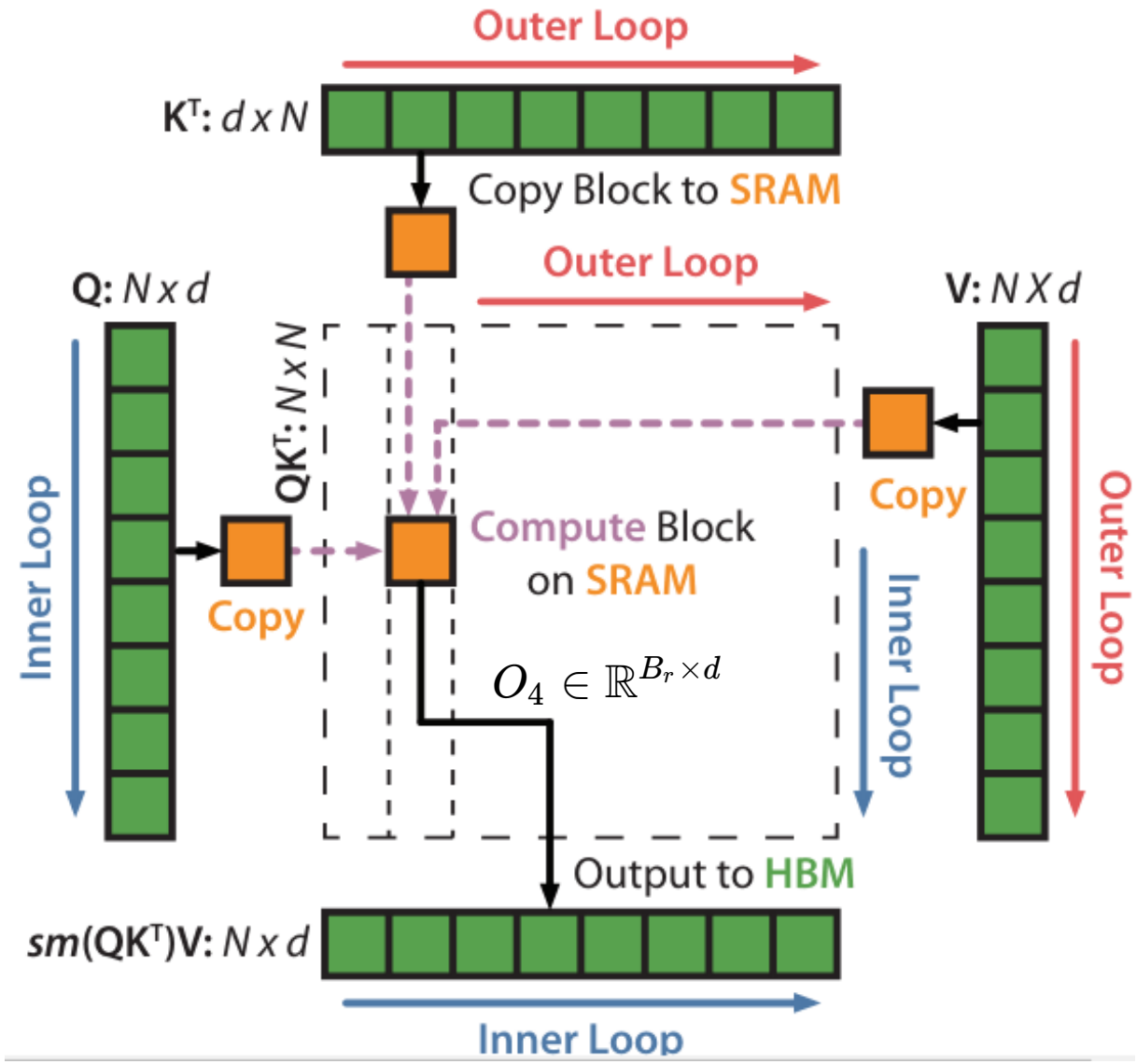
for $1 \leq i \leq T_r$: #iterate over blocks of Q

Load, Q_i, O_i, l_i, m_i from HBM to SRAM

on-chip, compute $S_{ij} = Q_i K_j^T \in \mathbb{R}^{B_r \times B_c}$

on-chip, compute block-wise statistics m_{ij}, l_{ij}, P_{ij}

on-chip, compute new values for m_i, l_i



$S = QK^T \qquad P = Softmax(S) \qquad O = PV$

for $1 \leq j \leq T_c$:

Load, K_j, V_j from HBM to SRAM

for $1 \leq i \leq T_r$: #iterate over blocks of Q

Load, Q_i, O_i, l_i, m_i from HBM to SRAM

on-chip, compute $S_{ij} = Q_i K_j^T \in \mathbb{R}^{B_r \times B_c}$

on-chip, compute block-wise statistics m_{ij}, l_{ij}, P_{ij}

on-chip, compute new values for m_i, l_i

Write, $O_i \leftarrow f(O_i, P_{ij}, V_j, m_i, l_i)$ to HBM

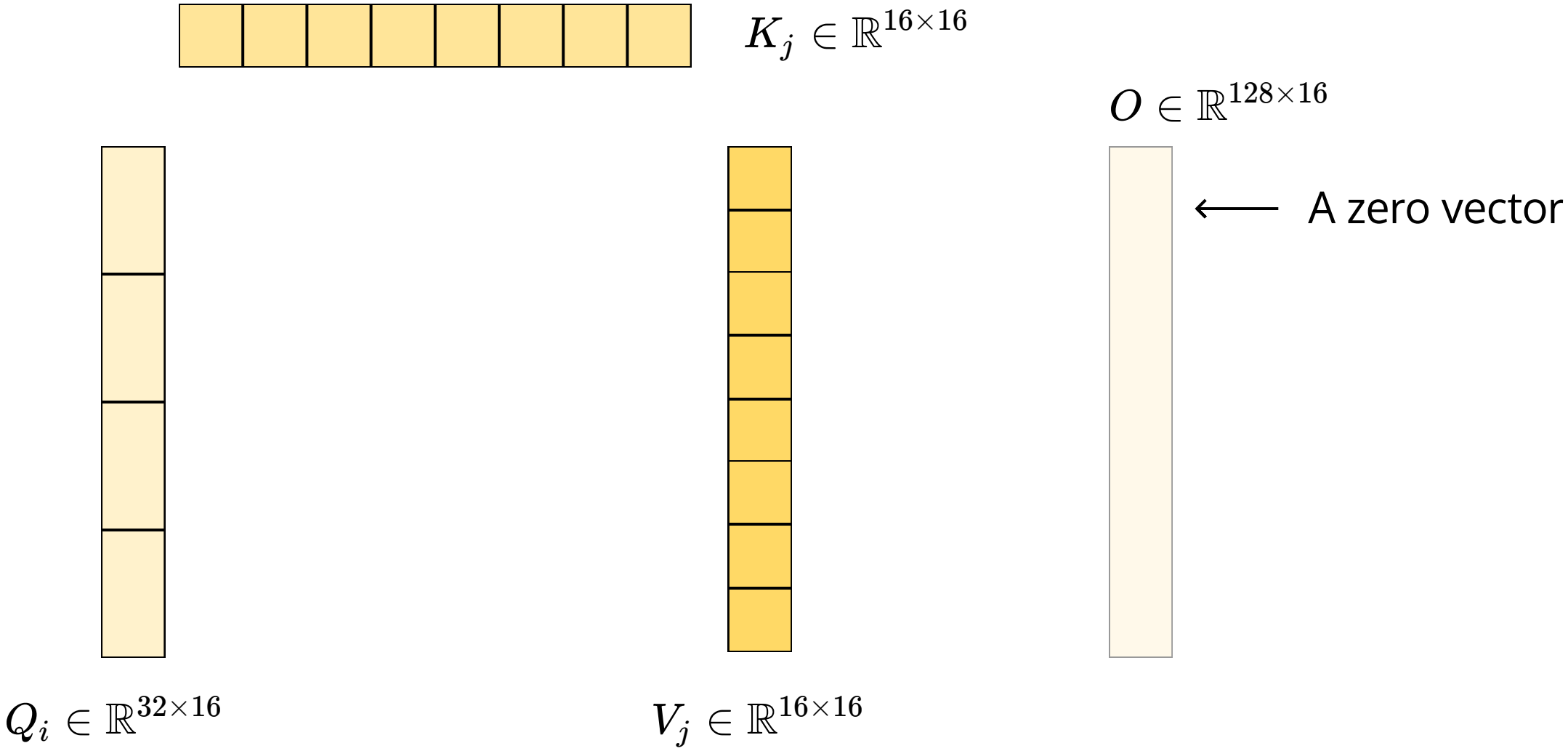
Write, $m_i \leftarrow m_i, l_i \leftarrow l_i$, to HBM

end for

end for

Return O

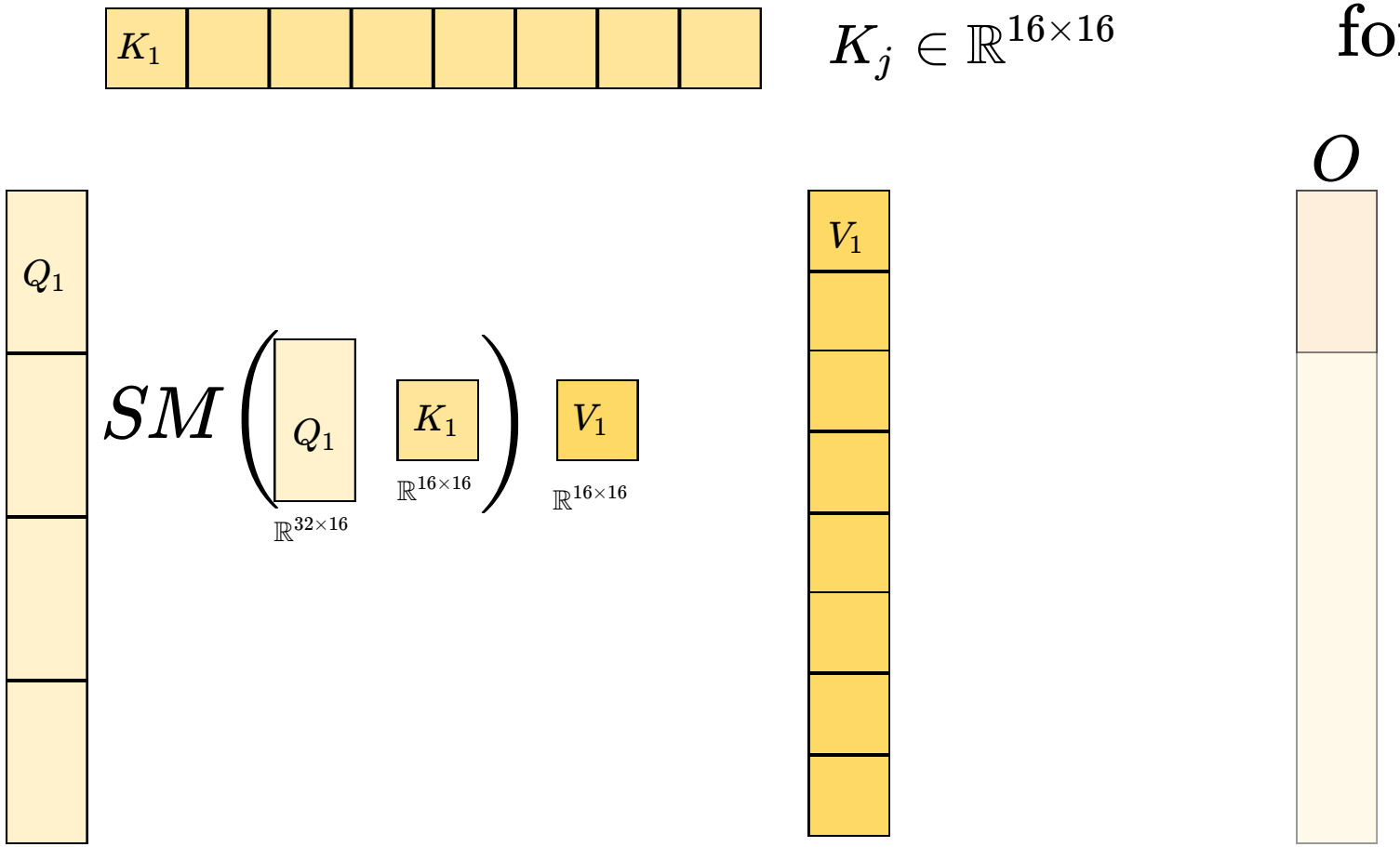
Suppose $N = 128, d = 16, B_c = 16, B_r = 32$



Suppose $N = 128, d = 16, B_c = 16, B_r = 32$

$$\text{for } j = 1$$

$$\text{for } i = 1$$



$K_j \in \mathbb{R}^{16\times16}$

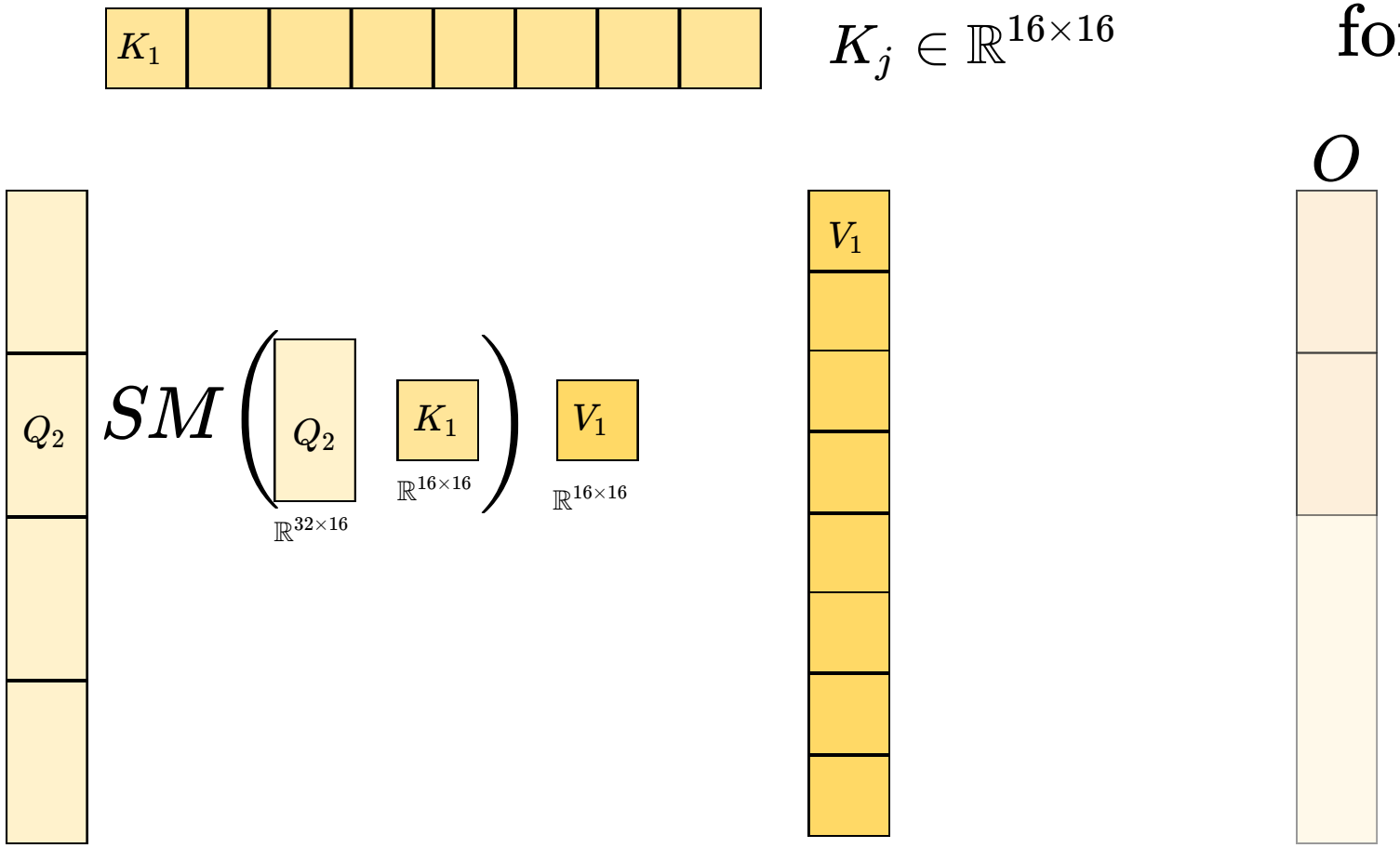
$Q_i \in \mathbb{R}^{32\times16}$

$V_j \in \mathbb{R}^{16\times16}$

Suppose $N = 128, d = 16, B_c = 16, B_r = 32$

$$\text{for } j = 1$$

$$\text{for } i = 2$$



$K_j \in \mathbb{R}^{16\times16}$

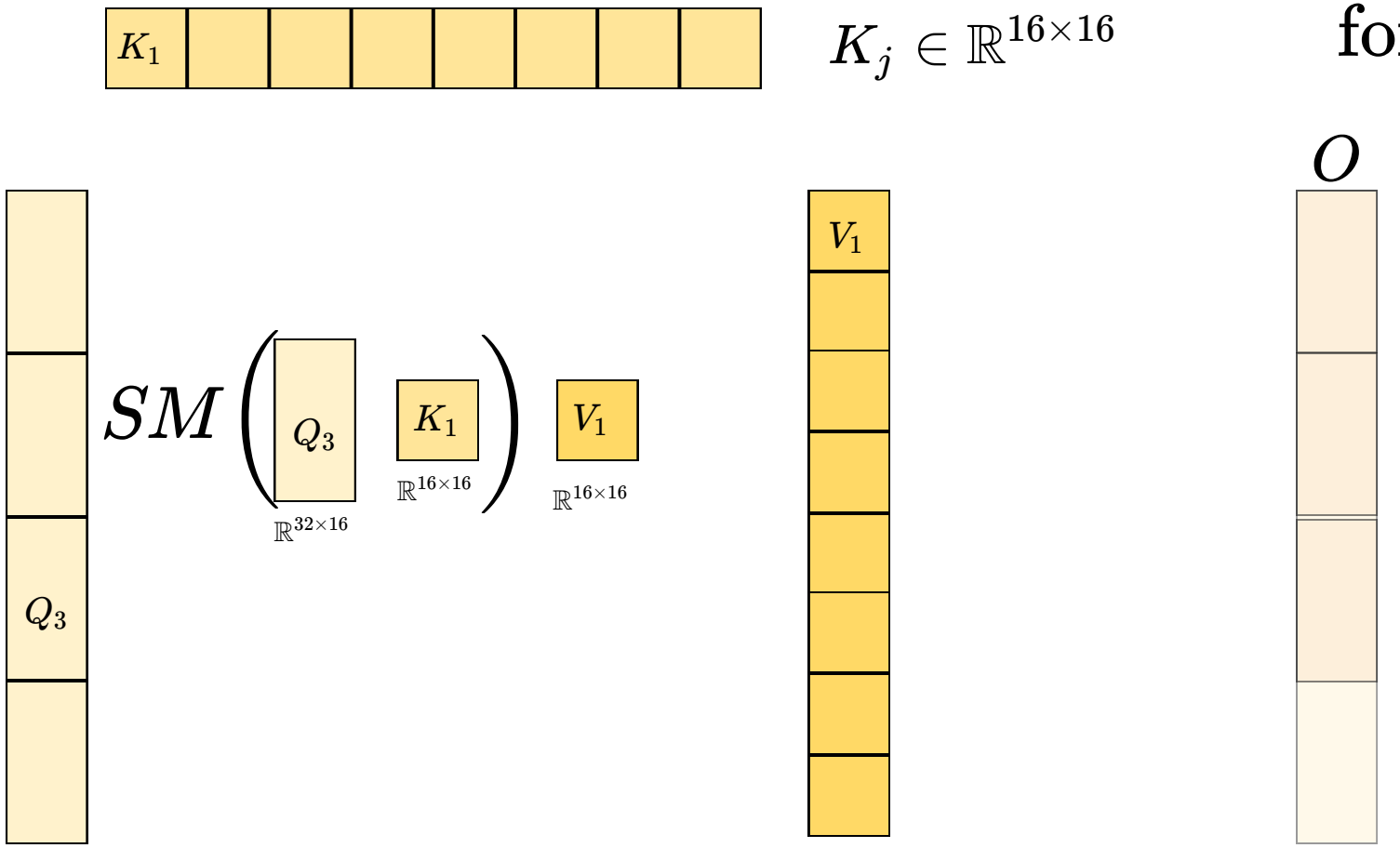
$Q_i \in \mathbb{R}^{32\times16}$

$V_j \in \mathbb{R}^{16\times16}$

Suppose $N = 128, d = 16, B_c = 16, B_r = 32$

$$\text{for } j = 1$$

$$\text{for } i = 3$$

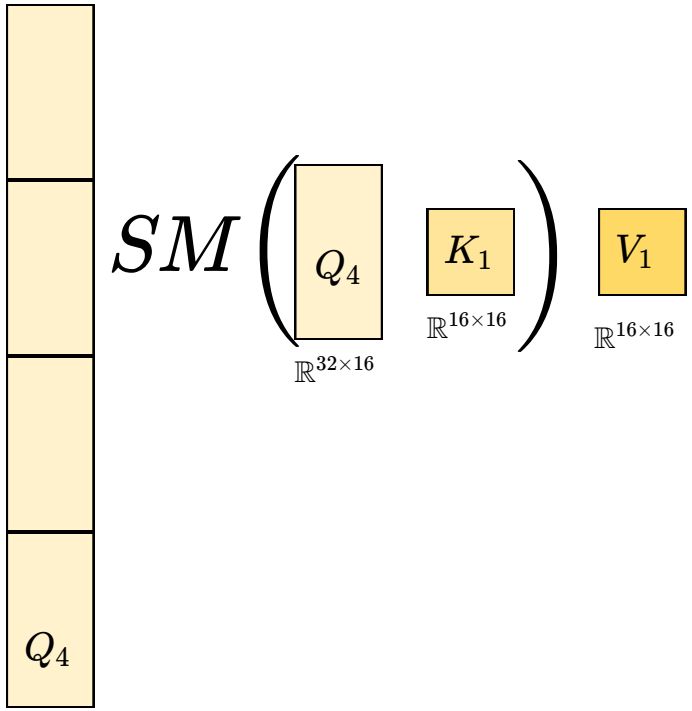


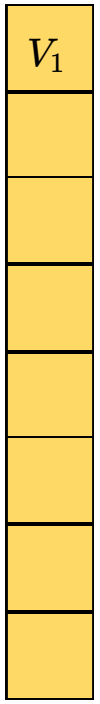
Suppose $N = 128, d = 16, B_c = 16, B_r = 32$

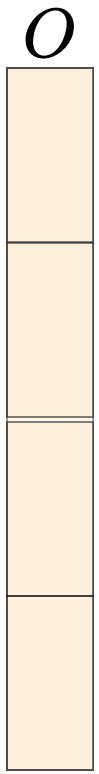
$$\text{for } j = 1$$

$$\text{for } i = 4$$

$K_j \in \mathbb{R}^{16\times16}$







Now we have m_1, l_1 computed from S_{11}

Next, increment the outer loop by 1 and compute m_2, l_2 from S_{22}

Iteratively update O_i's

$Q_i \in \mathbb{R}^{32\times16}$

$V_j \in \mathbb{R}^{16\times16}$

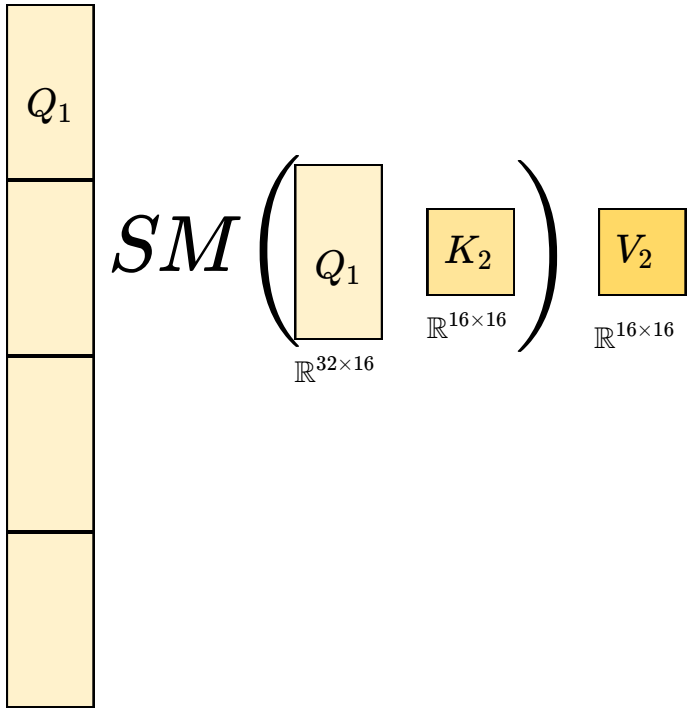
in HBM

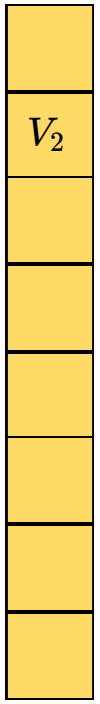
Suppose $N = 128, d = 16, B_c = 16, B_r = 32$

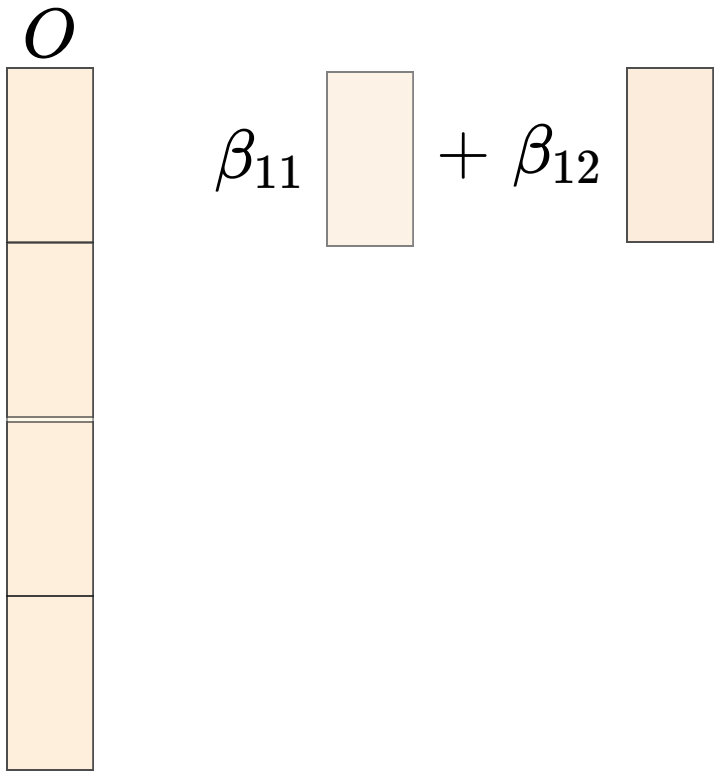
$$\text{for } j = 2$$

$$\text{for } i = 1$$

$K_j \in \mathbb{R}^{16\times16}$







$Q_i \in \mathbb{R}^{32\times16}$

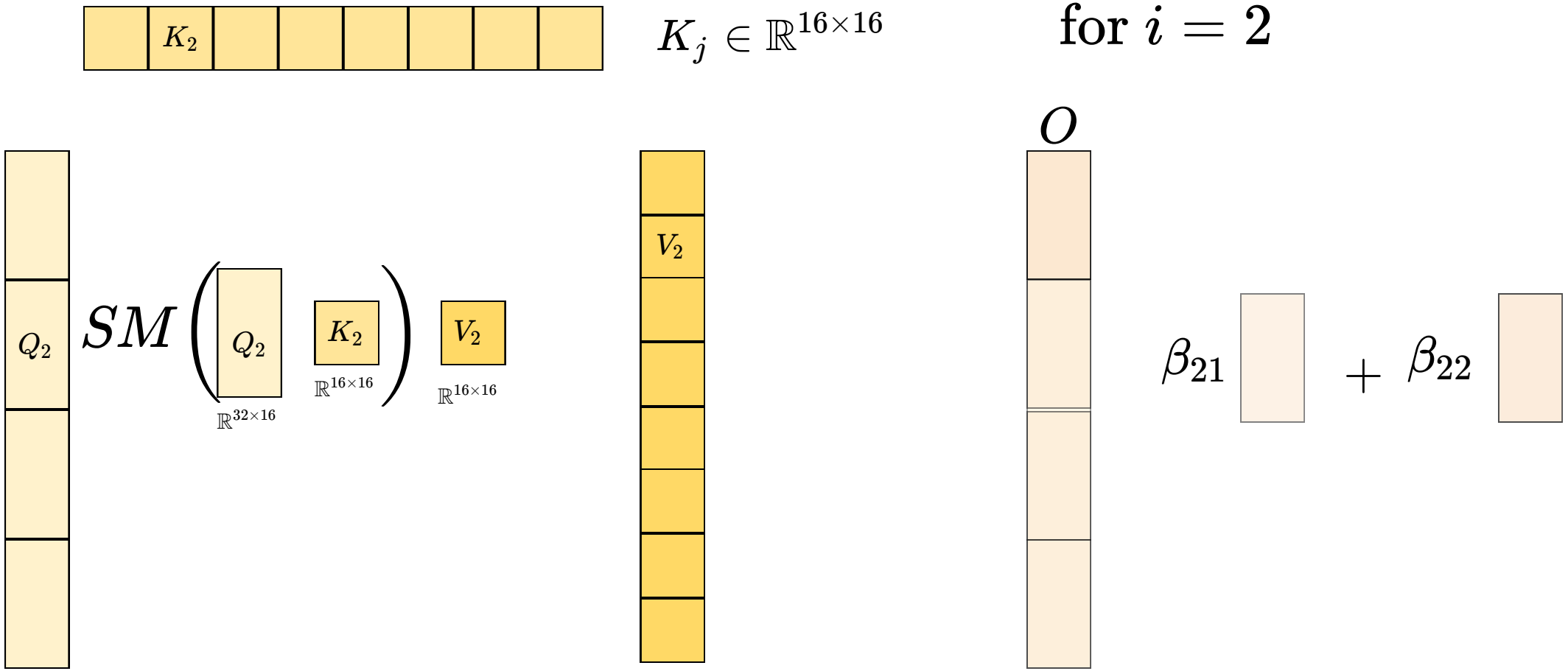
$V_j \in \mathbb{R}^{16\times16}$

in HBM **in SRAM**

Suppose $N = 128, d = 16, B_c = 16, B_r = 32$

$$\text{for } j = 2$$

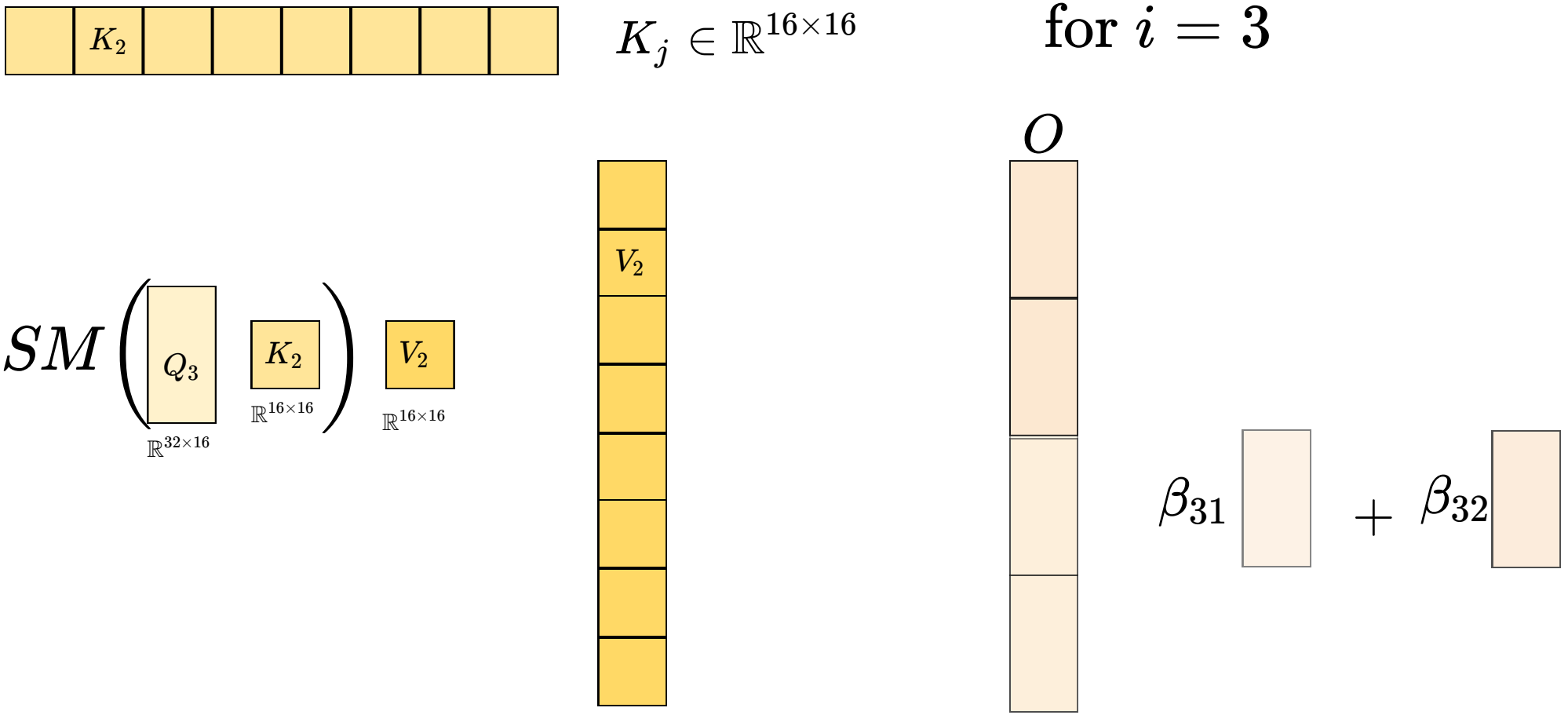
$$\text{for } i = 2$$



Suppose $N = 128, d = 16, B_c = 16, B_r = 32$

$$\text{for } j = 2$$

$$\text{for } i = 3$$

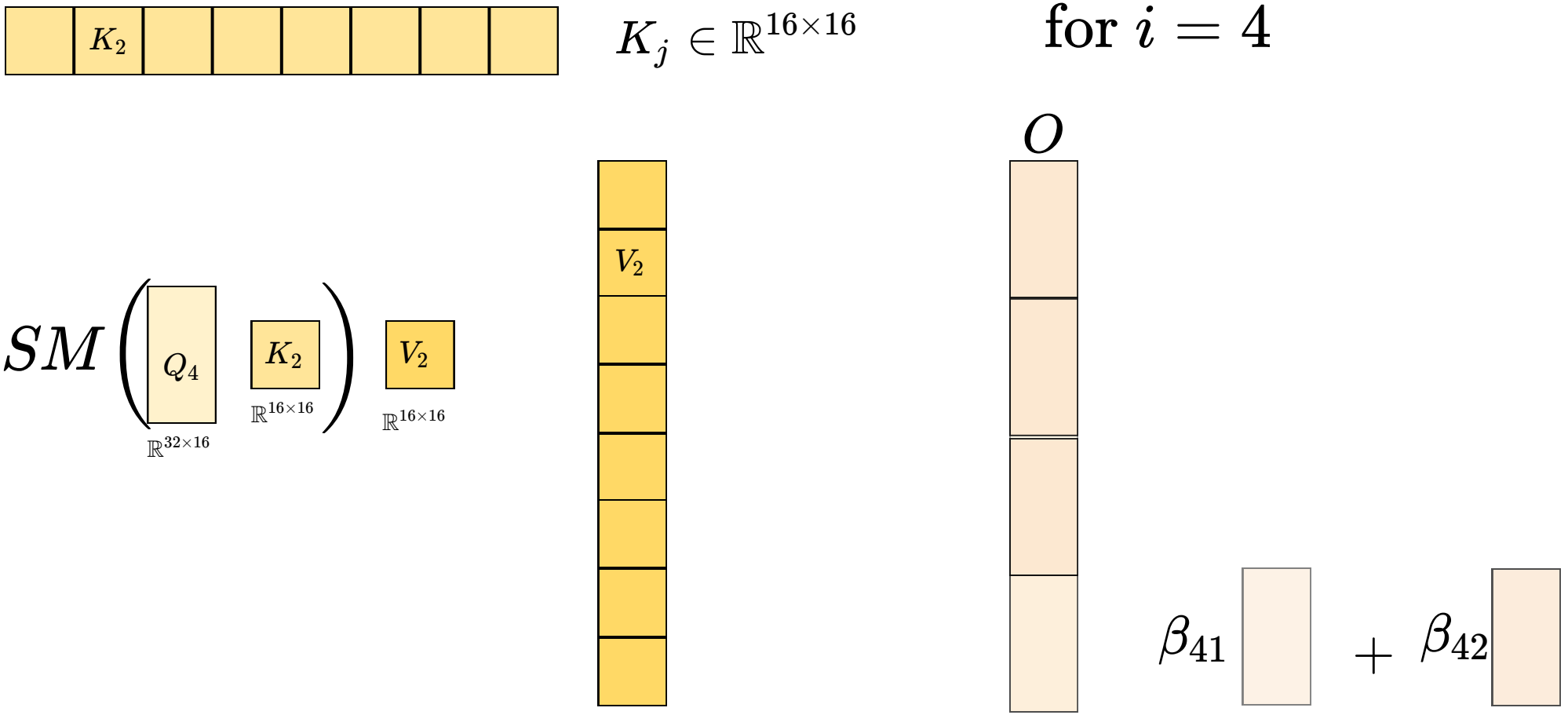


$Q_i \in \mathbb{R}^{32\times16}$ $V_j \in \mathbb{R}^{16\times16}$ in HBM in SRAM

Suppose $N = 128, d = 16, B_c = 16, B_r = 32$

$$\text{for } j = 2$$

$$\text{for } i = 4$$



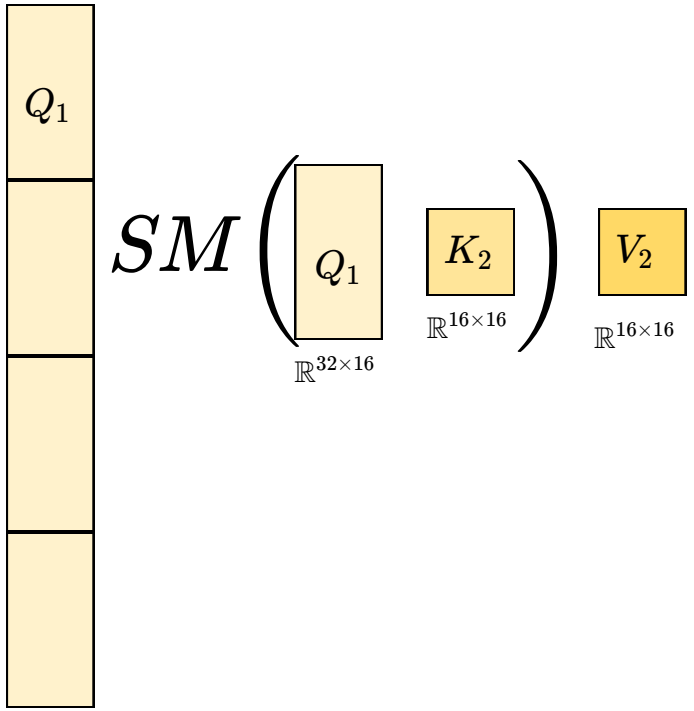
$Q_i \in \mathbb{R}^{32\times16}$ $V_j \in \mathbb{R}^{16\times16}$ **in HBM** **in SRAM**

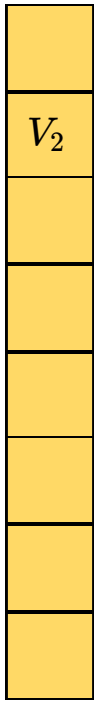
Suppose $N = 128, d = 16, B_c = 16, B_r = 32$

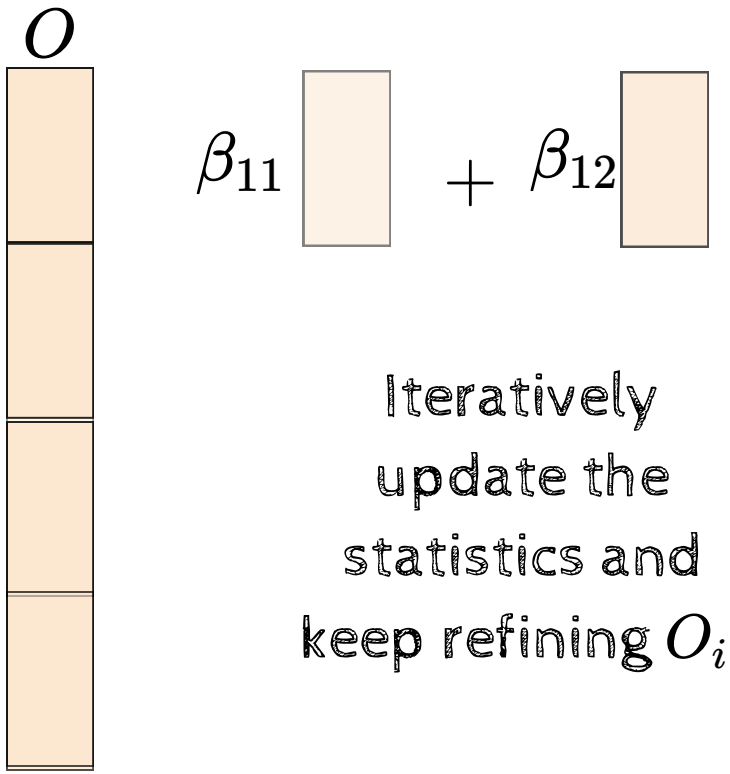
$$\text{for } j = 3$$

$$\text{for } i = 1$$

$K_j \in \mathbb{R}^{16\times16}$







Iteratively update the statistics and keep refining O_i

$Q_i \in \mathbb{R}^{32\times16}$

$V_j \in \mathbb{R}^{16\times16}$

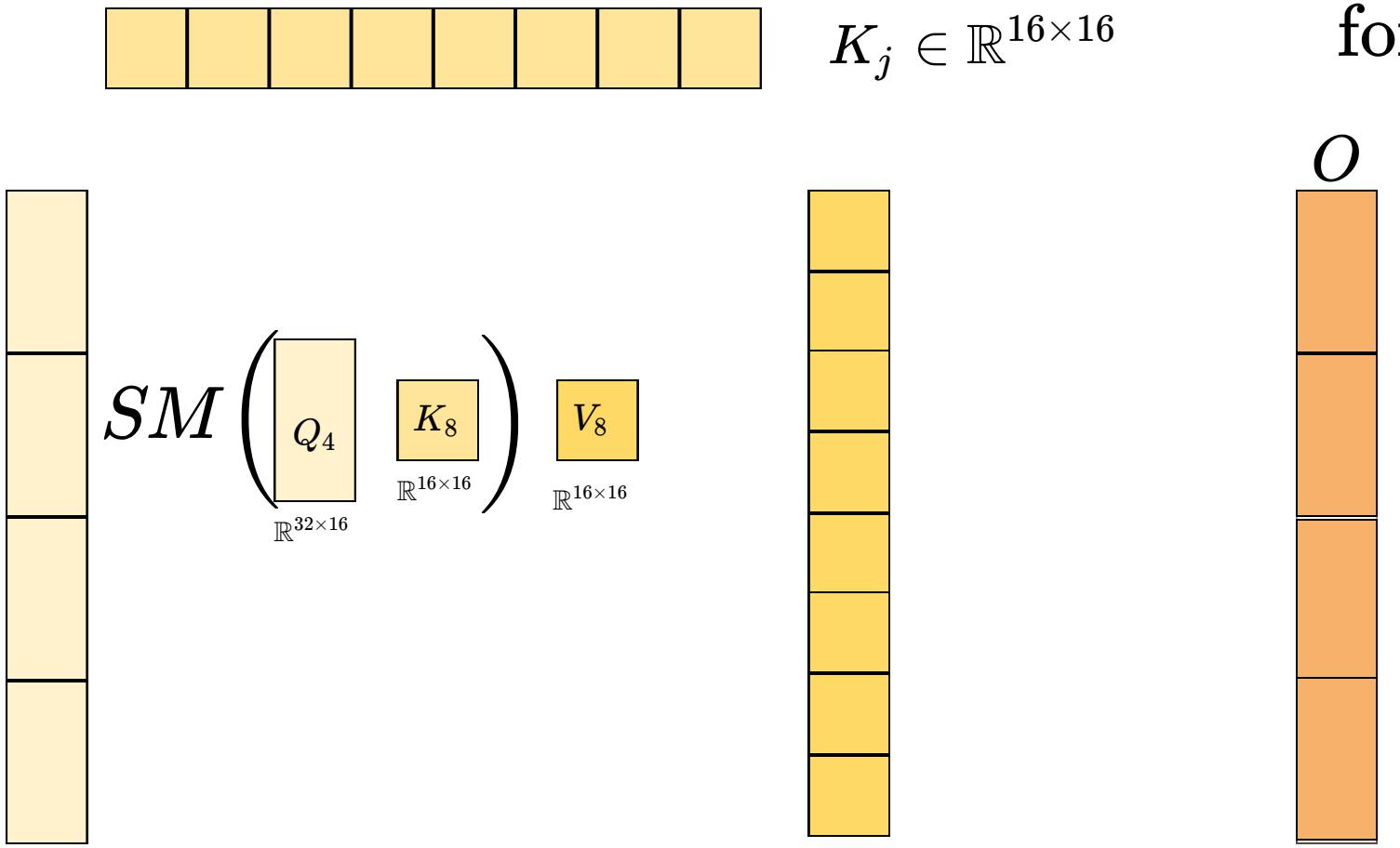
in HBM

in SRAM

Suppose $N = 128, d = 16, B_c = 16, B_r = 32$

$$\text{for } j = 8$$

$$\text{for } i = 4$$



Final output representation from a single head

in HBM

HBM Access

Standard attention requires $O(Nd + N^2)$ HBM accesses

FlashAttention requires $O(\frac{N^2 d^2}{M})$ HBM accesses.

Recomputation

We need to store $S = QK^T, P = softmax(S)$ for it to be used during backward propagation

However, kernel fusion does not store these intermediate quantities. Then how do we recover these?

We can recompute these quantities by using O and the statistics m, l

Did we achieve wall-clock speed up?

Model implementations	OpenWebText (ppl)	Training time (speedup)
GPT-2 small - Huggingface [87]	18.2	9.5 days (1.0×)
GPT-2 small - Megatron-LM [77]	18.2	4.7 days (2.0×)
GPT-2 small - FlashAttention	18.2	**2.7 days (3.5×)**
GPT-2 medium - Huggingface [87]	14.2	21.0 days (1.0×)
GPT-2 medium - Megatron-LM [77]	14.3	11.5 days (1.8×)
GPT-2 medium - FlashAttention	14.3	**6.9 days (3.0×)**

For the given compute budget, we can increase the context length from $1K$ to $4k$ using Flash Attention

Model implementations	Context length	OpenWebText (ppl)	Training time (speedup)
GPT-2 small - Megatron-LM	1k	18.2	4.7 days (1.0×)
GPT-2 small - FlashAttention	1k	18.2	**2.7 days (1.7×)**
GPT-2 small - FlashAttention	2k	17.6	3.0 days (1.6×)
GPT-2 small - FlashAttention	4k	**17.5**	3.6 days (1.3×)

There are many places where one could apply tiling and recomputation techniques to reduce the number of HBM accesses.

Could you think of one more common use case scenario where the operation is memory-bound? Hint: Not during training

Model Inference with autoregressive generation

We can not directly use Flash Attention (by parallelizing across queries and batch) in inference as we do not even have the whole Q, K, V matrices. It is gradually built for each time step.

However, recently a modified Flash Attention (that parallelizes across keys/values and sequence length) was adapted for inference. Take a look at Flash Decoding.

So, we need a different mechanism to reduce the number of accesses to memory

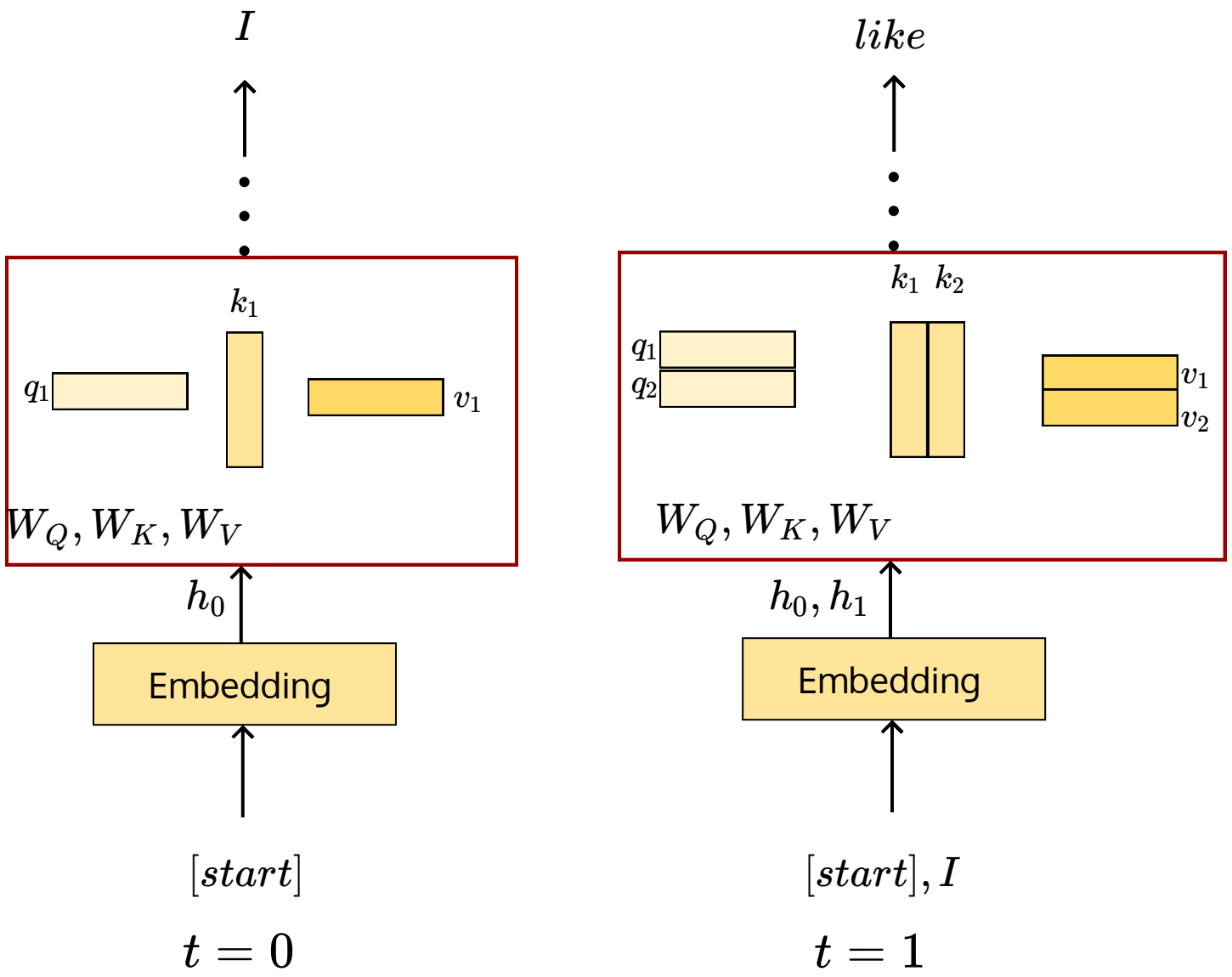
The methods for inference optimization by reducing memory access (CPU/GPU) were proposed years before the introduction of Flash Attention!

Module 5: Fast Inference

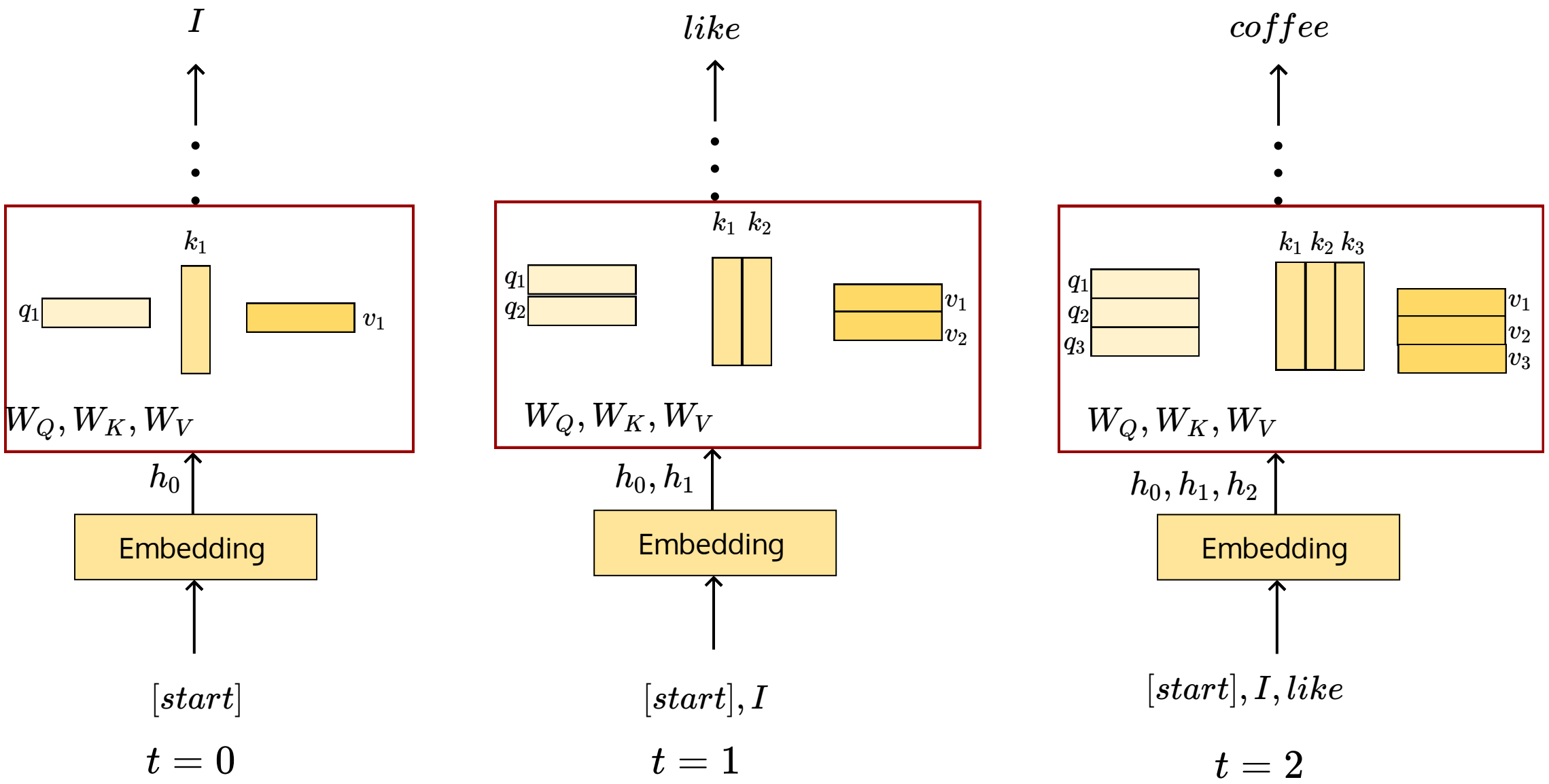
Mitesh M. Khapra

AI4Bharat, Department of Computer Science and Engineering, IIT Madras

Let's take a closer look at the naive implementation of autoregressive inference and see why it matters to optimize the inference pipeline

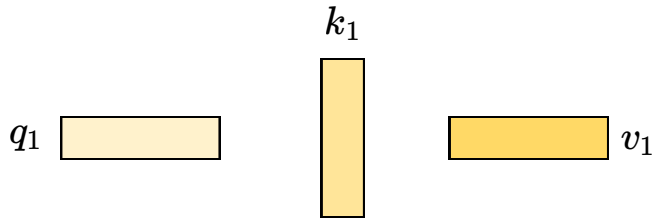


At each time step, we compute the keys-values for all the previous queries as well. This is a waste of compute and moreover increases the latency. Moreover, processing an LLM request can be 10x more expensive than a traditional keyword query. Can we do better?



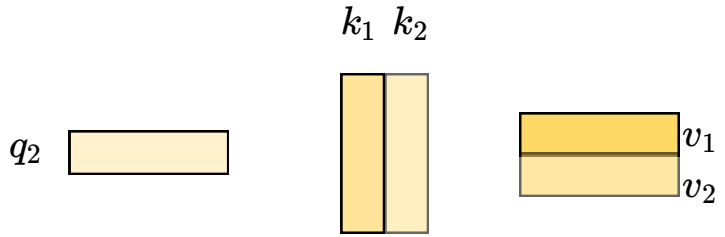
KV Caching

In the first time step, we start with a query, compute its key-value pair for the given input token when running the model in autoregressive mode.

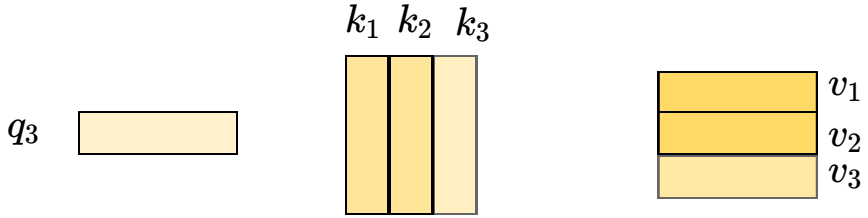


Store the key value in cache (usually GPU RAM which has HBM).

For the new query q_2 in time step 2, we reload k_1, v_1 from HBM and compute k_2, v_2



For the new query q_3 in time step 3, we reload keys and values k_1, v_1, k_2, v_2 from previous times steps from HBM and compute k_3, v_3



In this way, we trade off the memory for compute and the compute scales linearly $O(n)$

Since KV-caching uses GPU RAM where the model weights are stored, we cannot let the memory for KV-cache to grow indefinitely!

Let 's take an example.

KV cache per token (in B) =

$2 \times$ num_layers $\times$ (num_heads $\times$ dim_head) $\times$ precision_in_bytes

Model GPT-3

- num_layers = 96
- num_heads = 96
- dim_head = 128
- precision_in_bytes=4

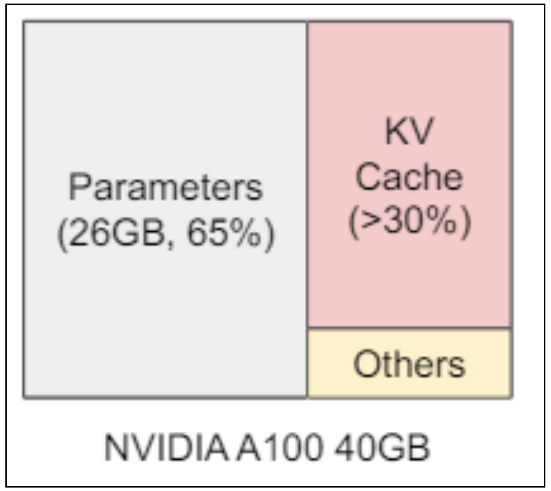
KV cache per token $= 2 \times 96 \times (96 \times 128) \times 4$

$= 9.4$ MB

For 2K context length $= 2048 \times 9.4$ MB

$= 19.3$ GB

to generate a sequence of length 2K requires 19.3 GB of memory for KV caching alone (we can't even use an A100 40 GB GPU).



A practical solution is to drop the keys and values of tokens that occured in the distant past (act similar to local windowed attention).

The other approach is to introduce model level modifications!

Multi-Query Attention (MQA)

Select one hyper-parameter out of three hyperparameters that we could exploit in the formula to reduce the memory requirement is _____

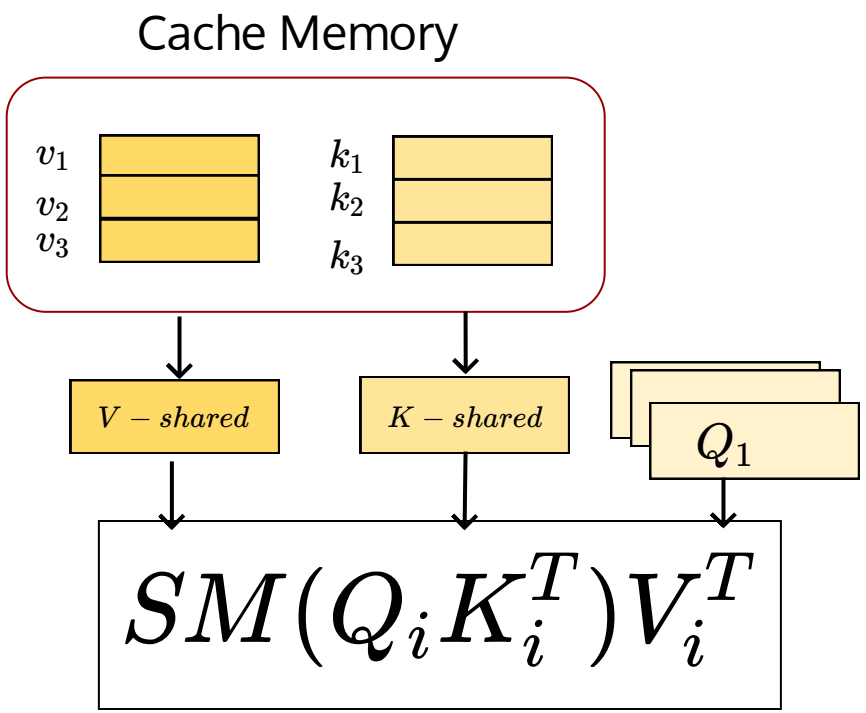
KV cache per token (in B) =
2 × num_layers × (num_heads × dim_head) × precision_in_bytes

Let's set $num_heads = 1$ (it does not imply we use only one head). The number 2 in the formula means that we access two vectors (key-value) for each head.

Therefore, it means we want to access key-value pairs once for all the heads!

How can we do that?

Share all the key-value pairs across the heads.



This greatly reduces the cache memory. However, the performance degrades significantly.

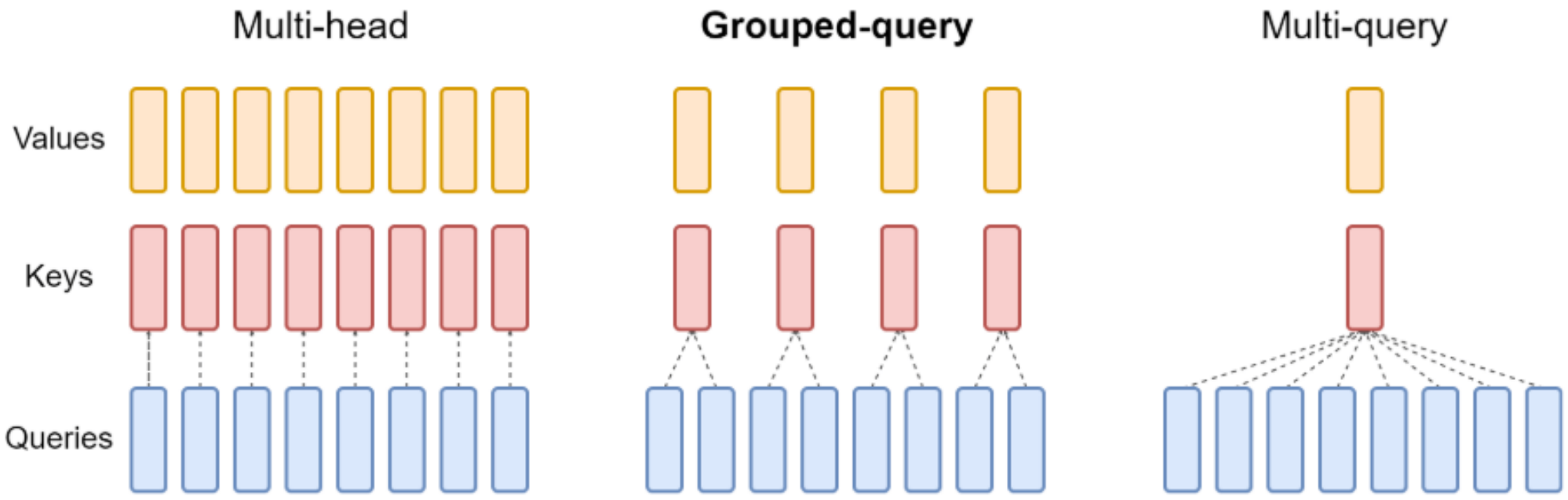
Moreover, this requires us to re-train the model with at least 5% of training data

Grouped-Query Attention (GQA)

On one extreme we have Multi-Head Attention where each head has a separate query, key and value vector.

On the other extreme we have Multi-Query Attention where each head has separate query but keys and values are shared across all heads.

Grouped Query Attention is just a middle ground between the two, where a group of queries share key-value pairs that result in performance close to MHA

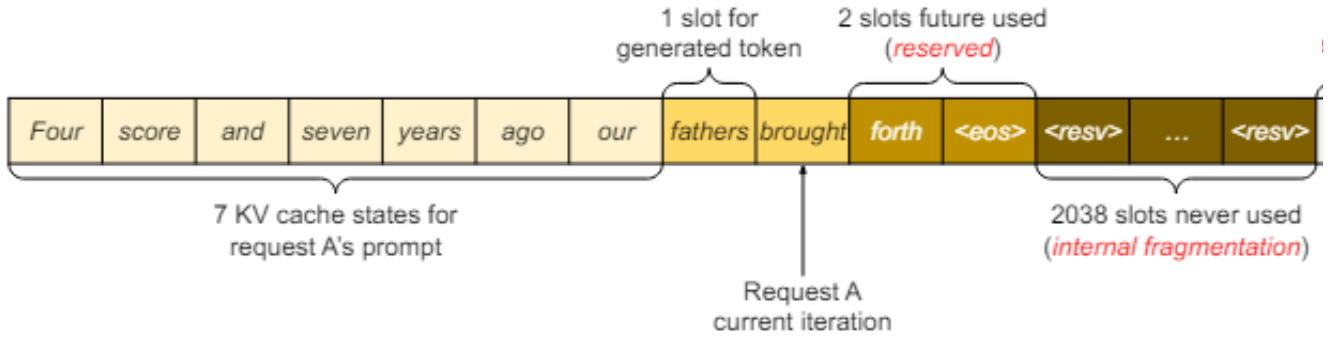


Paged Attention (PA)

Typically, fixed and contiguous memory is reserved for KV-caching (say, based on context length of the model).

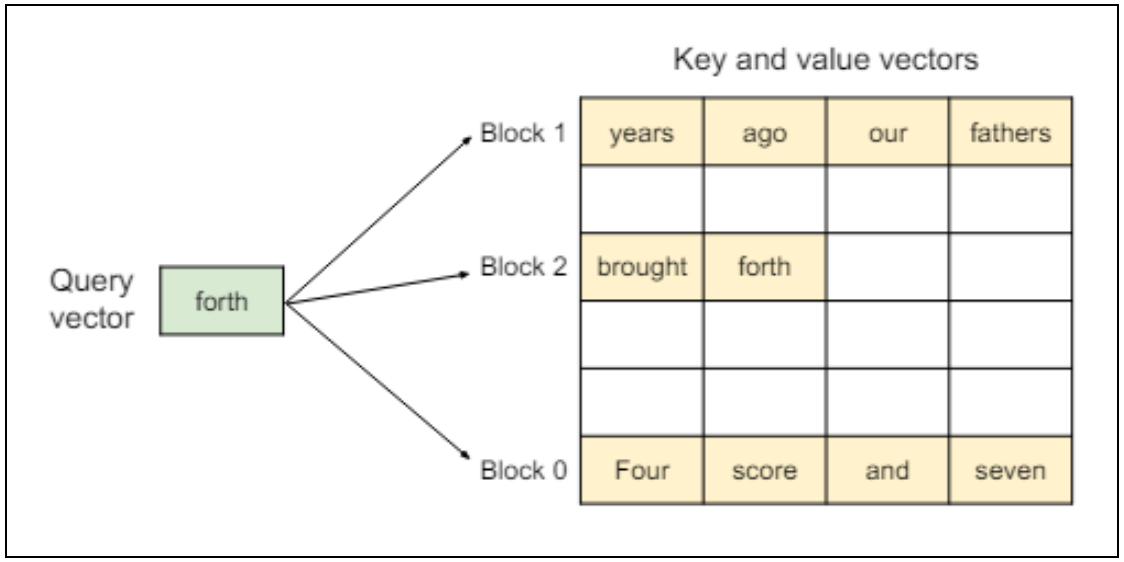
However, we do not know the length of the sequence that the model generates a priori.

The memory during the generation process is wasted if the model generated a sequence that is much smaller than the pre-fixed length.



A solution is to use a block of non-contiguous memory locations.

That is what Paged Attention does



References

Efficient Transformers : A survey

Generating Wikipedia by Summarizing Long Sequences

BigBird: Transformers for Longer Sequences

Longformer: The Long-Document Transformer

Reformer: The Efficient Transformer

Blog: A deep dive into reformer

LONG RANGE ARENA: A Benchmark for efficient transformers

Flash Attention: Fast and Memory Efficient Exact attention

Fast transformer decoding: One-write head is all you need

GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints

Efficient Memory Management for Large Language Model Serving with PagedAttention